
IT5429E-2-25-2 (25.2B01)

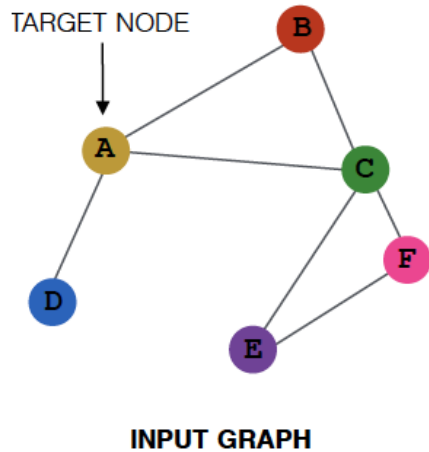
Graph Analytics for Big Data

Lecture 8: Graph Transformer

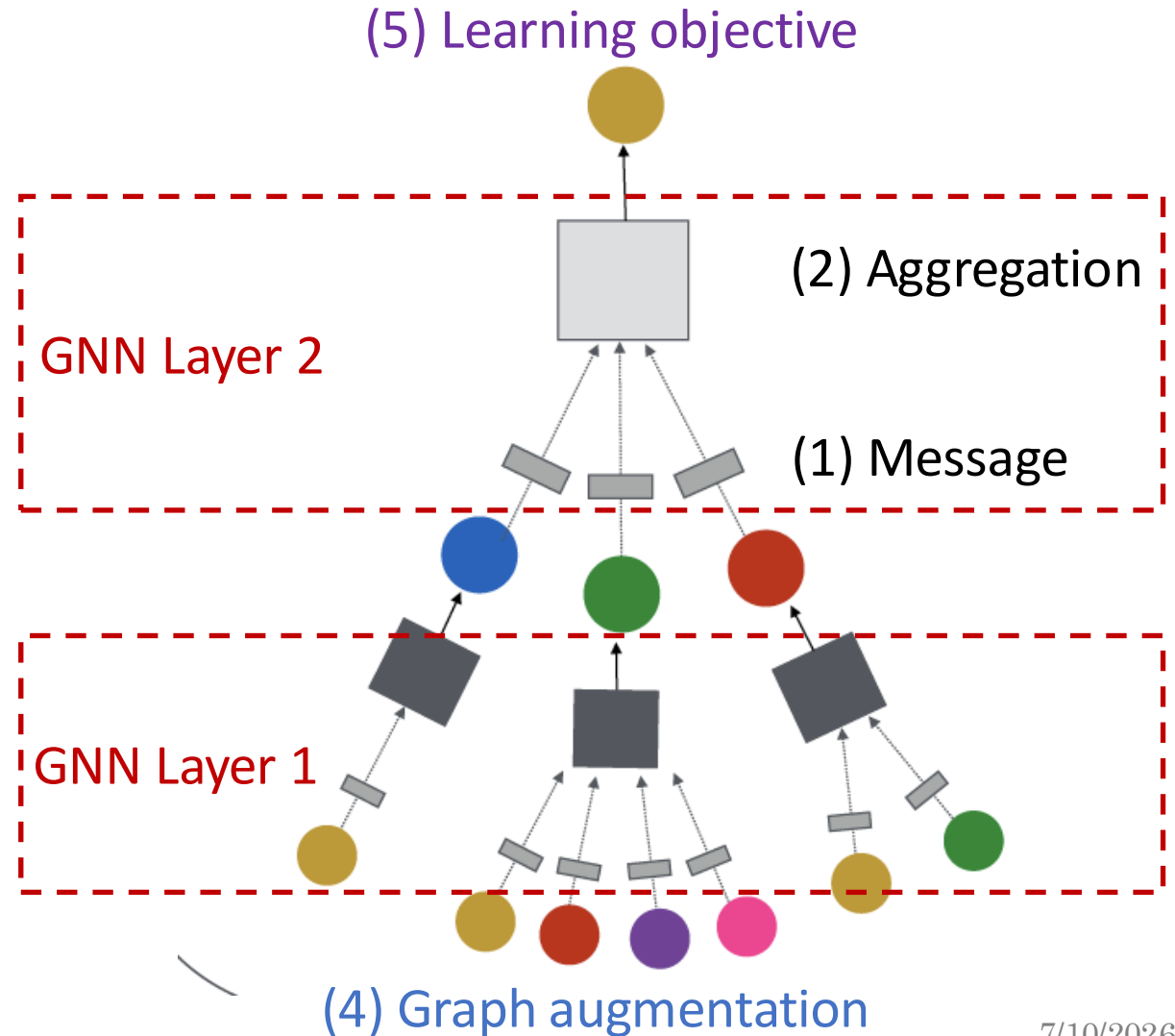
Many slides are adapted from <https://web.stanford.edu/class/cs224w/>



Recap: A General GNN Framework

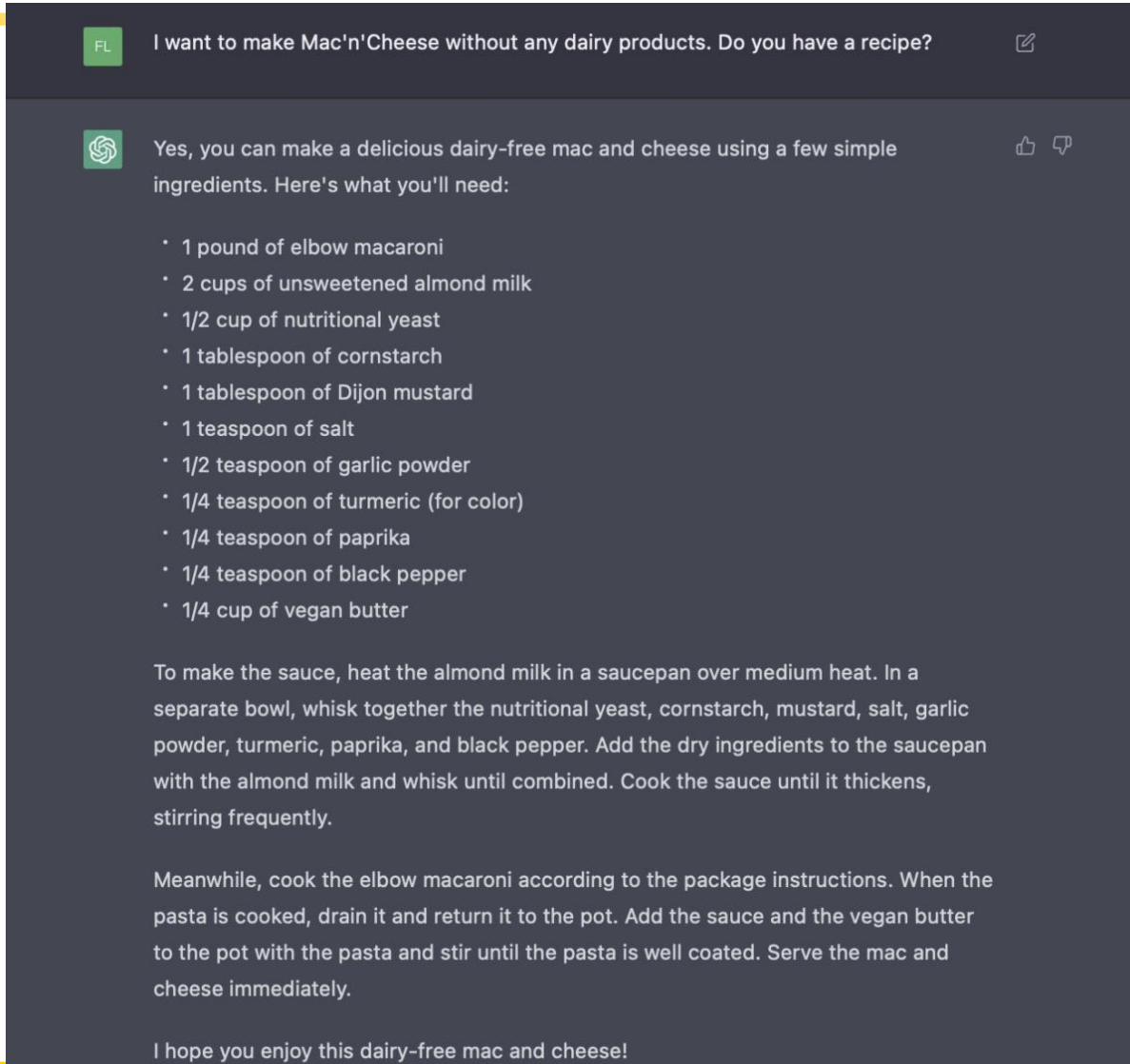


(3) Layer connectivity



Transformers:

The Backbone of Generative AI



FL I want to make Mac'n'Cheese without any dairy products. Do you have a recipe? 

 Yes, you can make a delicious dairy-free mac and cheese using a few simple ingredients. Here's what you'll need:  

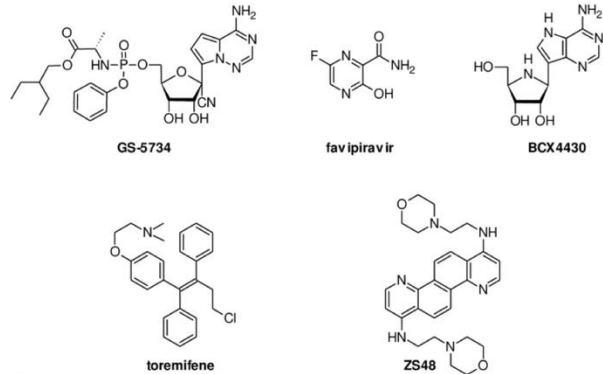
- 1 pound of elbow macaroni
- 2 cups of unsweetened almond milk
- 1/2 cup of nutritional yeast
- 1 tablespoon of cornstarch
- 1 tablespoon of Dijon mustard
- 1 teaspoon of salt
- 1/2 teaspoon of garlic powder
- 1/4 teaspoon of turmeric (for color)
- 1/4 teaspoon of paprika
- 1/4 teaspoon of black pepper
- 1/4 cup of vegan butter

To make the sauce, heat the almond milk in a saucepan over medium heat. In a separate bowl, whisk together the nutritional yeast, cornstarch, mustard, salt, garlic powder, turmeric, paprika, and black pepper. Add the dry ingredients to the saucepan with the almond milk and whisk until combined. Cook the sauce until it thickens, stirring frequently.

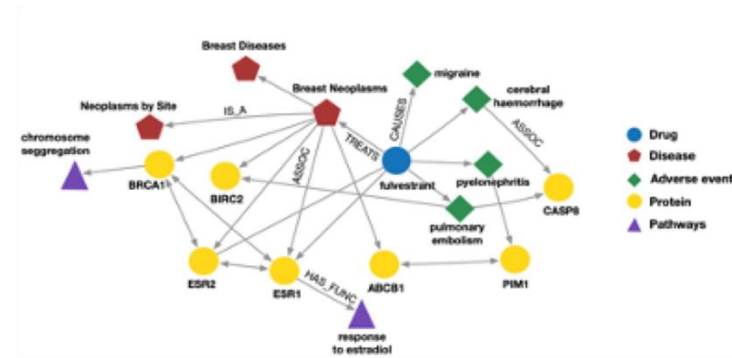
Meanwhile, cook the elbow macaroni according to the package instructions. When the pasta is cooked, drain it and return it to the pot. Add the sauce and the vegan butter to the pot with the pasta and stir until the pasta is well coated. Serve the mac and cheese immediately.

I hope you enjoy this dairy-free mac and cheese!

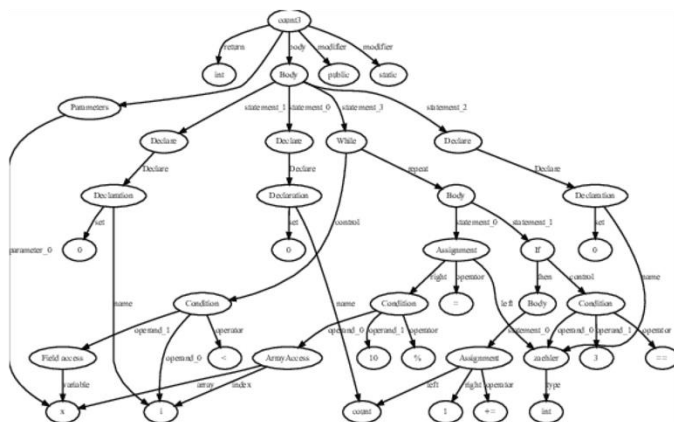
Goal: Bring Power of Transformer to Graphs



Small molecules



Knowledge graphs



Code graphs

Image credit: ResearchGate

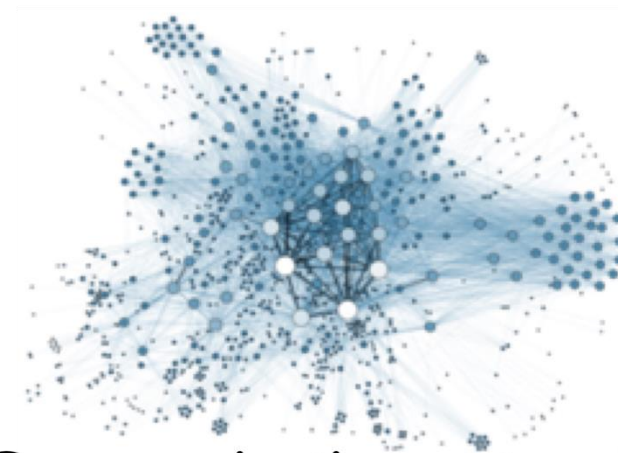


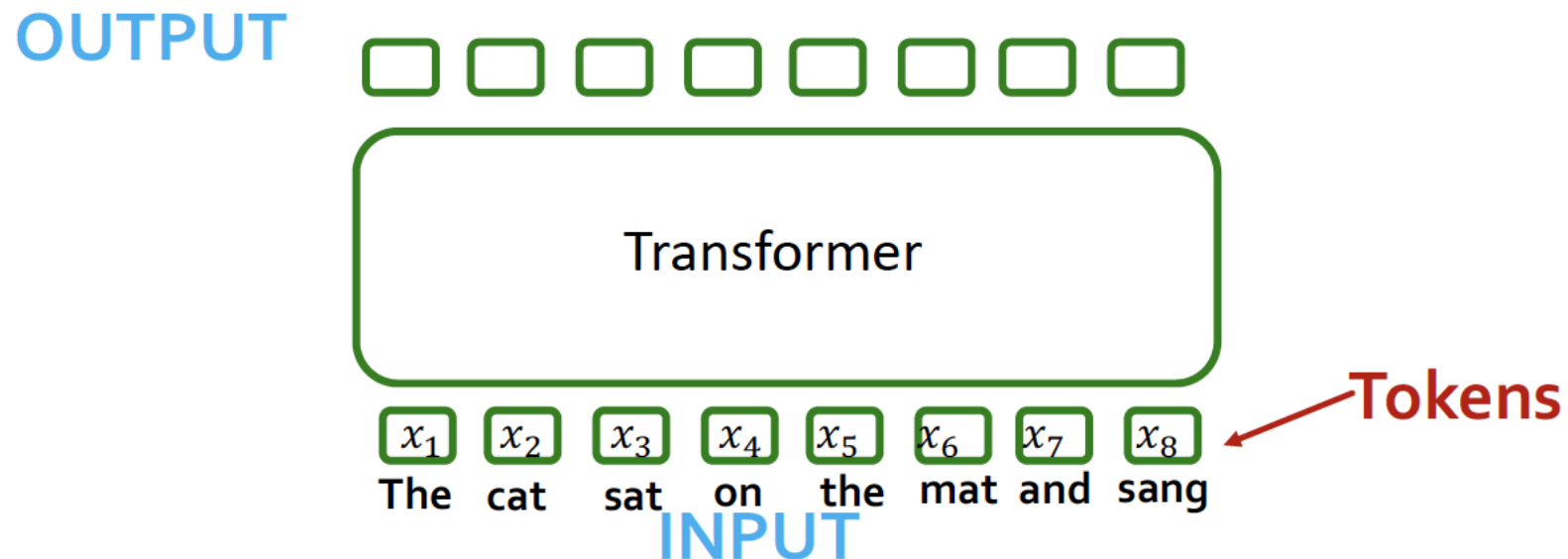
Image credit: Lumen Learning

Outline

- Part 1:
 - Introducing Transformers
 - Relation to message passing GNNs
- Part 2:
 - A new design landscape for graph Transformers
- Part 3:
 - Sign invariant Laplacian positional encodings for graph Transformers

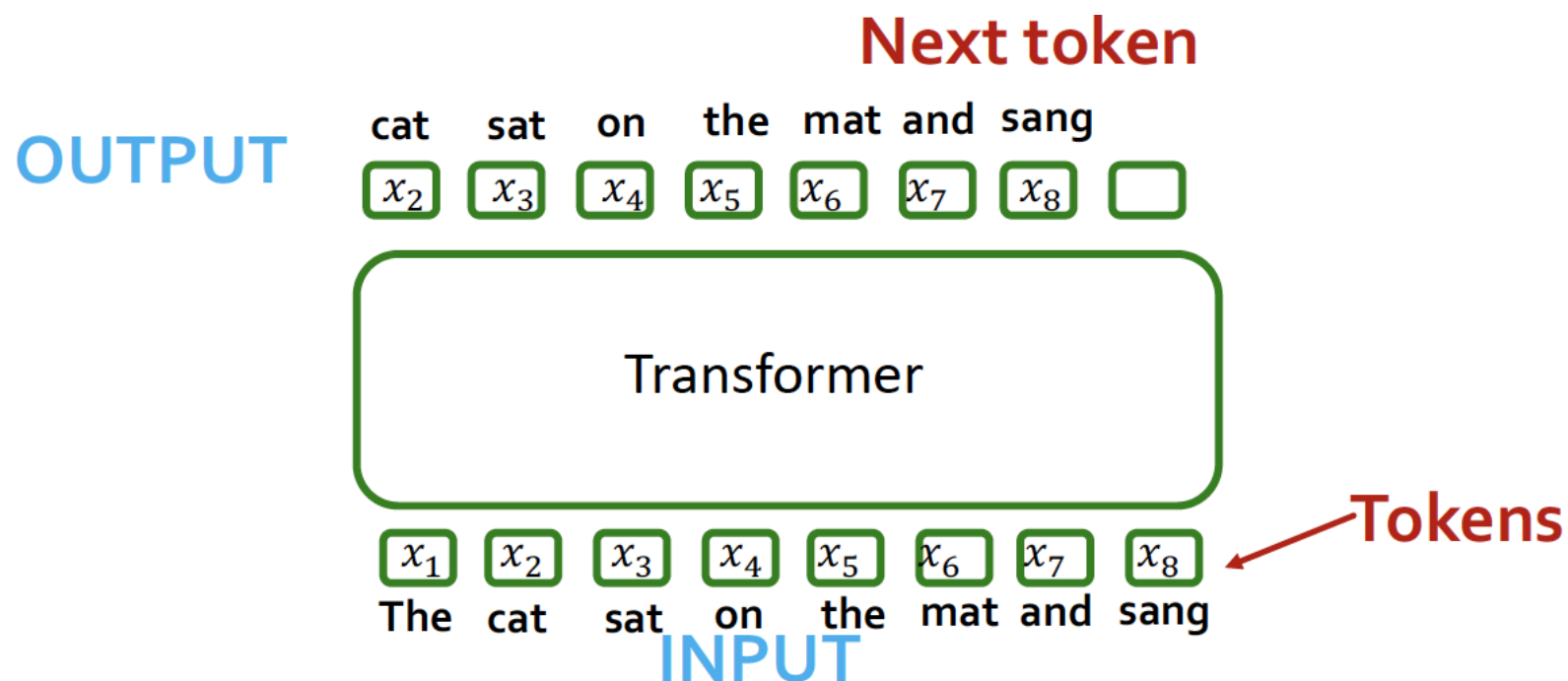
Transformer Ingest Tokens

- Transformers map 1D sequences of vectors to 1D sequences of vectors



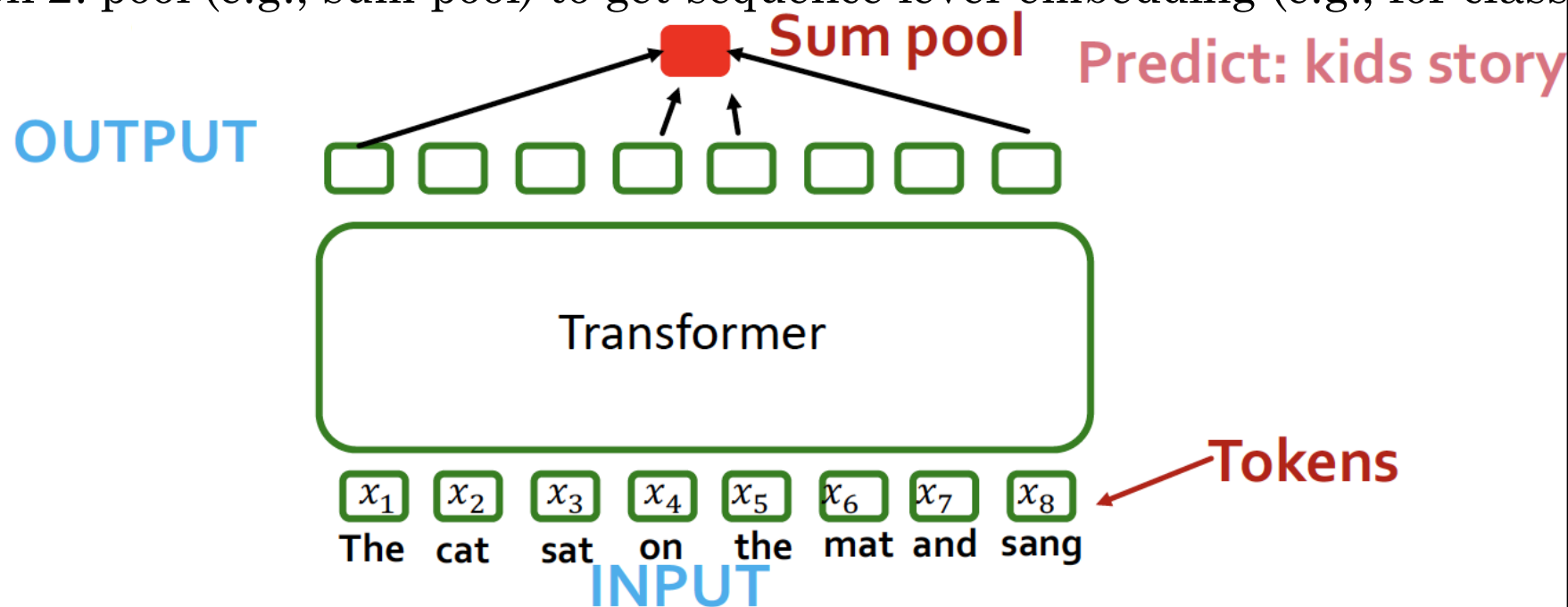
Transformer Ingest Tokens

- Transformers map 1D sequences of vectors to 1D sequences of vectors
 - Tokens describe a "piece" of data – e.g., a word
- What output sequence?
 - Option 1: next token => GPT



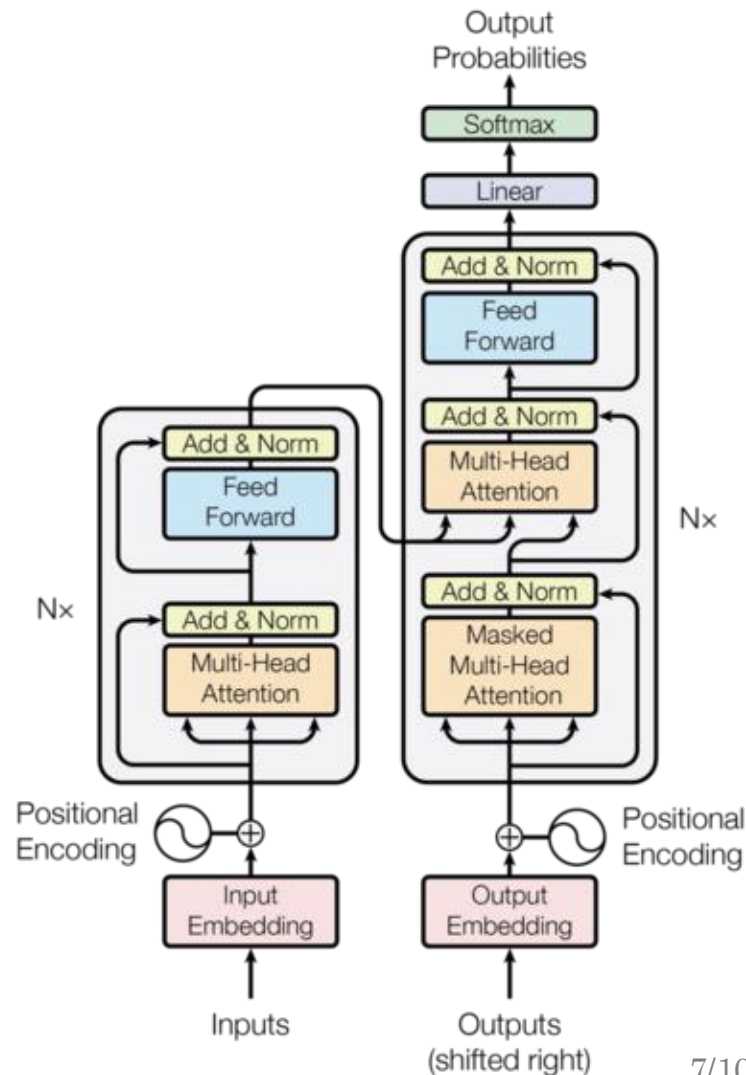
Transformer Ingest Tokens

- Transformers map 1D sequences of vectors to 1D sequences of vectors
 - Tokens describe a "piece" of data – e.g., a word
- What output sequence?
 - Option 1: next token => GPT
 - Option 2: pool (e.g., sum-pool) to get sequence level-embedding (e.g., for classification task)



Transformer Blueprint

- How are tokens processed?
- Lots of components
 - Normalization
 - Feed forward networks
 - Positional encoding
 - Multi-head self-attention
- What does self-attention block do?



Self-Attention

- Intuition

- As the model processes each word, **self attention** allows it to look at other positions in the input sequence for clues that can help lead to a better encoding for this word.

- Example:

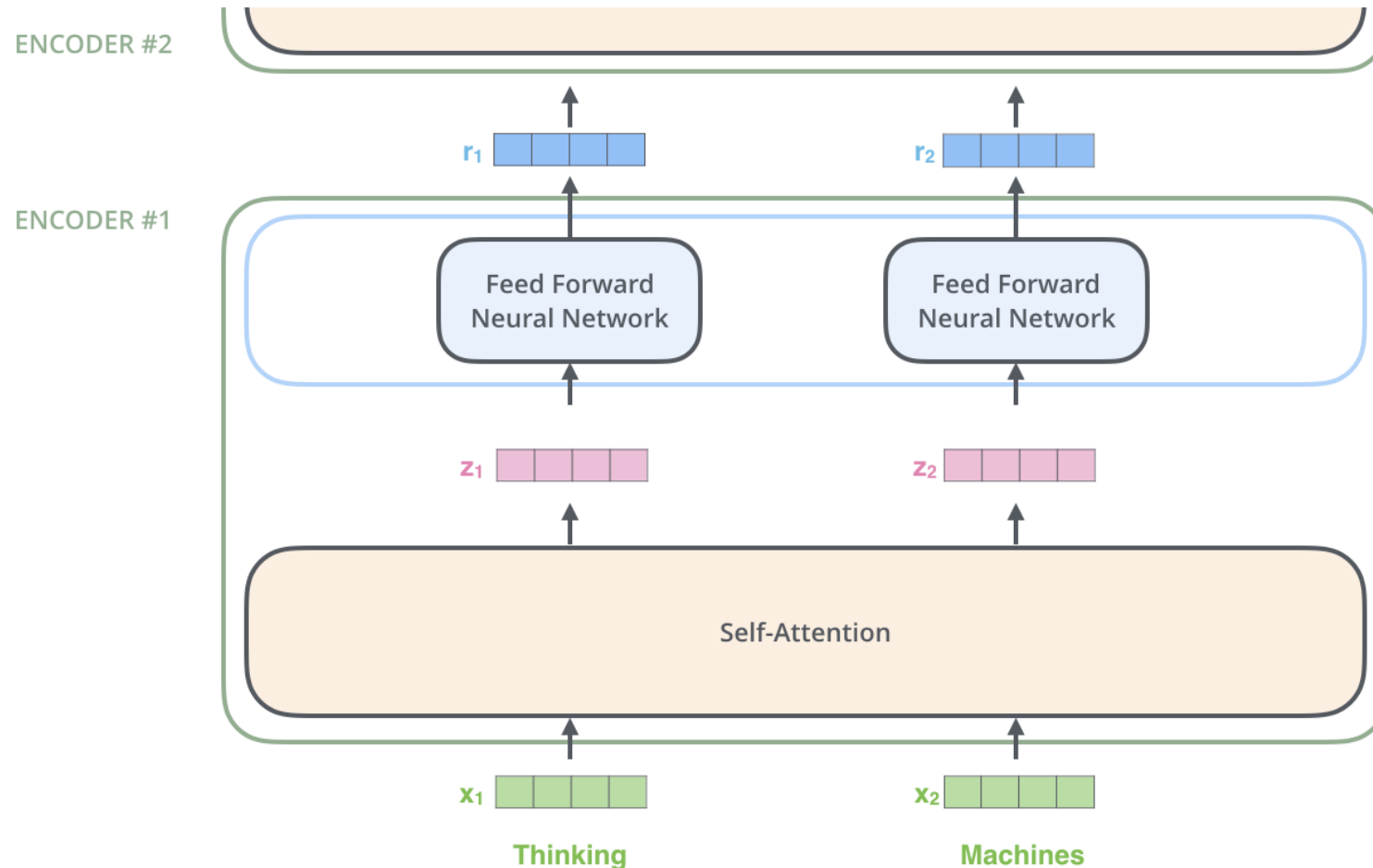
The animal didn't cross the street because it was too tired.



- When a model processes the word “it”, **self-attention** allows it to associate “it” with “animal”.

Self-Attention

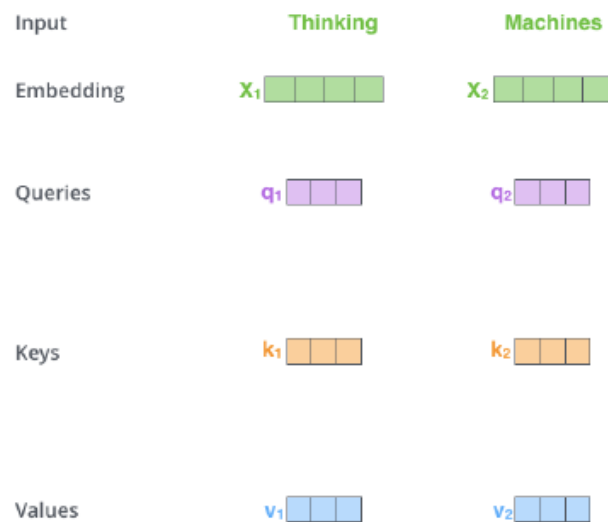
- Before “multi-head” self-attention, what is “single head” self-attention?



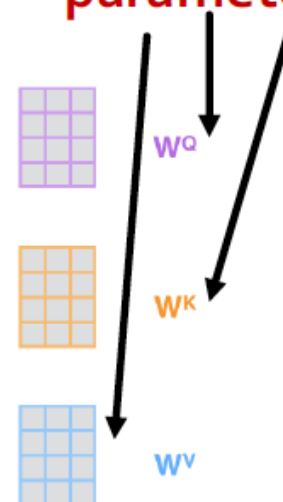
Self-Attention

- **Step 1:** compute “key, value, query” for each input

Step 1



Model parameters

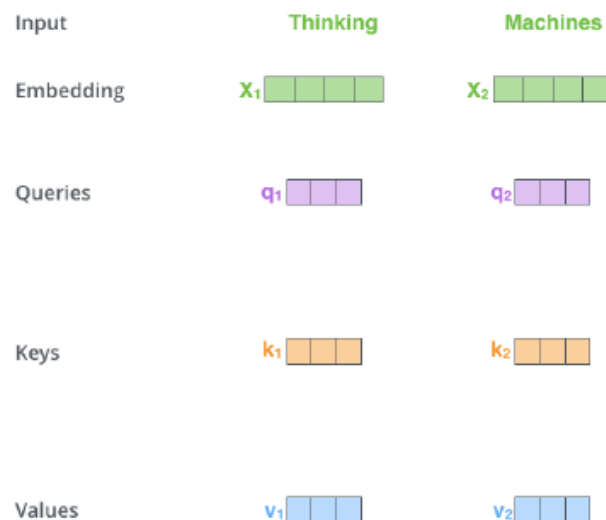


Self-Attention

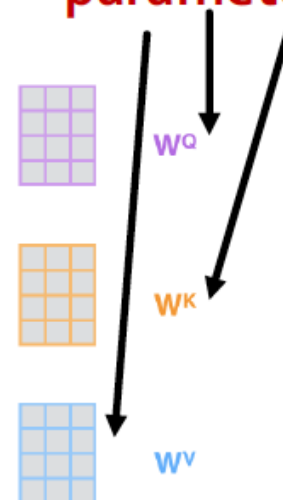
- Step 1: compute “key, value, query” for each input

- Step 2 (just for x_1): compute scores between pairs, turn into probabilities (same for x_2)

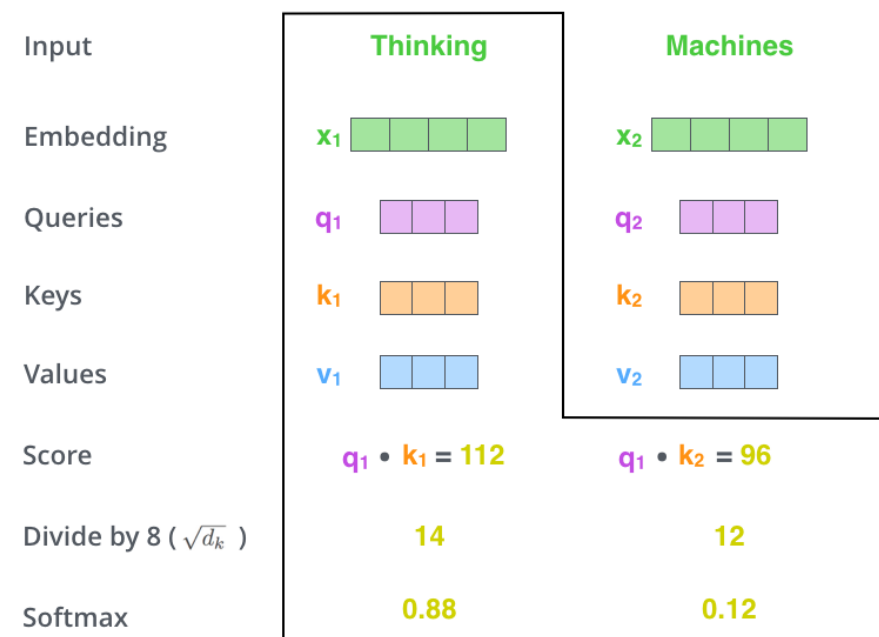
Step 1



Model parameters



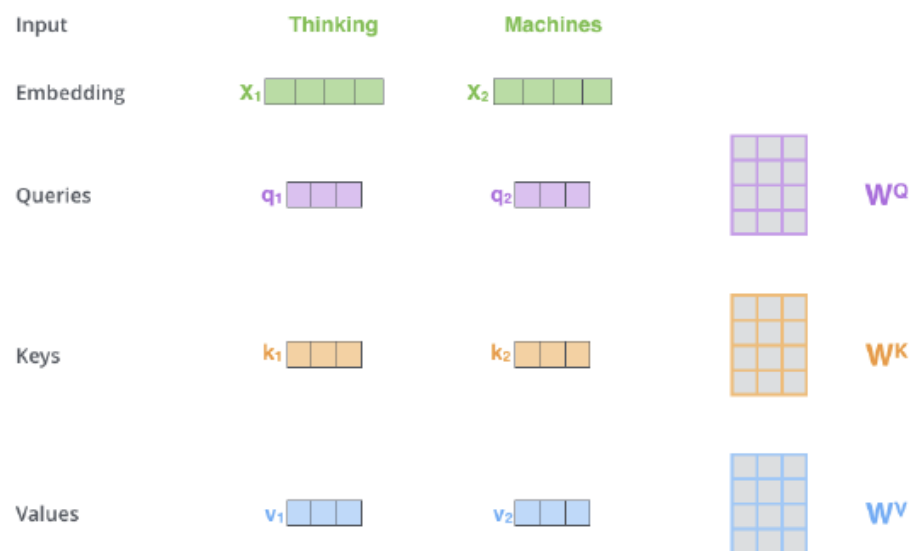
Step 2



Self-Attention

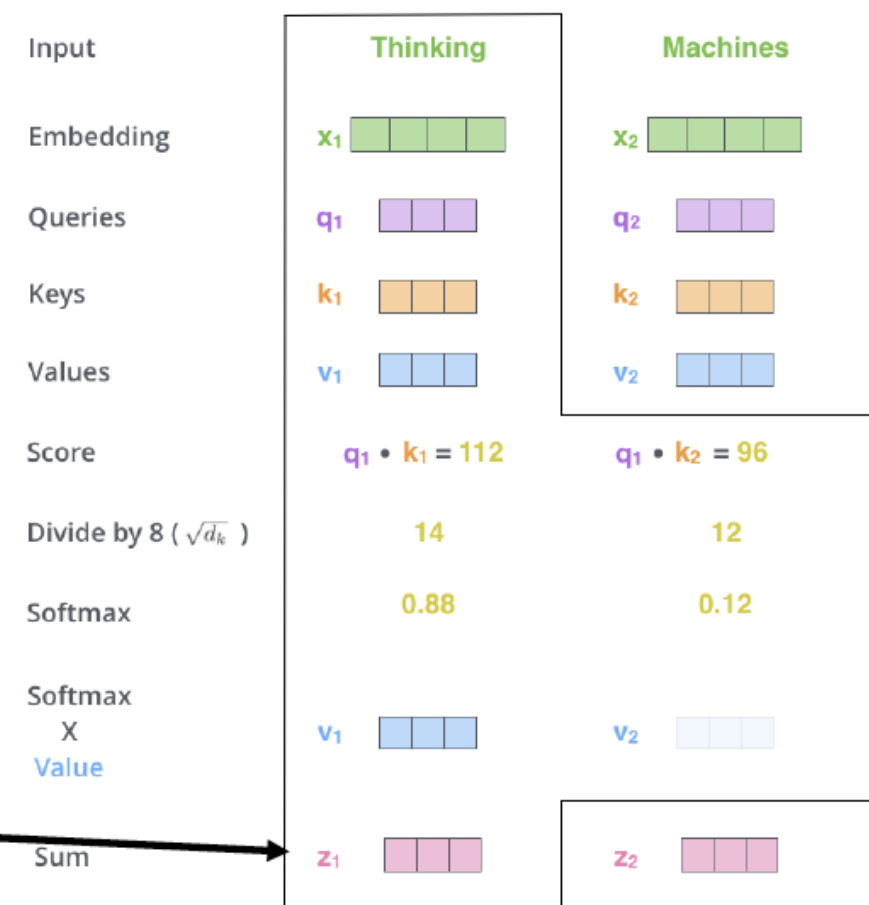
- Step 1: compute “key, value, query” for each input

Step 1



- Step 2 (just for x_1): compute scores between pairs, turn into probabilities (same for x_2)

Step 2



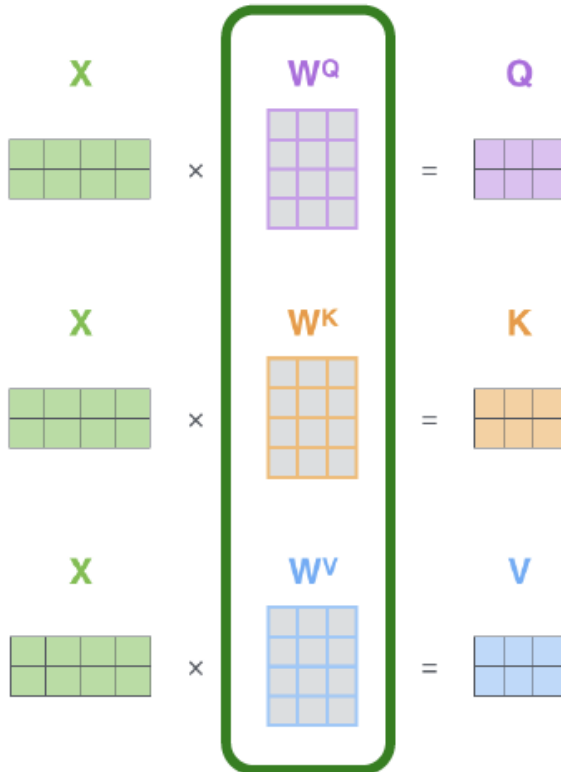
Step 3

$$z_1 = 0.88v_1 + 0.12v_2$$

Self-Attention

- Same calculation in matrix form

Step 1



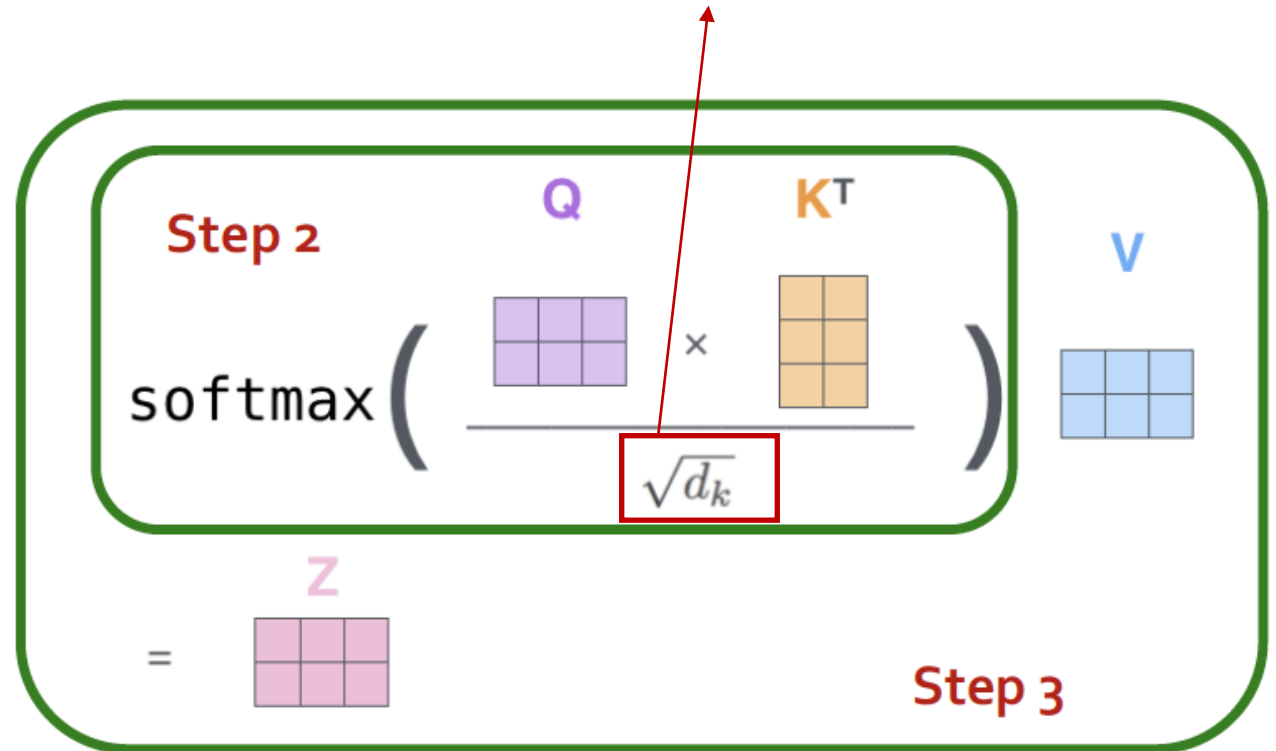
Model
parameters

Large similarities will cause softmax to saturate and give vanishing gradients.

Recall $a \cdot b = |a| |b| \cos(\text{angle})$

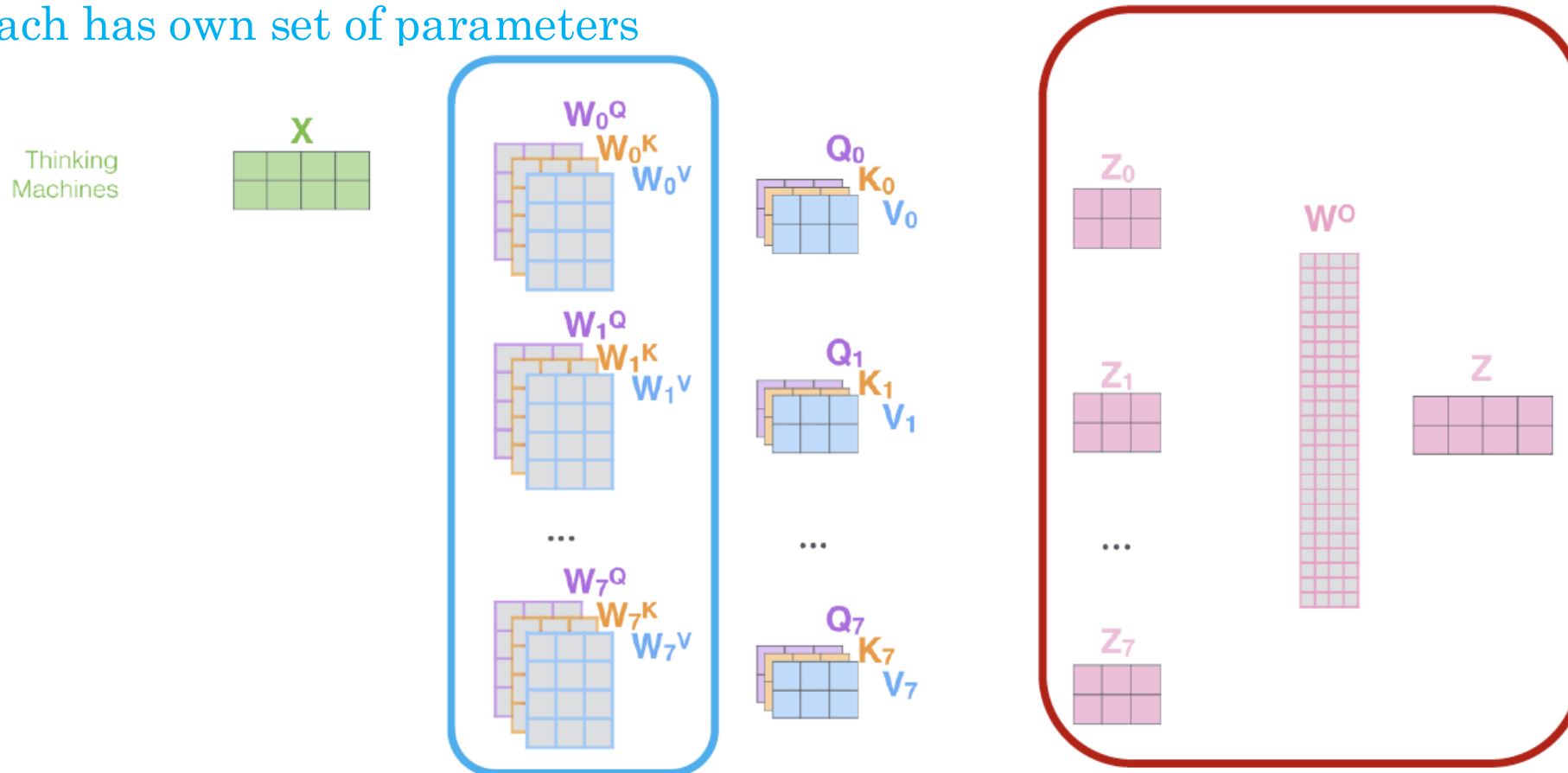
Suppose that a and b are constant vectors of dimension D .

$$\text{Then } |a| = (\sum_i a^2)^{\frac{1}{2}} = a\sqrt{D}$$



Multi-Head Self-Attention

- Do many self-attentions in parallel, and **combine**
- Different heads can learn different “similarities” between inputs
- **Each has own set of parameters**



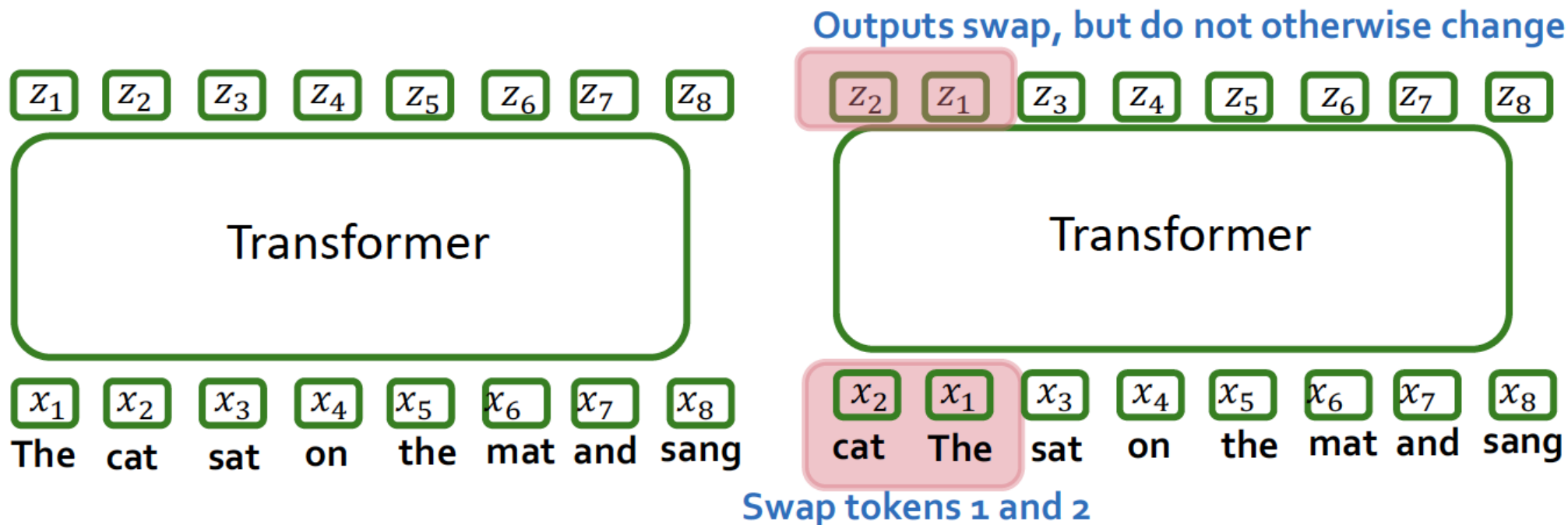
Token Ordering

Token Ordering

- First recall update formula: $z_1 = \sum_{j=1}^n \text{softmax}(q_1^T k_j) v_j$
- Key Observation: order of tokens does not matter!!!

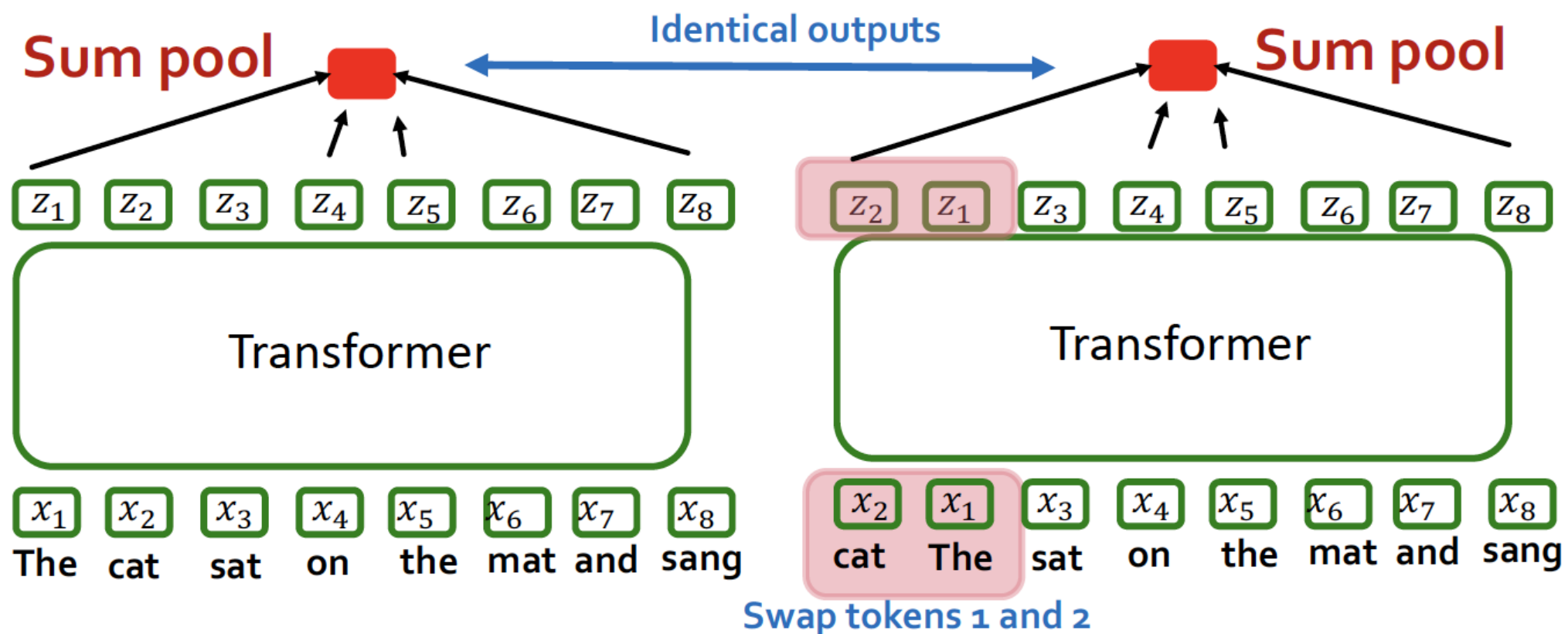
Token Ordering

- First recall update formula: $z_1 = \sum_{j=1}^n \text{softmax}(q_1^T k_j) v_j$
- Key Observation: order of tokens does not matter!!!



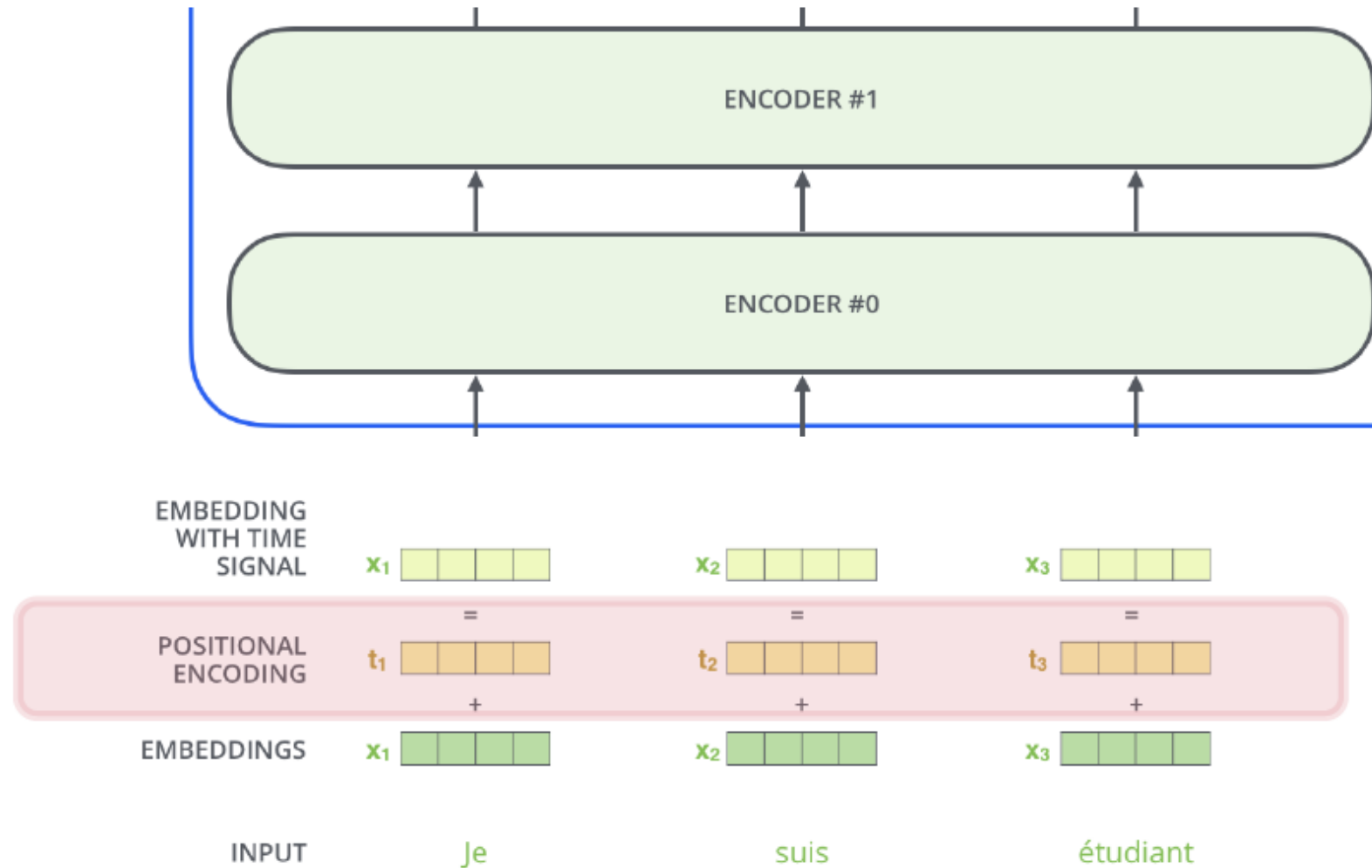
Token Ordering

- This is a problem
- Same predictions no matter what order the words are in!
 - How to fix?

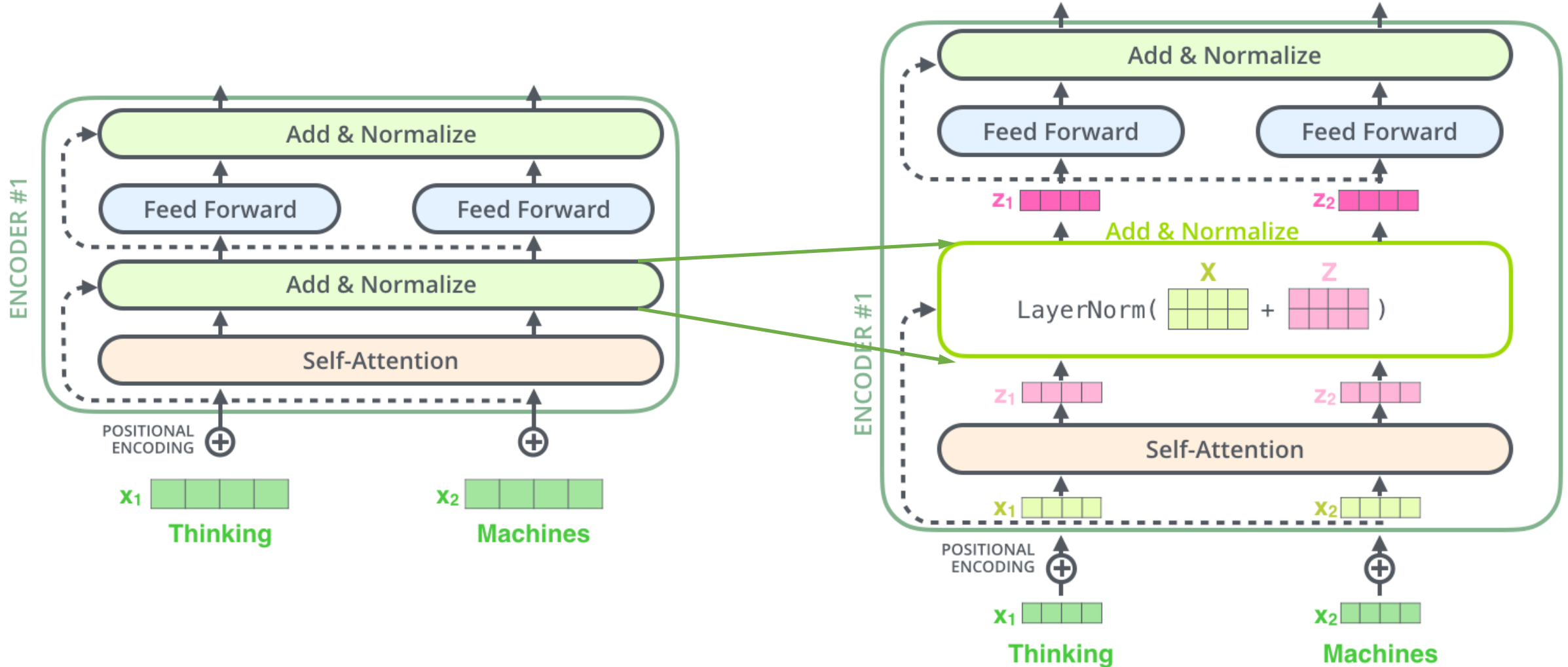


Positional Encodings

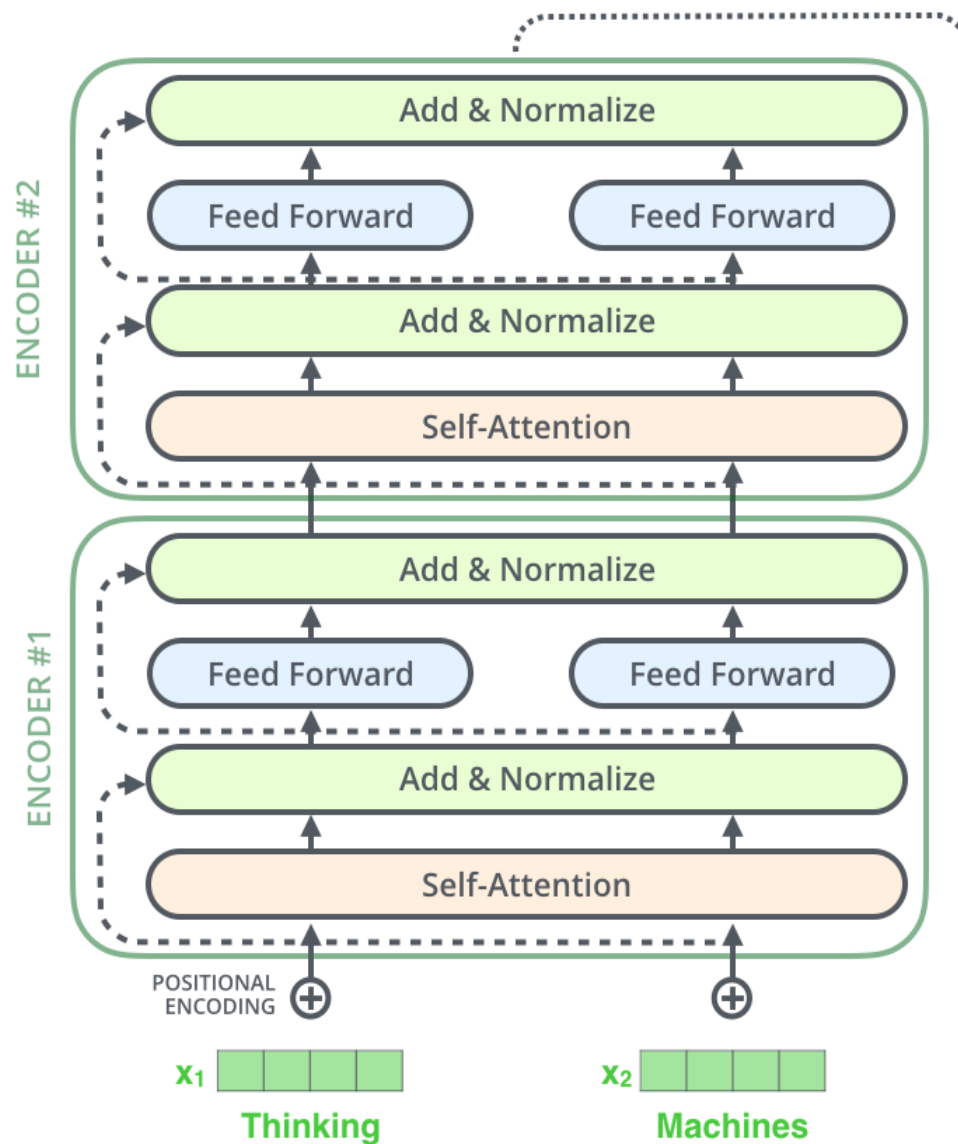
- Transformer doesn't know order of inputs
- Extra positional features needed so it knows that
 - Je = word 1,
 - suis = word 2
 - etc.
- For NLP, positional encoding vectors are learnable parameters



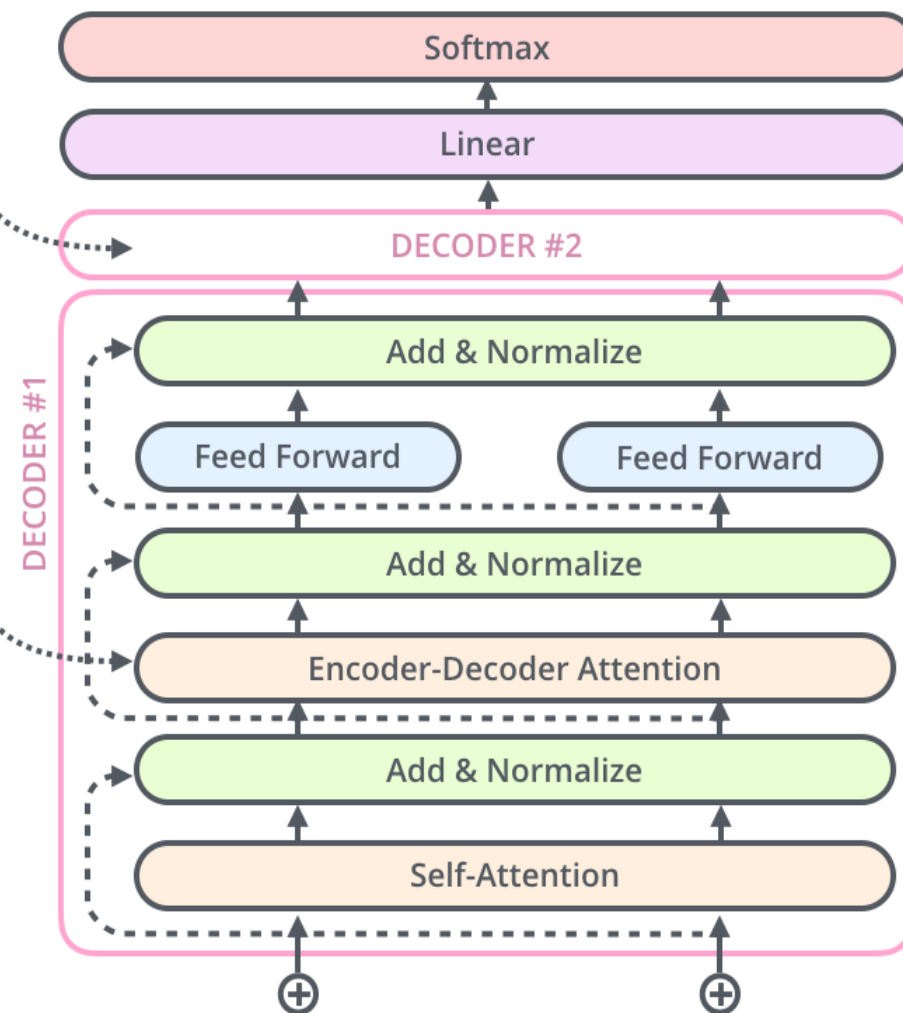
Residual Connection & Normalization



Transformer: Encoder



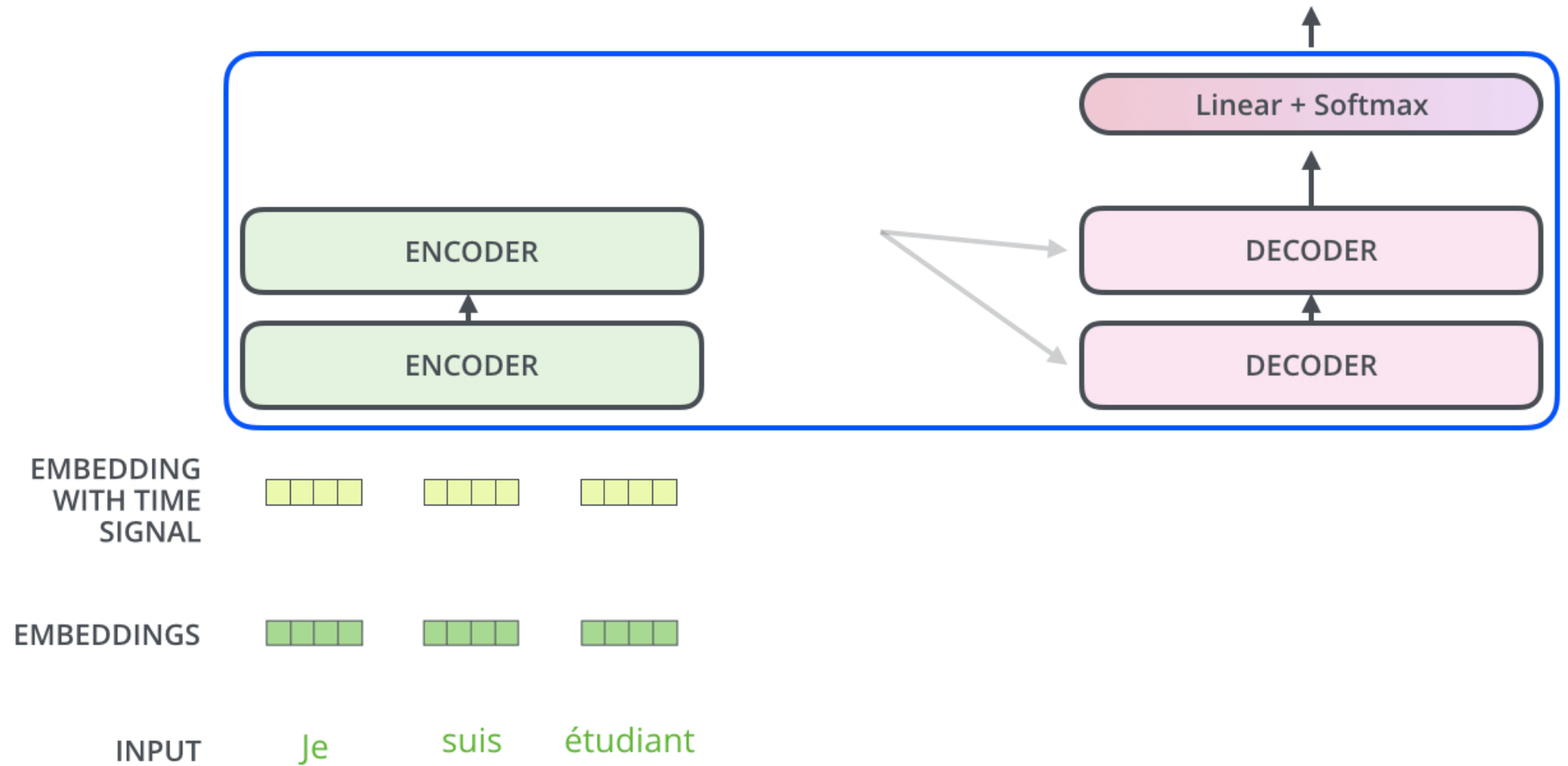
Transformer: Decoder



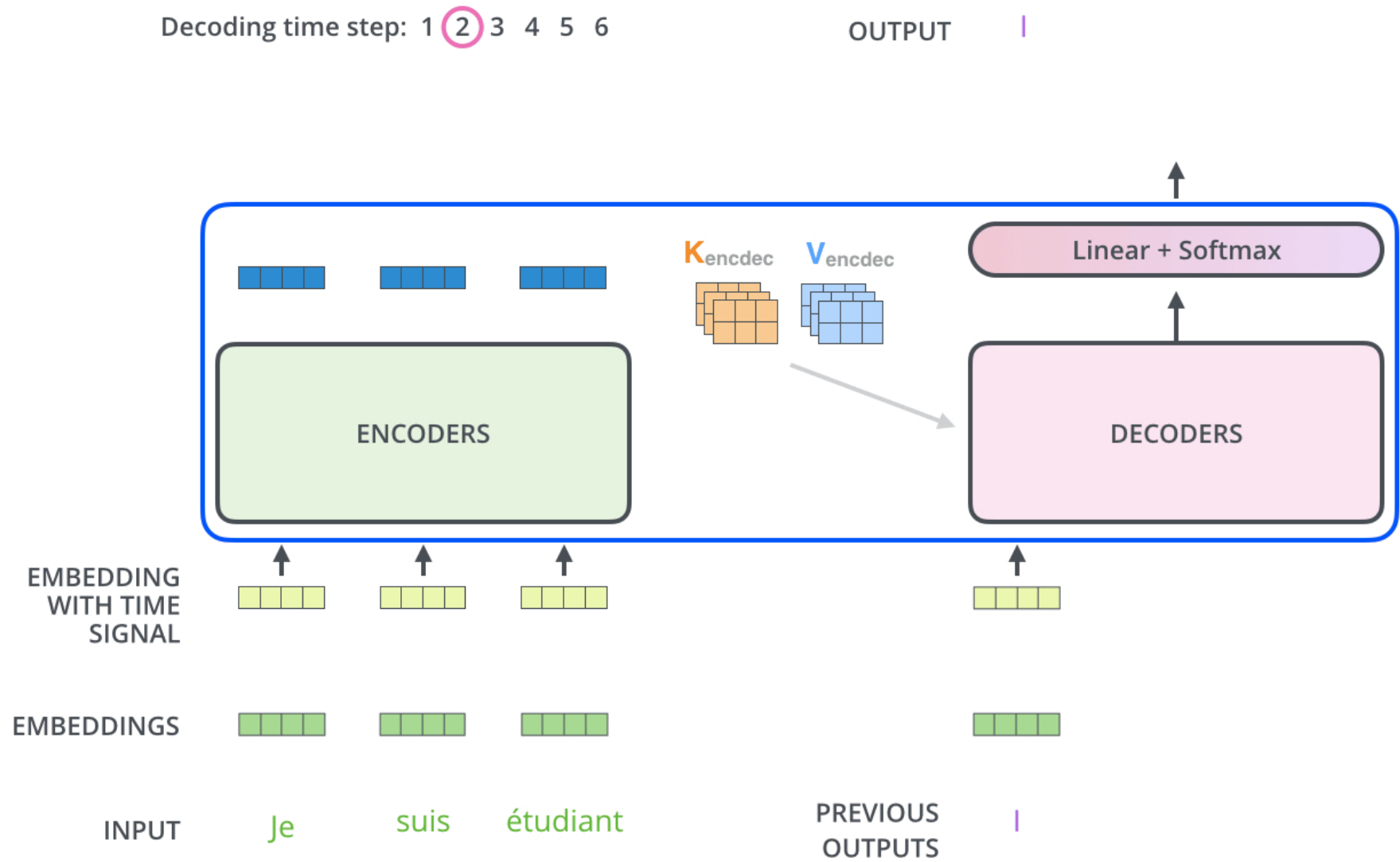
Decoder

Decoding time step: 1 2 3 4 5 6

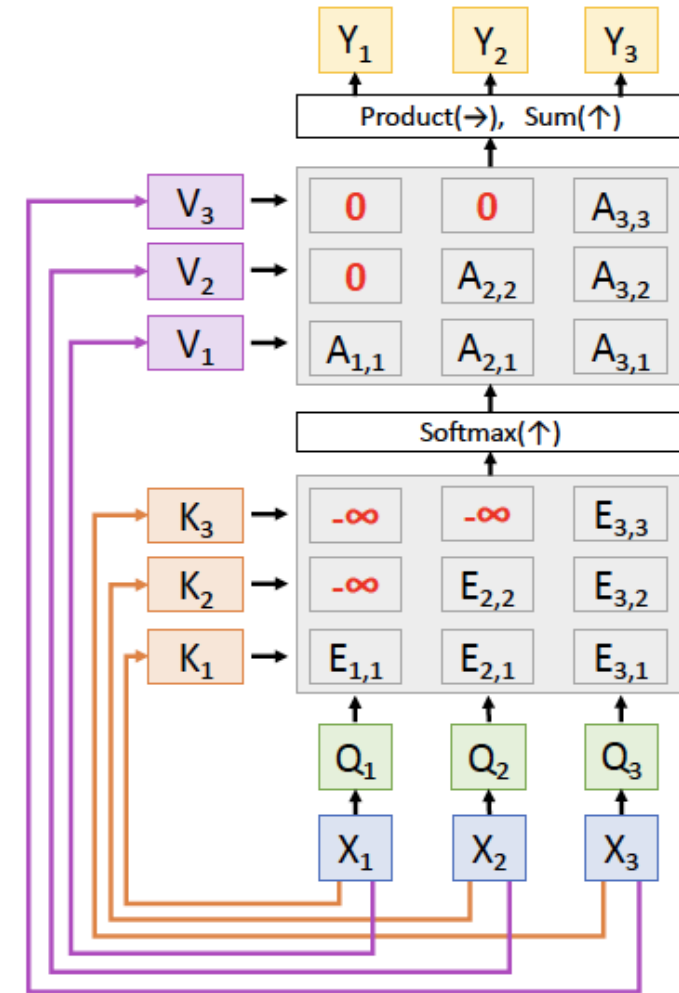
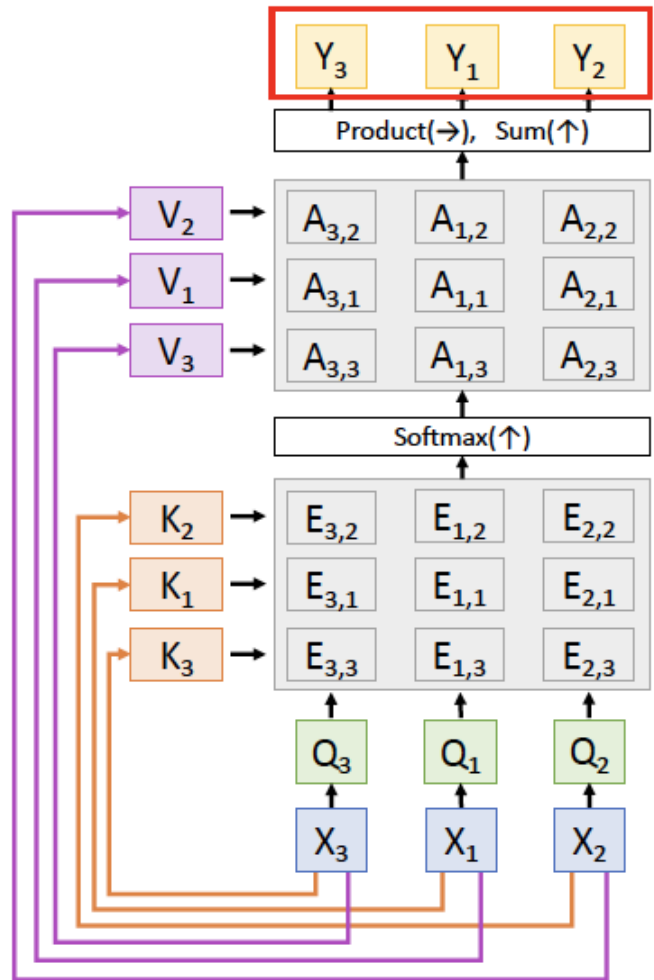
OUTPUT



Decoder

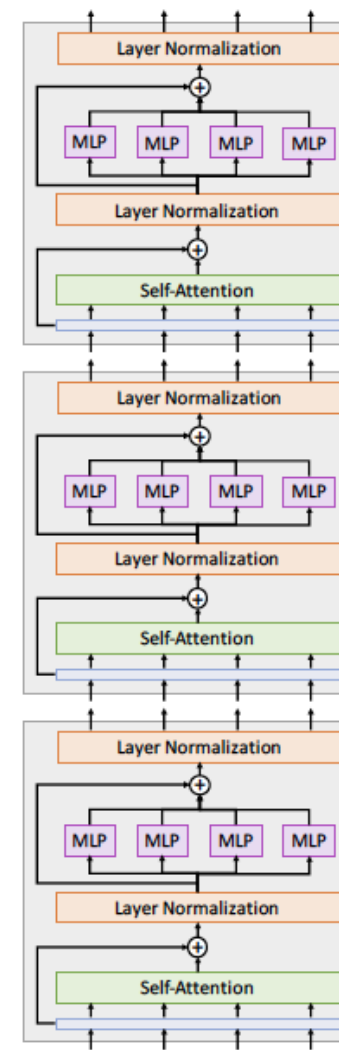


Decoder: Masked Self-Attention Layer



The Transformer: Transfer Learning

- “ImageNet Moment for Natural Language Processing”
- Pretraining:
 - Download a lot of text from the internet
 - Train a giant Transformer model for language modeling
- Finetuning:
 - Fine-tune the Transformer on your own NLP task

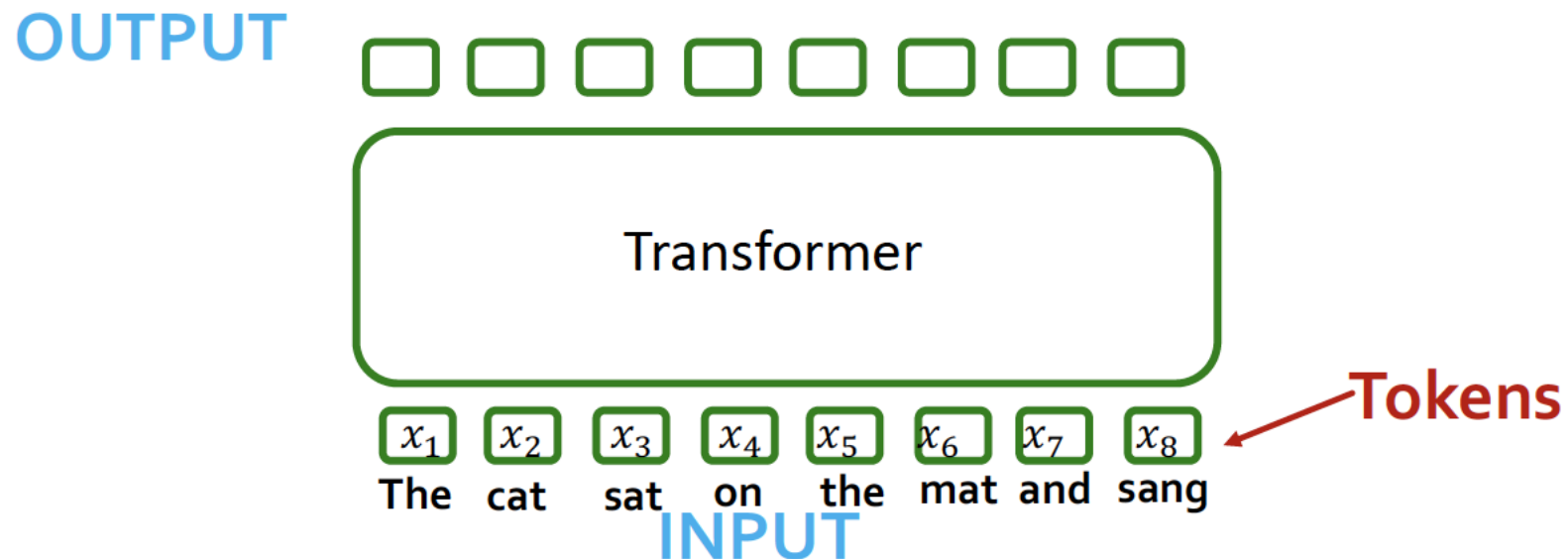


Scaling Up Transformers

Model	Layers	Width	Heads	Params	Data	Training
Transformer-Base	12	512	8	65M		8x P100 (12 hours)
Transformer-Large	12	1024	16	213M		8x P100 (3.5 days)
BERT-Base	12	768	12	110M	13 GB	
BERT-Large	24	1024	16	340M	13 GB	
XLNet-Large	24	1024	16	~340M	126 GB	512x TPU-v3 (2.5 days)
RoBERTa	24	1024	16	355M	160 GB	1024x V100 GPU (1 day)
GPT-2	48	1600	?	1.5B	40 GB	
Megatron-LM	72	3072	32	8.3B	174 GB	512x V100 GPU (9 days)
Turing-NLG	78	4256	28	17B	?	256x V100 GPU
GPT-3	96	12,288	96	175B	694GB	?
Gopher	80	16,384	128	280B	10.55 TB	4096x TPUv3 (38 days)

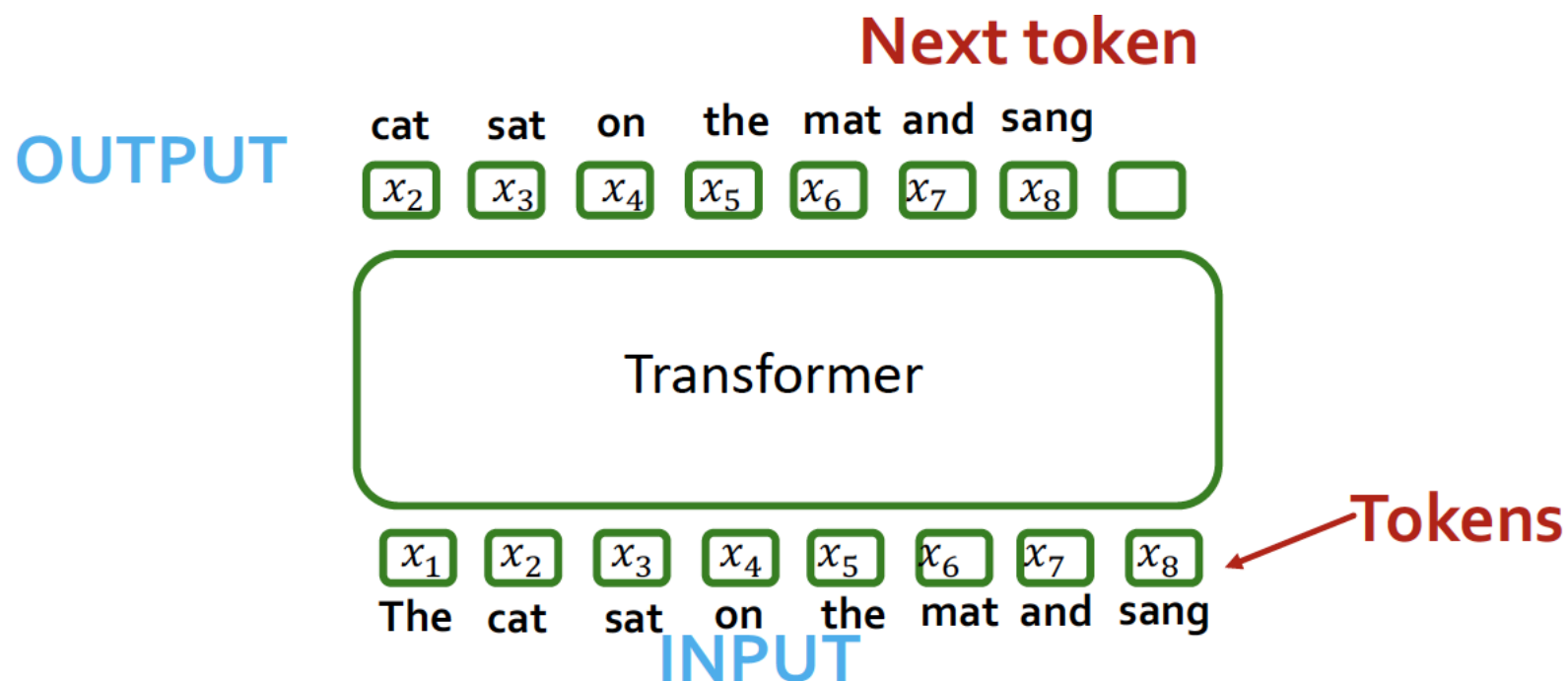
Transformer Ingest Tokens

- Transformers map 1D sequences of vectors to 1D sequences of vectors



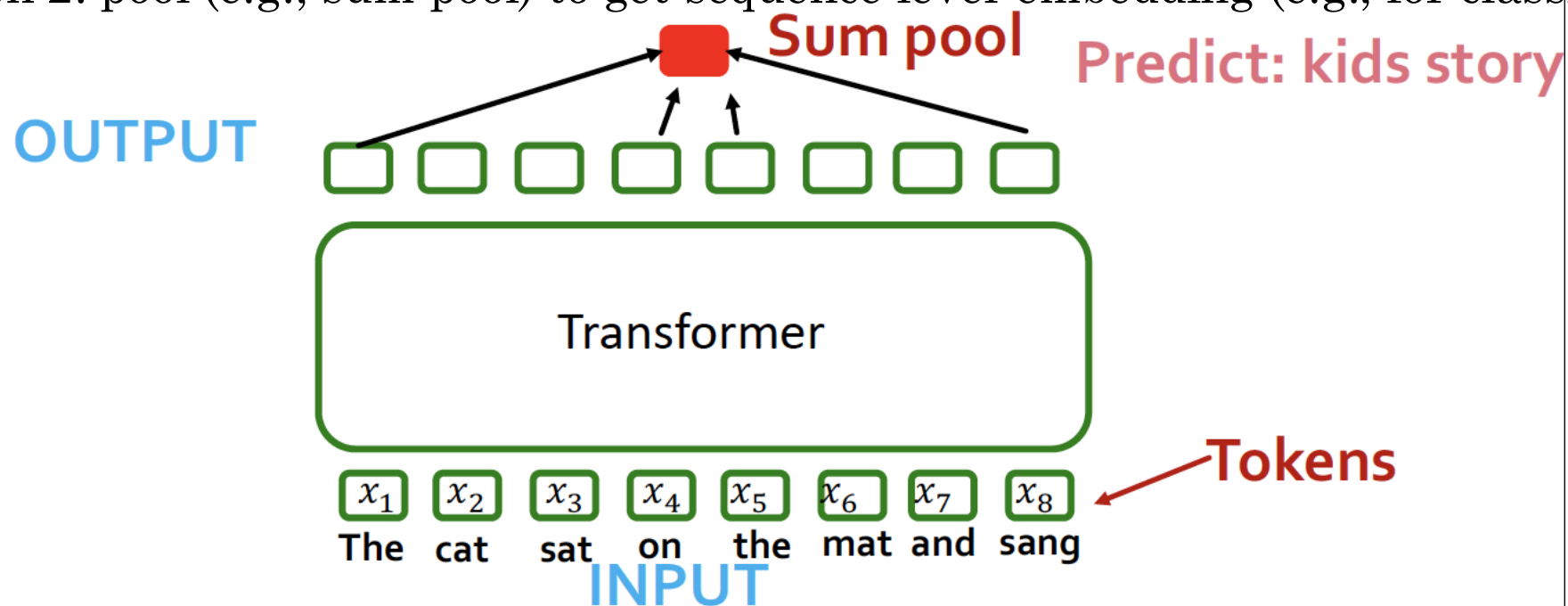
Transformer Ingest Tokens

- Transformers map 1D sequences of vectors to 1D sequences of vectors
 - Tokens describe a "piece" of data – e.g., a word
- What output sequence?
 - Option 1: next token => GPT



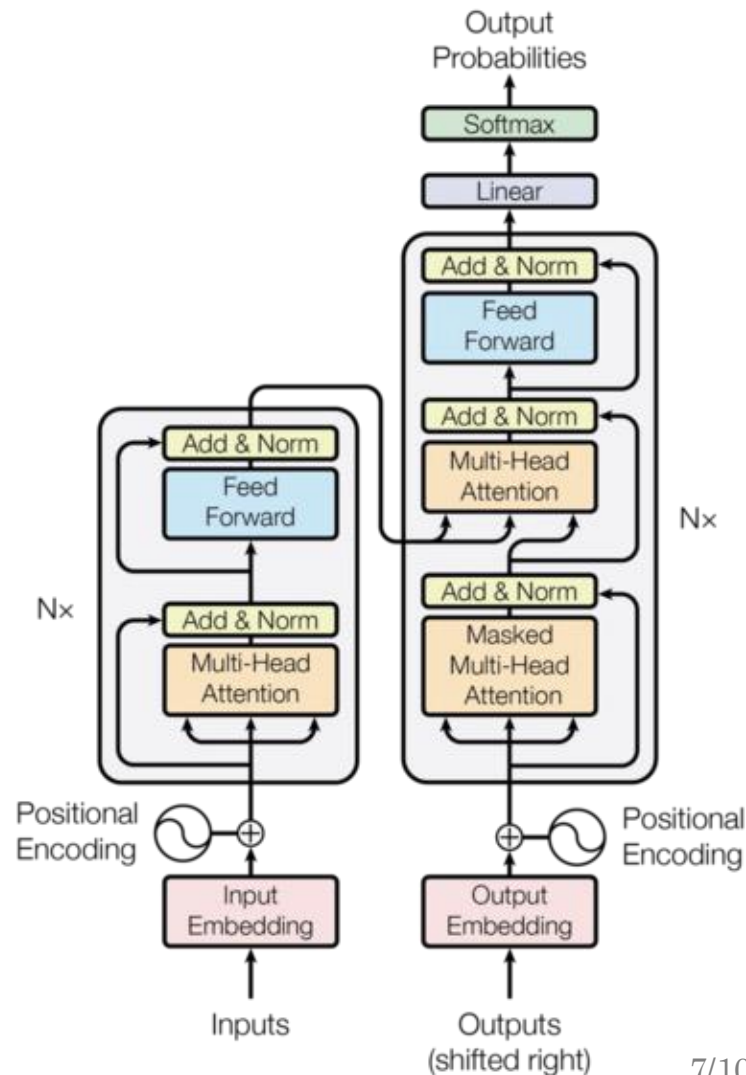
Transformer Ingest Tokens

- Transformers map 1D sequences of vectors to 1D sequences of vectors
 - Tokens describe a "piece" of data – e.g., a word
- What output sequence?
 - Option 1: next token => GPT
 - Option 2: pool (e.g., sum-pool) to get sequence level-embedding (e.g., for classification task)



Transformer Blueprint

- How are tokens processed?
- Lots of components
 - Normalization
 - Feed forward networks
 - Positional encoding
 - Multi-head self-attention
- What does self-attention block do?



Self-Attention

- Intuition

- As the model processes each word, **self attention** allows it to look at other positions in the input sequence for clues that can help lead to a better encoding for this word.

- Example:

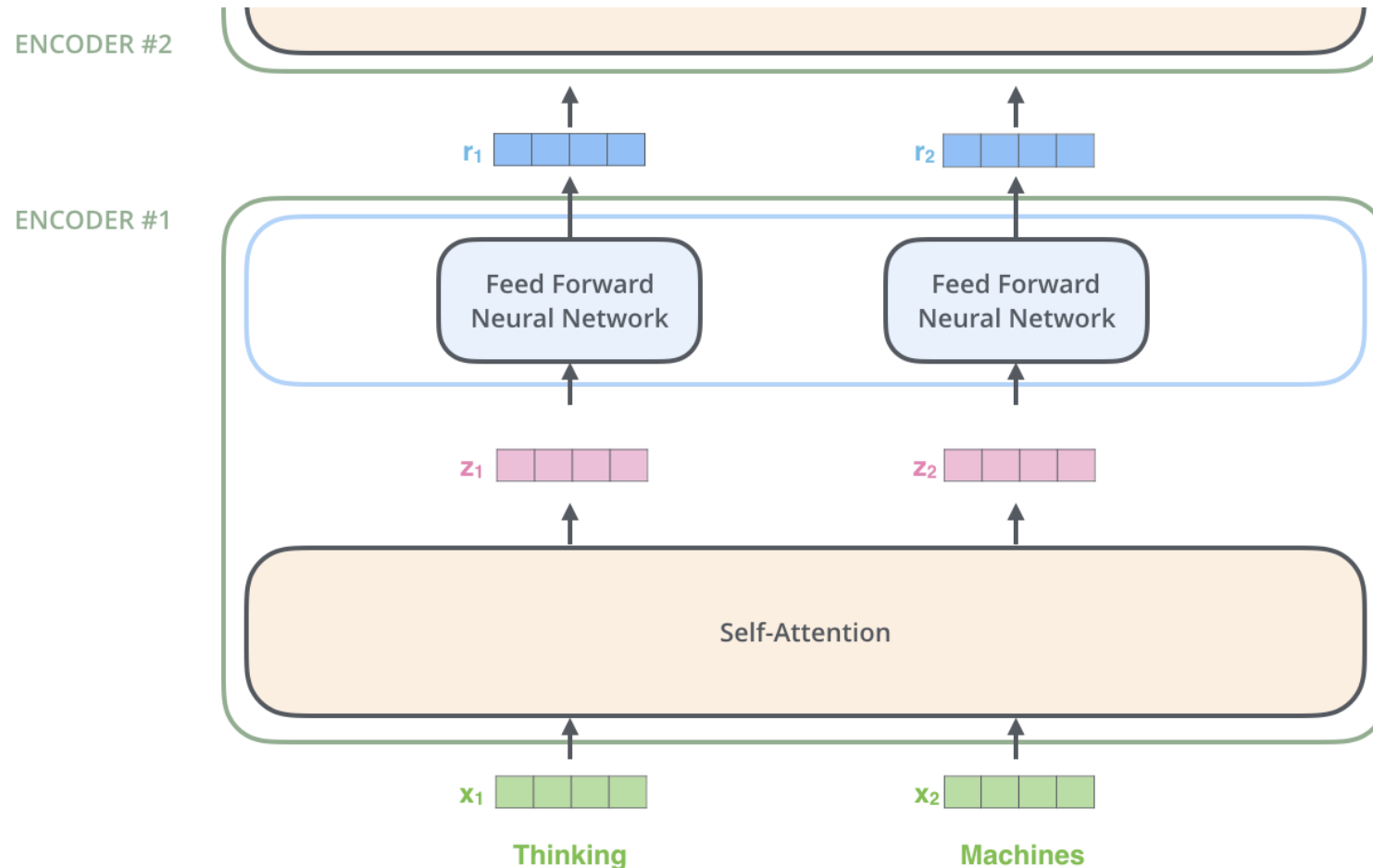
The animal didn't cross the street because it was too tired.



- When a model processes the word “it”, **self-attention** allows it to associate “it” with “animal”.

Self-Attention

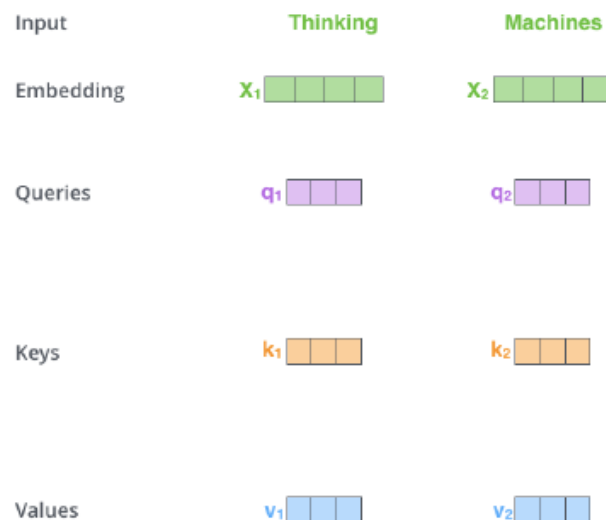
- Before “multi-head” self-attention, what is “single head” self-attention?



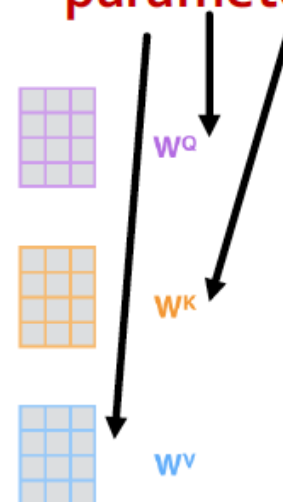
Self-Attention

- **Step 1:** compute “key, value, query” for each input

Step 1



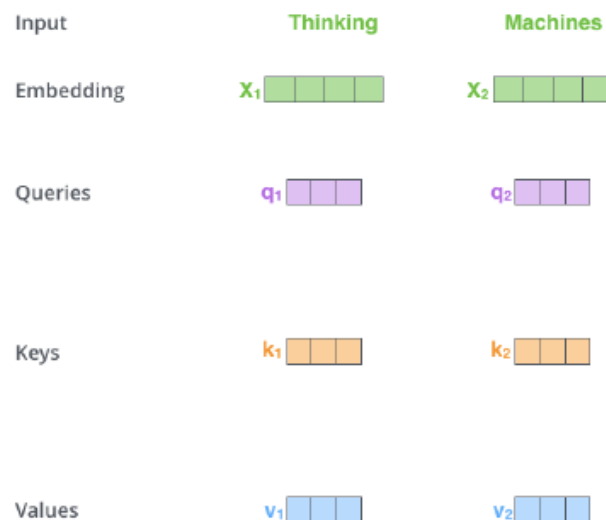
Model parameters



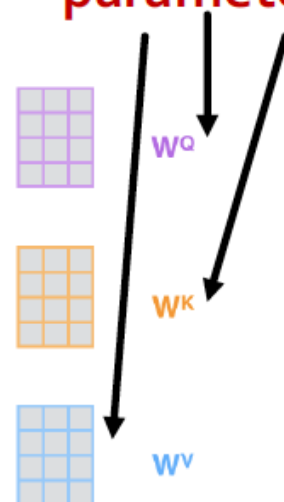
Self-Attention

- Step 1: compute “key, value, query” for each input

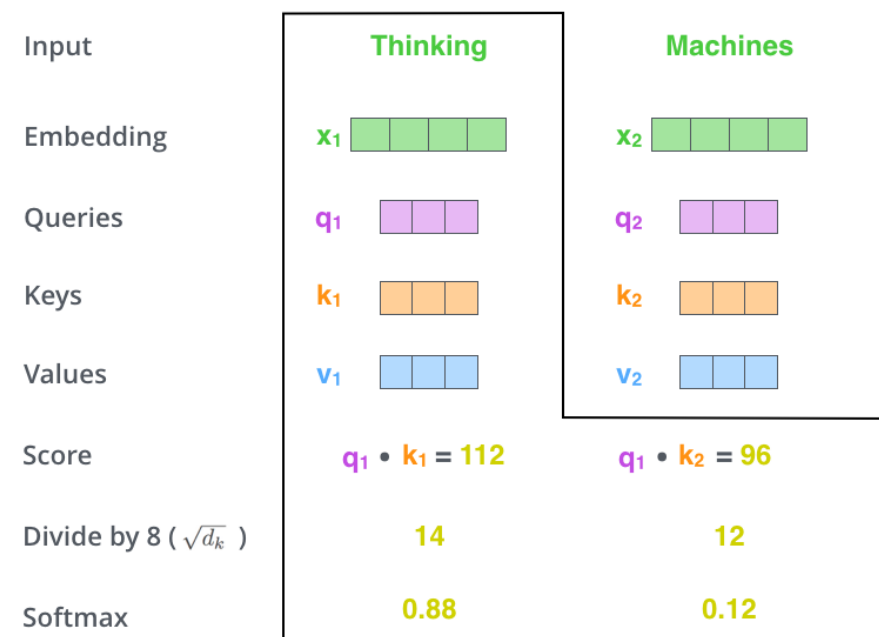
Step 1



Model parameters



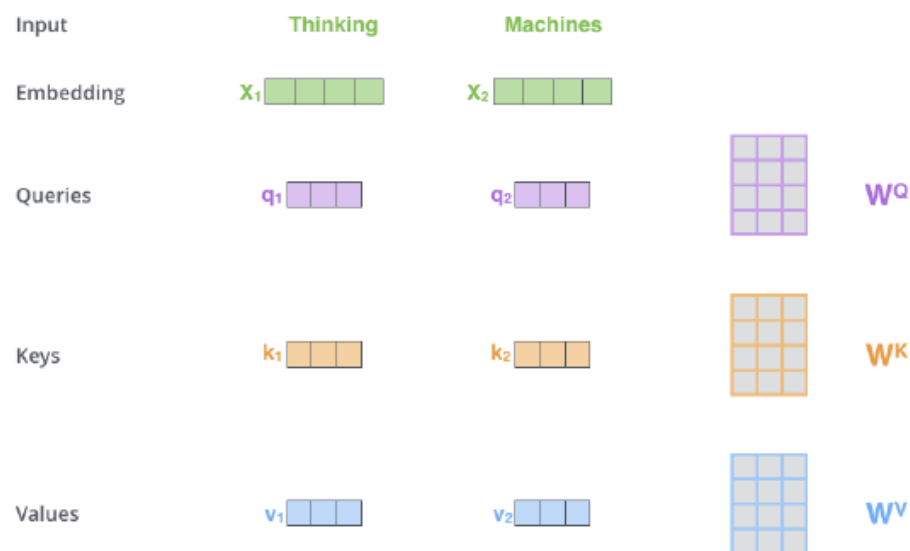
Step 2



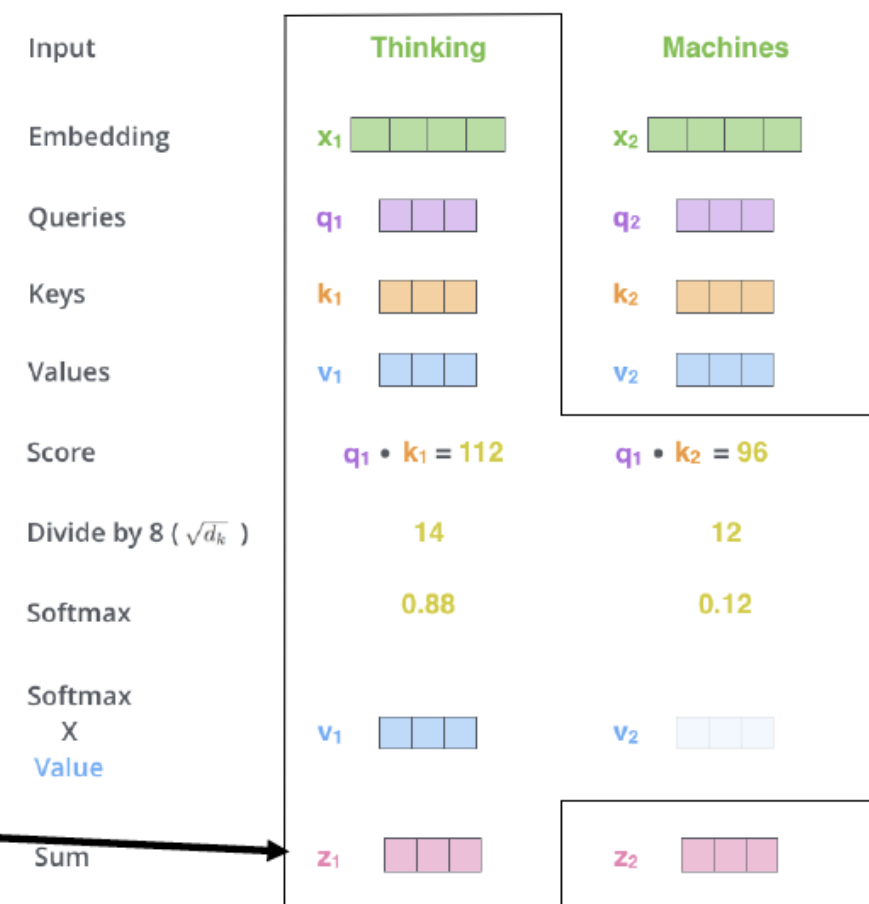
Self-Attention

- **Step 1:** compute “key, value, query” for each input
- **Step 2 (just for x_1):** compute scores between pairs, turn into probabilities (same for x_2)
- **Step 3:** get new embedding z_1 by weighted sum of v_1, v_2

Step 1



Step 2



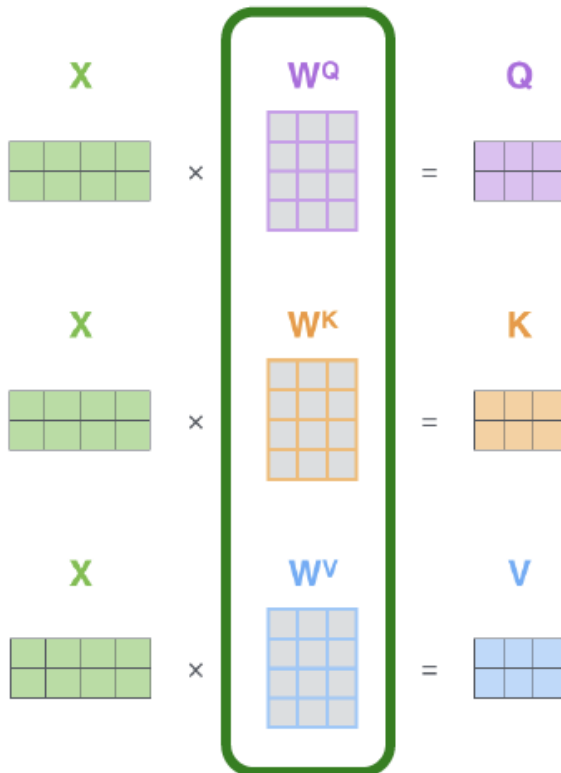
Step 3

$$z_1 = 0.88v_1 + 0.12v_2$$

Self-Attention

- Same calculation in matrix form

Step 1



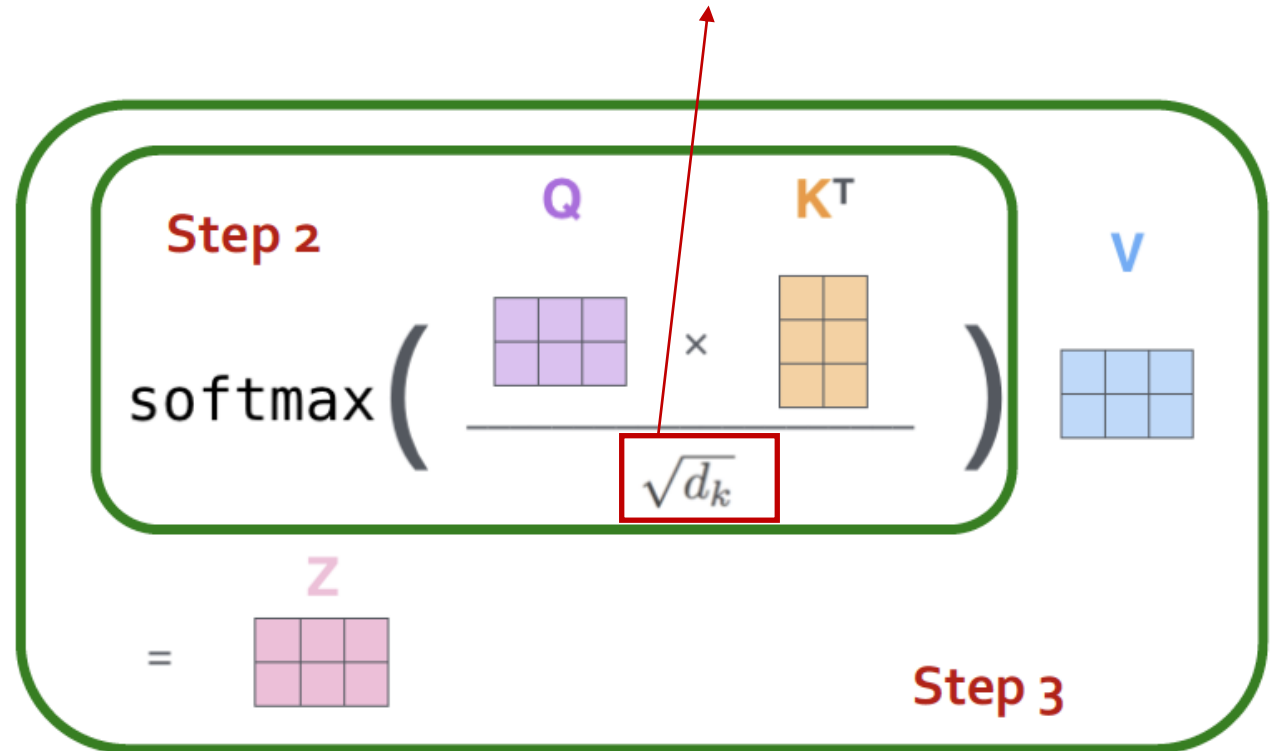
Model
parameters

Large similarities will cause softmax to saturate and give vanishing gradients.

Recall $a \cdot b = |a| |b| \cos(\text{angle})$

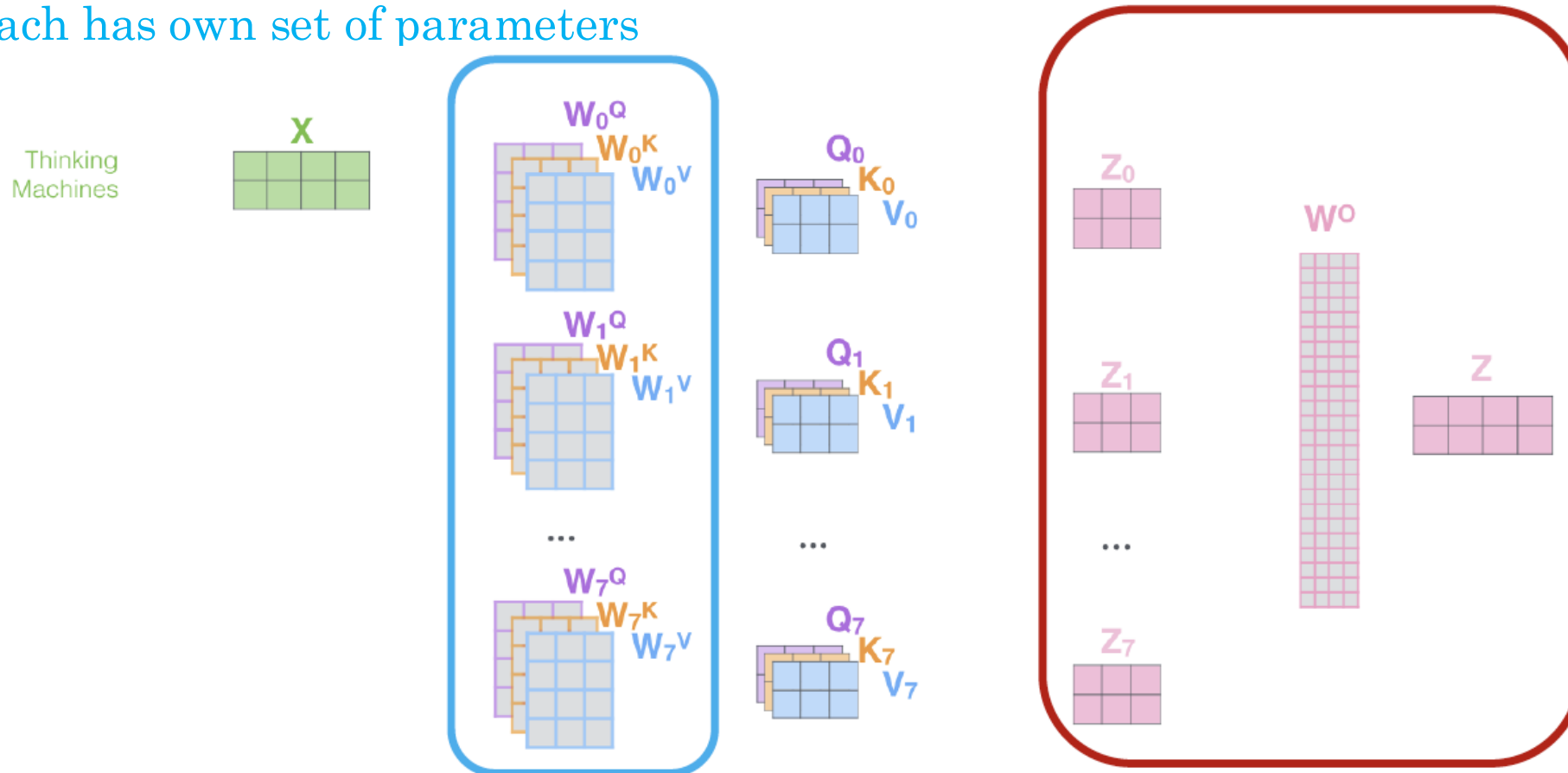
Suppose that a and b are constant vectors of dimension D .

$$\text{Then } |a| = (\sum_i a^2)^{\frac{1}{2}} = a\sqrt{D}$$



Multi-Head Self-Attention

- Do many self-attentions in parallel, and **combine**
- Different heads can learn different “similarities” between inputs
- **Each has own set of parameters**



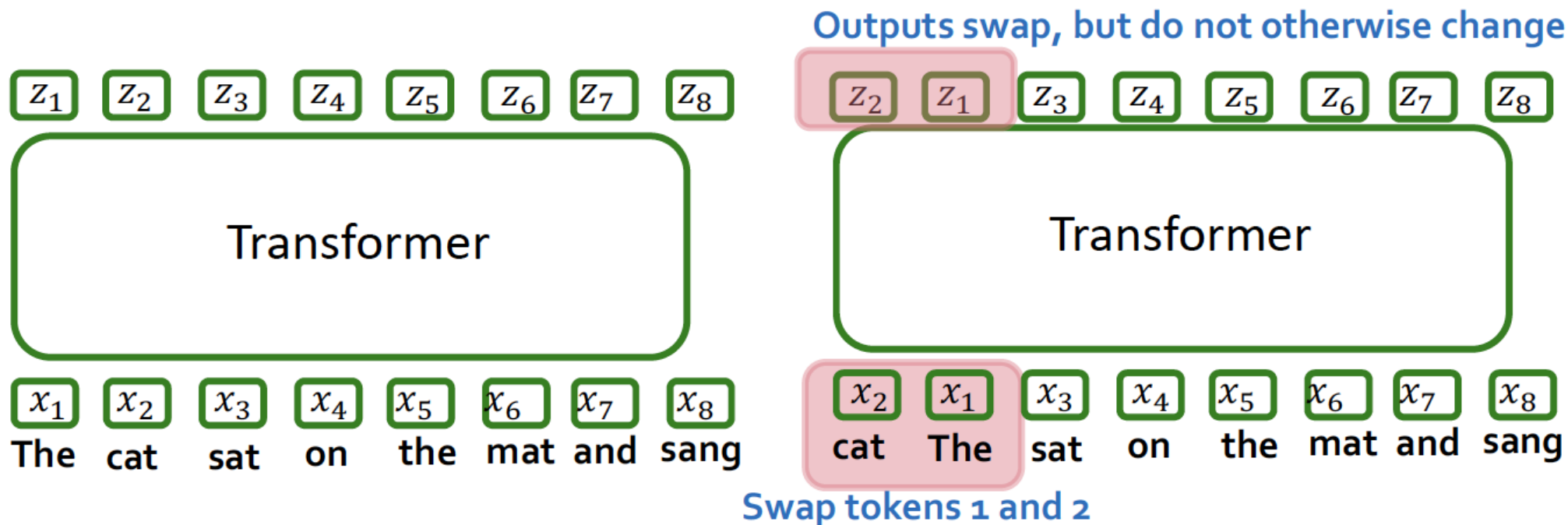
Token Ordering

Token Ordering

- First recall update formula: $z_1 = \sum_{j=1}^n \text{softmax}(q_1^T k_j) v_j$
- Key Observation: order of tokens does not matter!!!

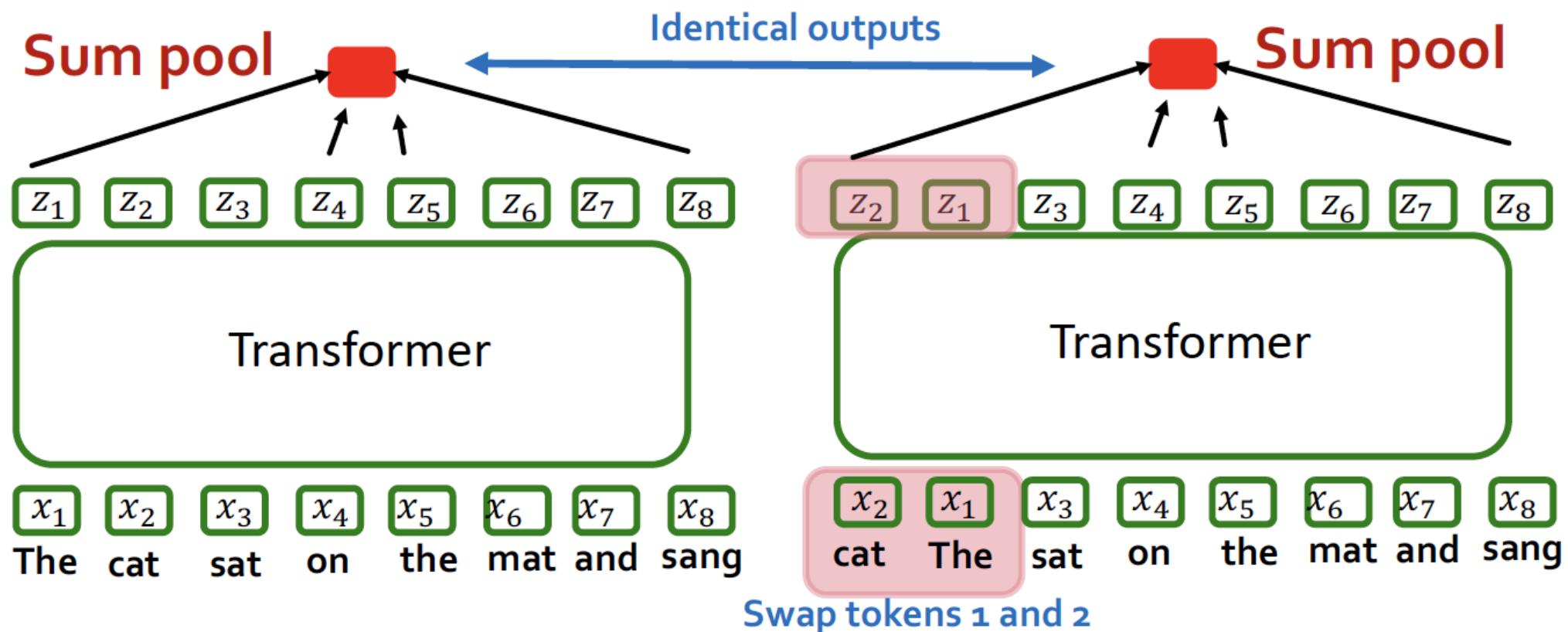
Token Ordering

- First recall update formula:
$$z_1 = \sum_{j=1}^n \text{softmax}(q_1^T k_j) v_j$$
- Key Observation: order of tokens does not matter!!!



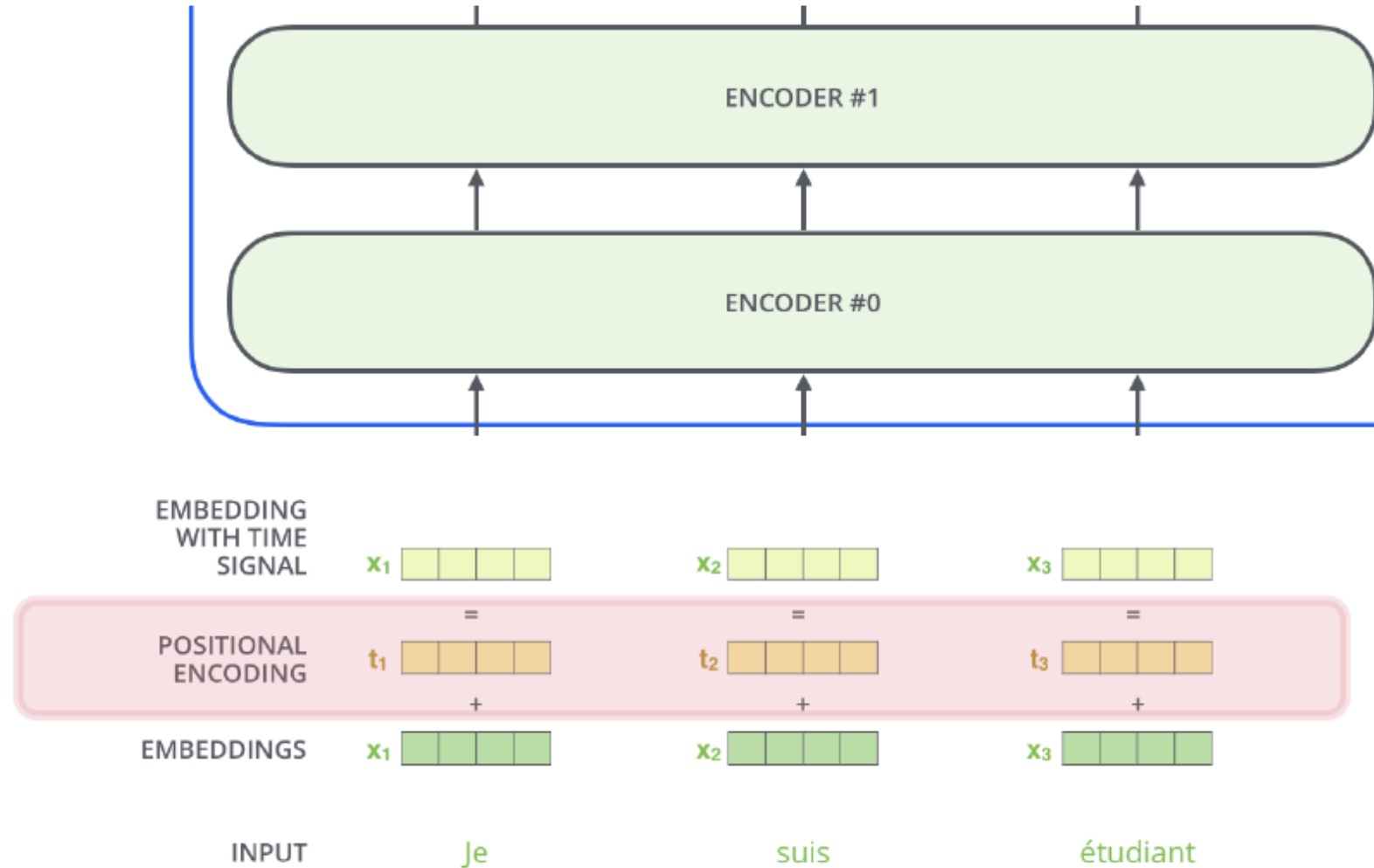
Token Ordering

- This is a problem
- Same predictions no matter what order the words are in!
 - How to fix?

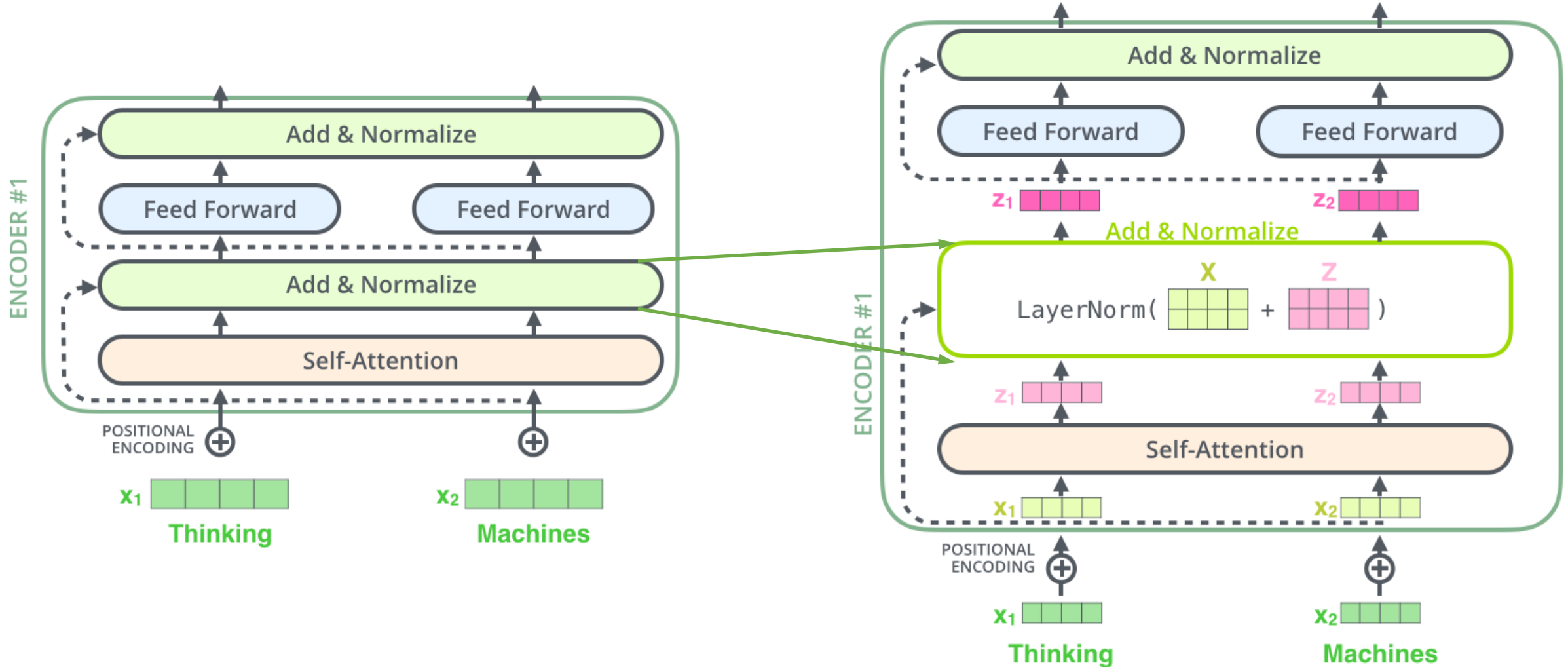


Positional Encodings

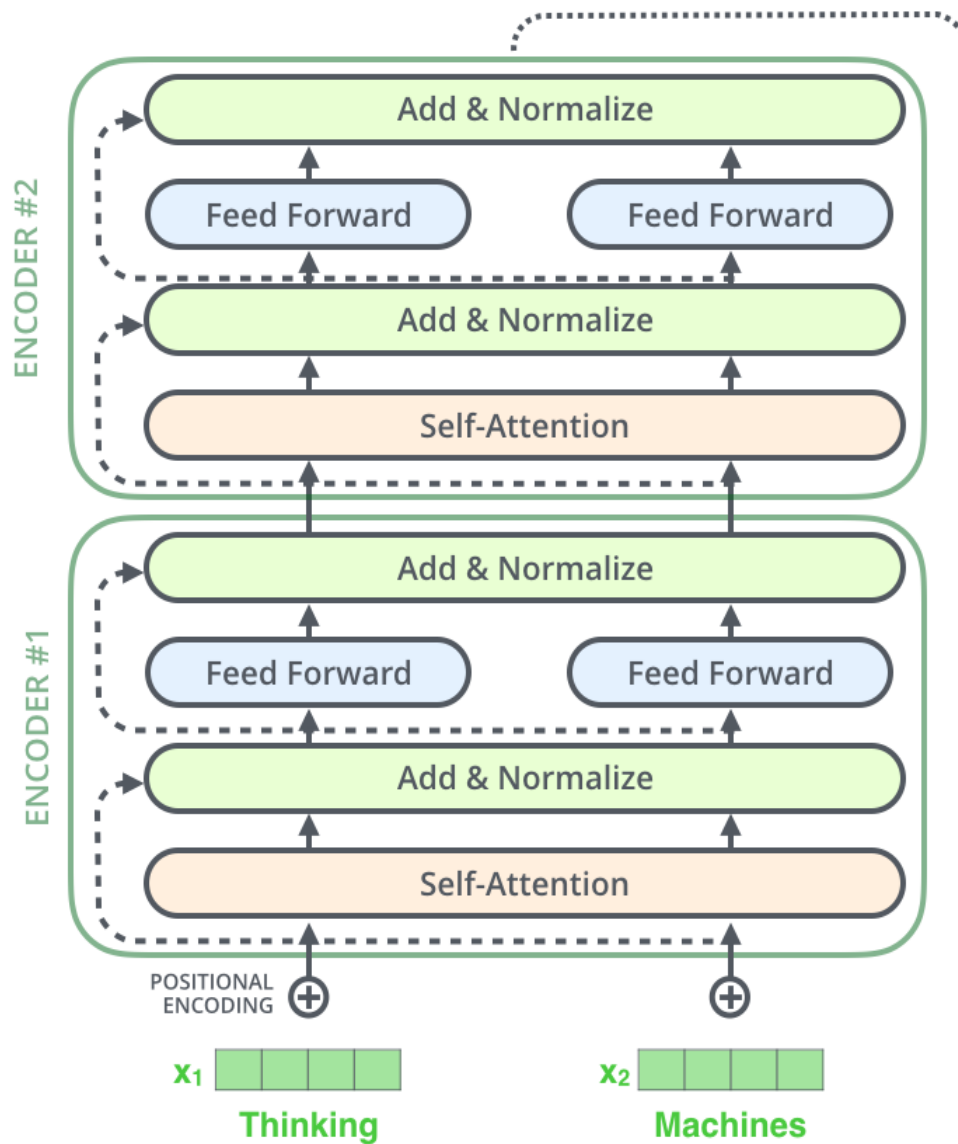
- Transformer doesn't know order of inputs
- Extra positional features needed so it knows that
 - Je = word 1,
 - suis = word 2
 - etc.
- For NLP, positional encoding vectors are learnable parameters



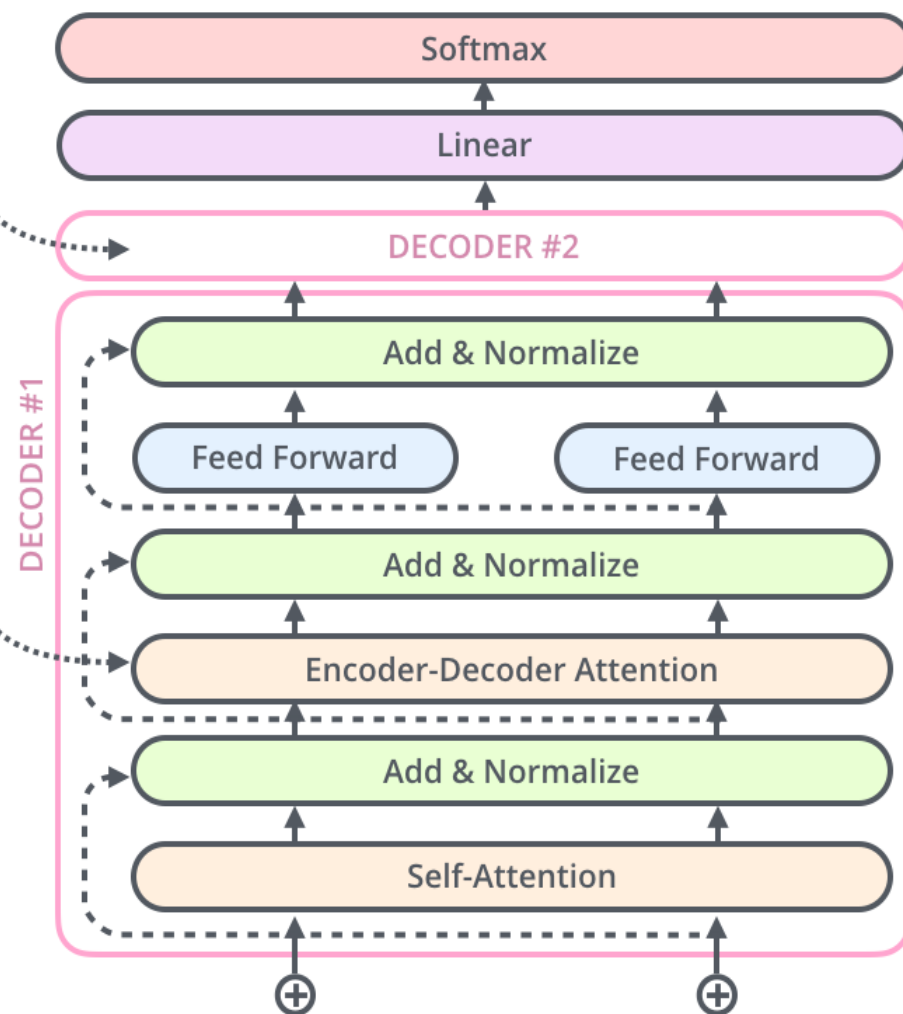
Residual Connection & Normalization



Transformer: Encoder



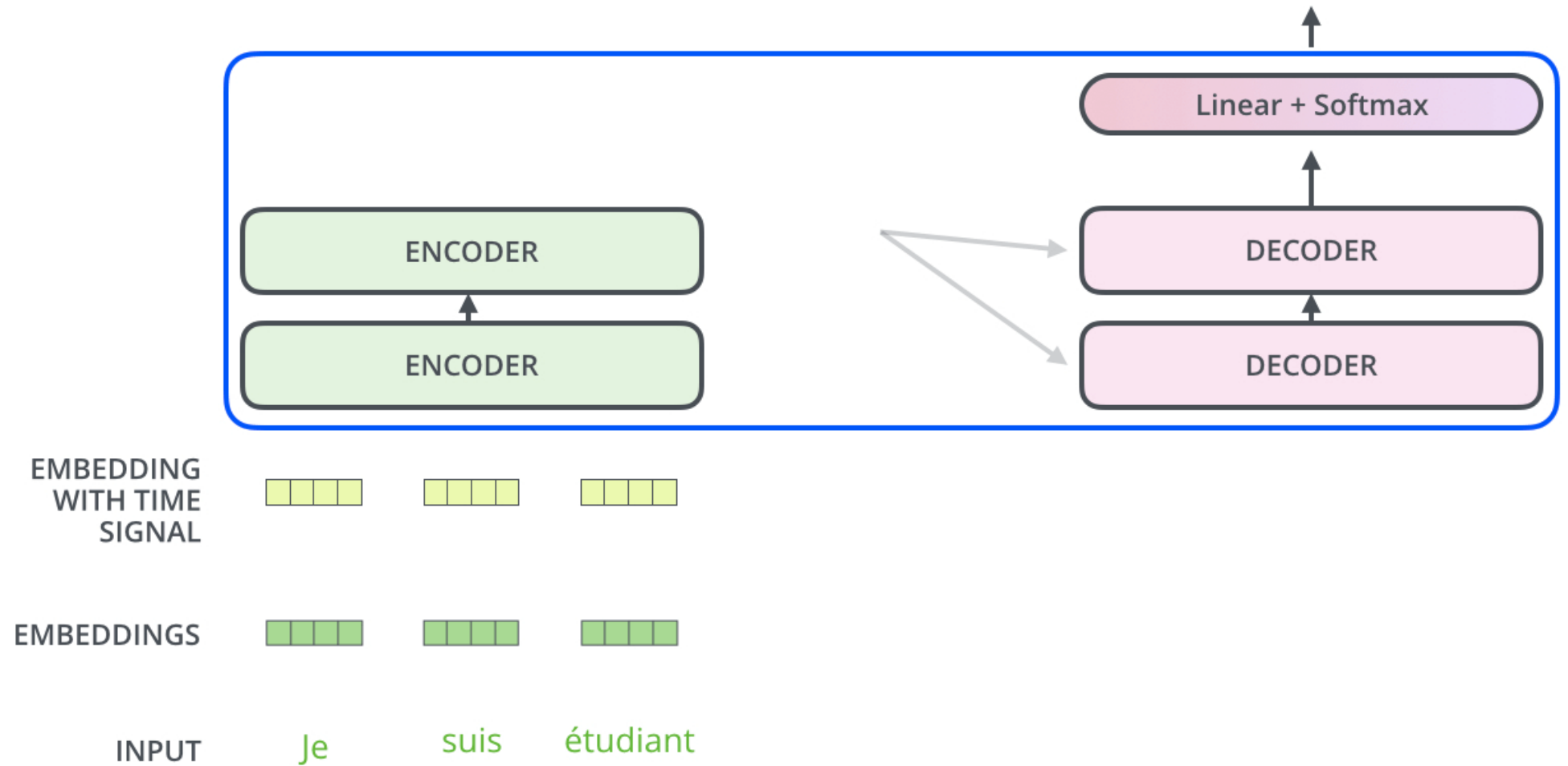
Transformer: Decoder



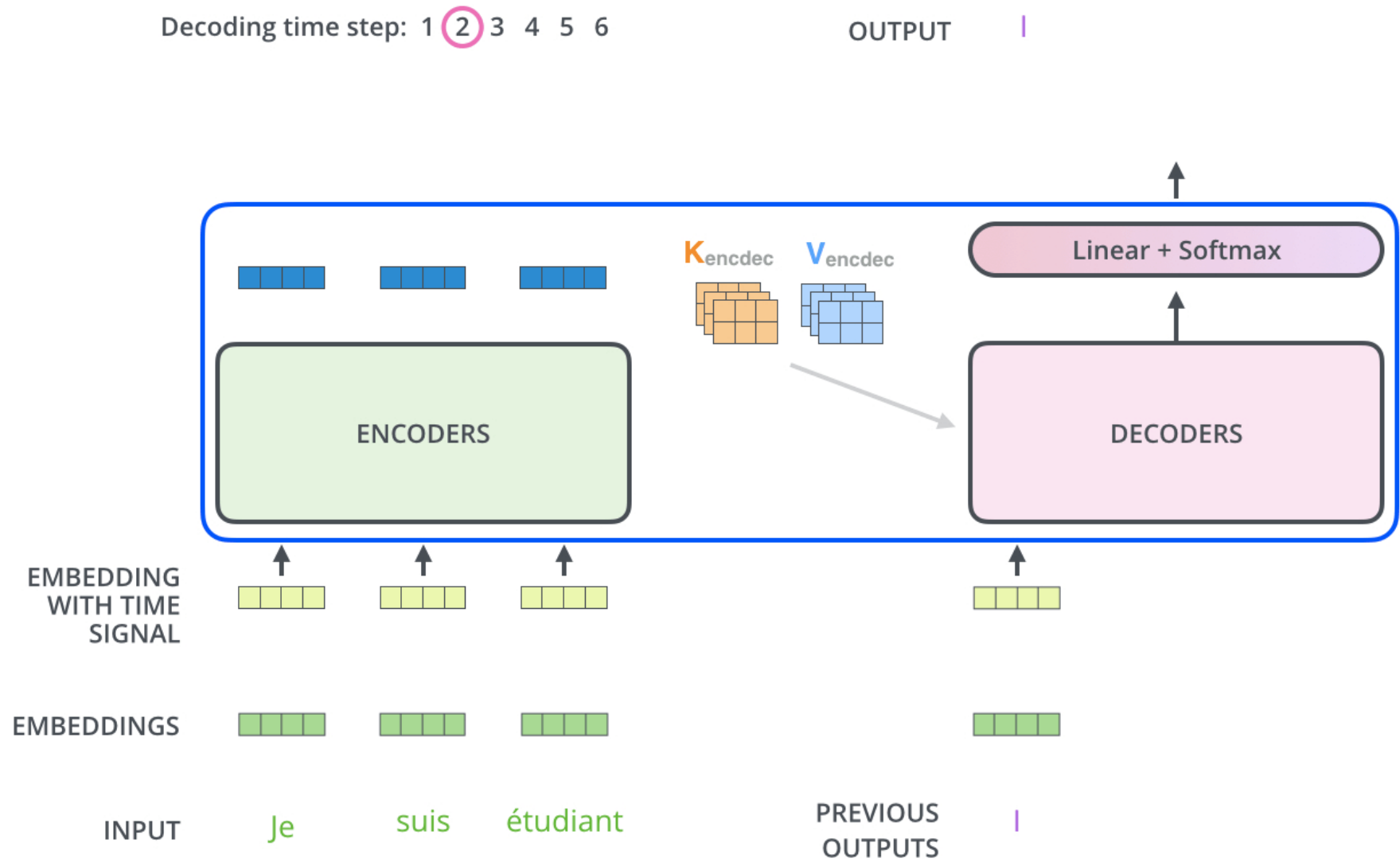
Decoder

Decoding time step: 1 2 3 4 5 6

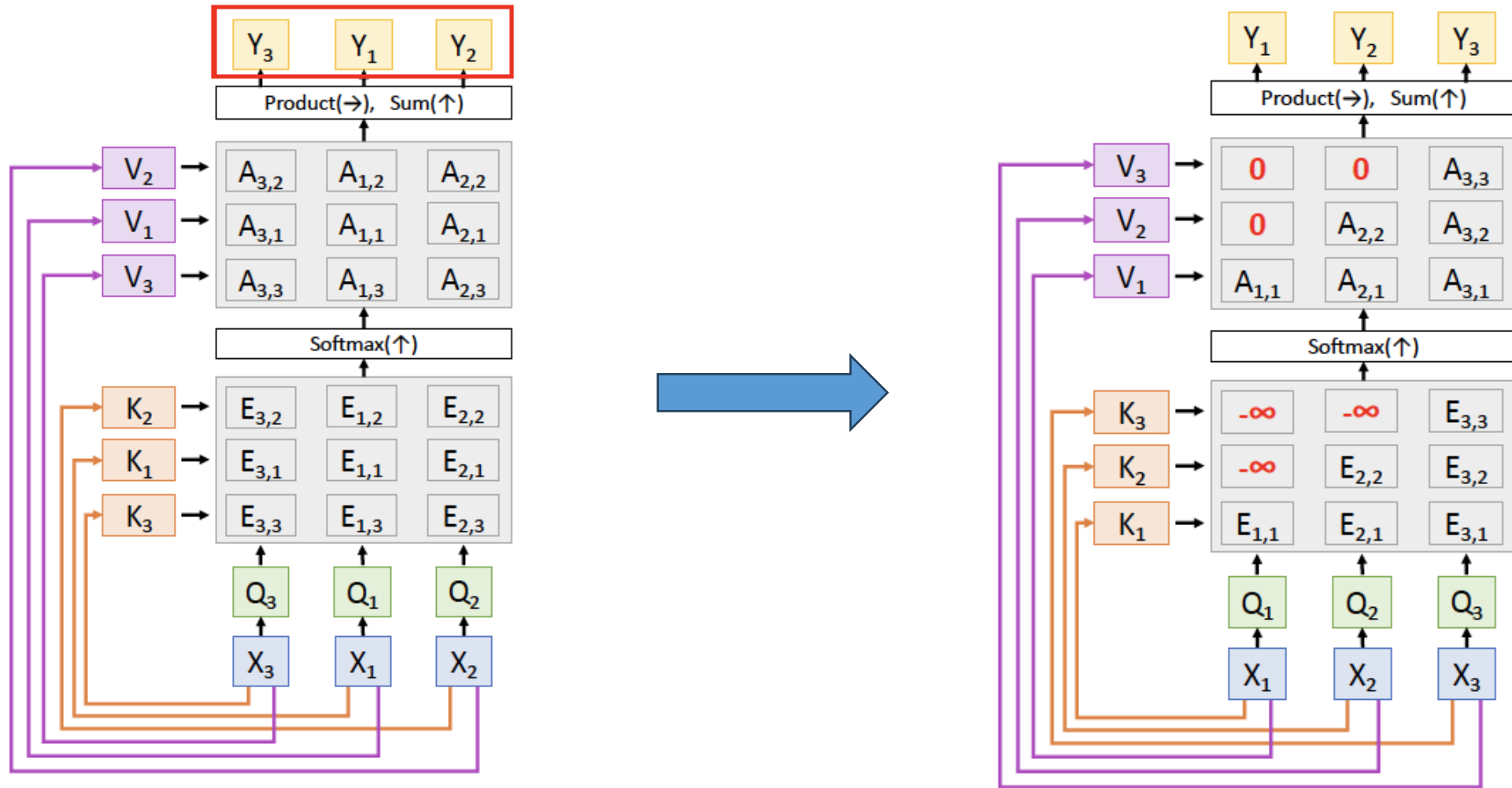
OUTPUT



Decoder

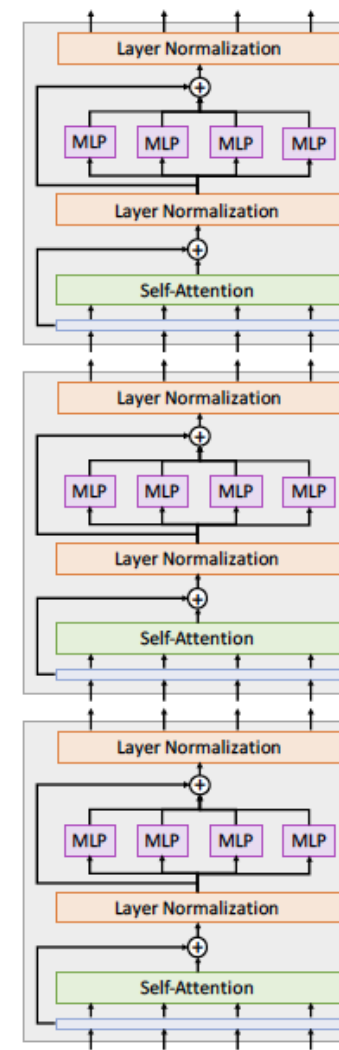


Decoder: Masked Self-Attention Layer



The Transformer: Transfer Learning

- “ImageNet Moment for Natural Language Processing”
- Pretraining:
 - Download a lot of text from the internet
 - Train a giant Transformer model for language modeling
- Finetuning:
 - Fine-tune the Transformer on your own NLP task



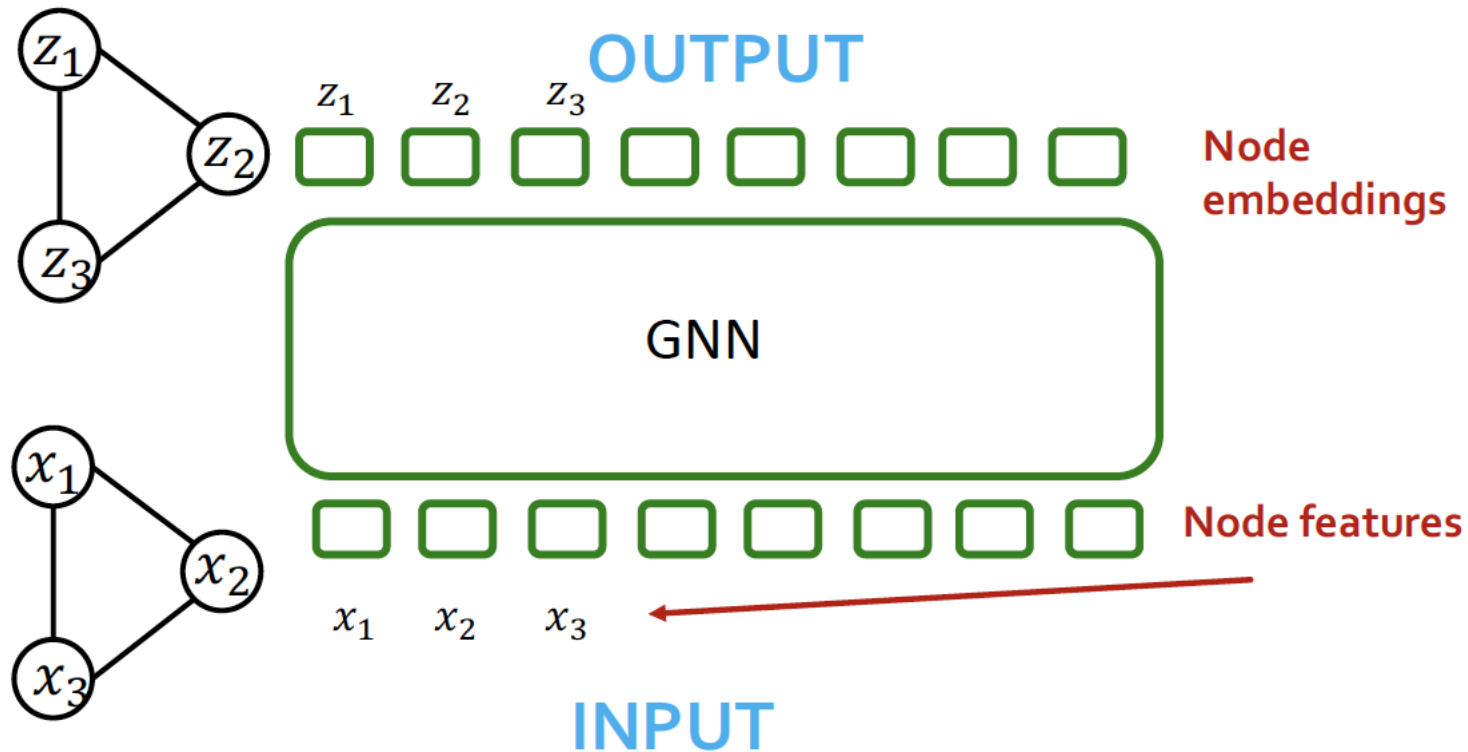
Scaling Up Transformers

Model	Layers	Width	Heads	Params	Data	Training
Transformer-Base	12	512	8	65M		8x P100 (12 hours)
Transformer-Large	12	1024	16	213M		8x P100 (3.5 days)
BERT-Base	12	768	12	110M	13 GB	
BERT-Large	24	1024	16	340M	13 GB	
XLNet-Large	24	1024	16	~340M	126 GB	512x TPU-v3 (2.5 days)
RoBERTa	24	1024	16	355M	160 GB	1024x V100 GPU (1 day)
GPT-2	48	1600	?	1.5B	40 GB	
Megatron-LM	72	3072	32	8.3B	174 GB	512x V100 GPU (9 days)
Turing-NLG	78	4256	28	17B	?	256x V100 GPU
GPT-3	96	12,288	96	175B	694GB	?
Gopher	80	16,384	128	280B	10.55 TB	4096x TPUv3 (38 days)

Transformers for Graph Data

Comparing Transformers with GNN

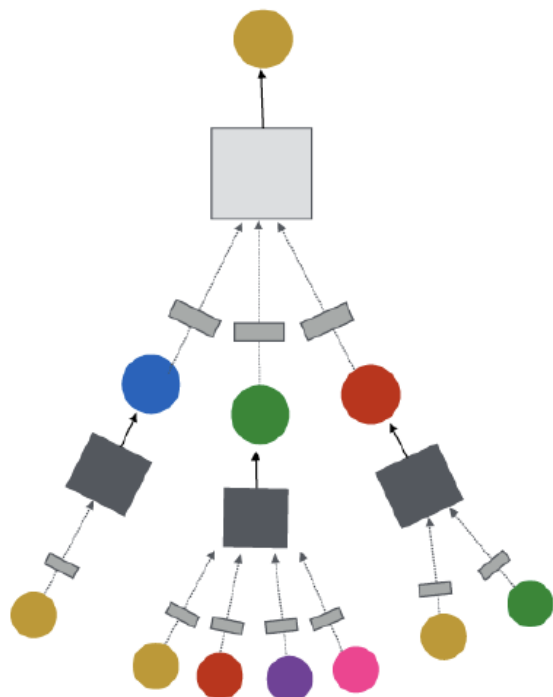
- **Similarity:** GNNs also take in a sequence of vectors (in no particular order) and output a sequence of embeddings
- **Difference:** GNNs use message passing, Transformer uses self-attention



Comparing Transformers with GNN

- **Difference:** GNNs use message passing, Transformer uses self-attention
- Are self-attention and message passing really different?

Message Passing



Vs.

Self-attention

$$\begin{aligned} X &\times W^Q = Q \\ X &\times W^K = K \\ X &\times W^V = V \\ \text{softmax} \left(\frac{Q \times K^T}{\sqrt{d_k}} \right) &\times V = Z \end{aligned}$$

Self-Attention versus Message Passing

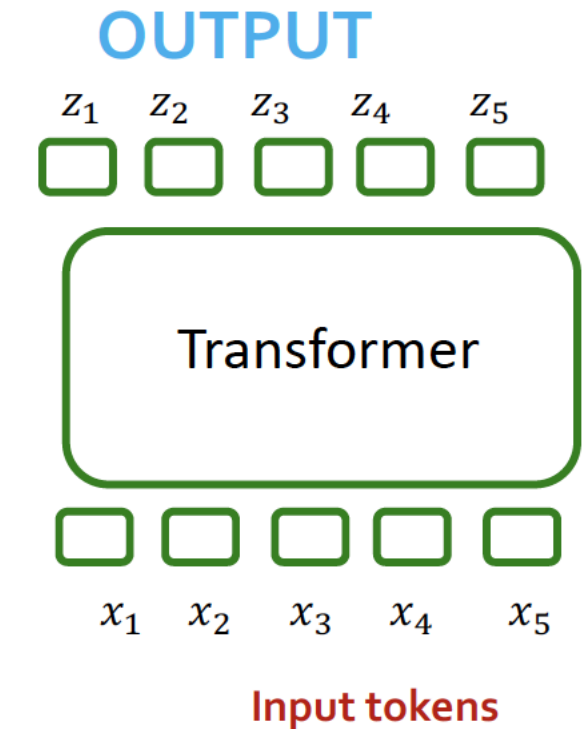
Interpreting the Self-Attention Update

- Recall formula for attention update

$$\begin{aligned} \text{Att}(X) &= \text{softmax}(K^T Q)V \\ &= \text{softmax}\left((XW^K)^T (XW^Q)\right)(XW^V) \end{aligned}$$

Input stored row-wise

$$X = [\cdots x_i \cdots]$$



Interpreting the Self-Attention Update

- Recall formula for attention update

$$\begin{aligned} \text{Att}(X) &= \text{softmax}(K^T Q)V \\ &= \text{softmax}\left((XW^K)^T (XW^Q)\right)(XW^V) \end{aligned}$$

Input stored row-wise
 $X = [\cdots x_i \cdots]$

- This formula gives the embedding for all tokens simultaneously
- What if we simplify to just token x_1 ?

$$z_1 = \sum_{j=1}^5 \text{softmax}_j(q_1^T k_j) v_j \quad \text{How to interpret this?}$$

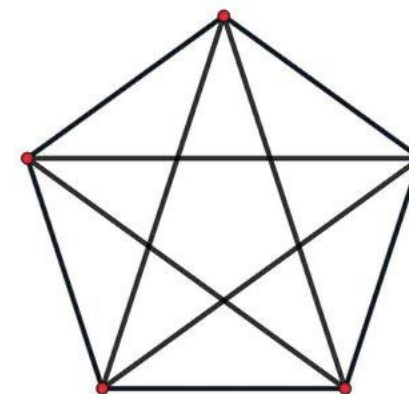
- Steps for computing new embedding for token 1:

- Compute message from j : $MSG(x_j) = (v_j, k_j) = (W^V x_j, W^K x_j)$
- Compute query for 1: $q_1 = W^Q x_1$

- Aggregate all messages: $Agg(\{MSG(x_j): j\}, q_1) = \sum_{j=1}^n \text{softmax}(q_1^T k_j) v_j$

Self-Attention as Message Passing

- Takeaway: Self-attention can be written as message + aggregation – i.e., it is a GNN!
- But so far there is no graph – just tokens.
 - So what graph is this a GNN on?
- Clearly tokens = nodes, but what are the edges?
- Key observation:
 - Token 1 depends on (receives “messages” from) all other tokens
 - The graph is fully connected!
- Alternatively: if you only sum over $j \in N(i)$, you get ~GAT



$$z_1 = \sum_{j=1}^n \text{softmax}(q_1^T k_j) v_j$$

- Steps for computing new embedding for token 1:

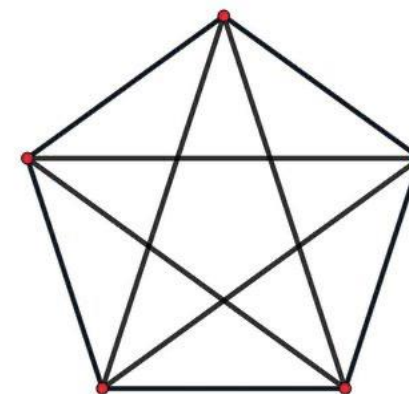
1. Compute message from j: $MSG(x_j) = (v_j, k_j) = (W^V x_j, W^K x_j)$

2. Compute query for 1: $q_1 = W^Q x_1$

3. Aggregate all messages: $Agg(\{MSG(x_j): j\}, q_1) = \sum_{j=1}^n \text{softmax}(q_1^T k_j) v_j$

Self-Attention as Message Passing

- Takeaway 1: Self-attention is a special case of message passing
- Takeaway 2: It is message passing on the fully connected graph
- Takeaway 3: Given a graph G , if you constrain the self-attention softmax to only be over j adjacent to i nodes, you get $\sim$ GAT!



- Steps for computing new embedding for token 1:

1. Compute message from j : $MSG(x_j) = (v_j, k_j) = (W^V x_j, W^K x_j)$

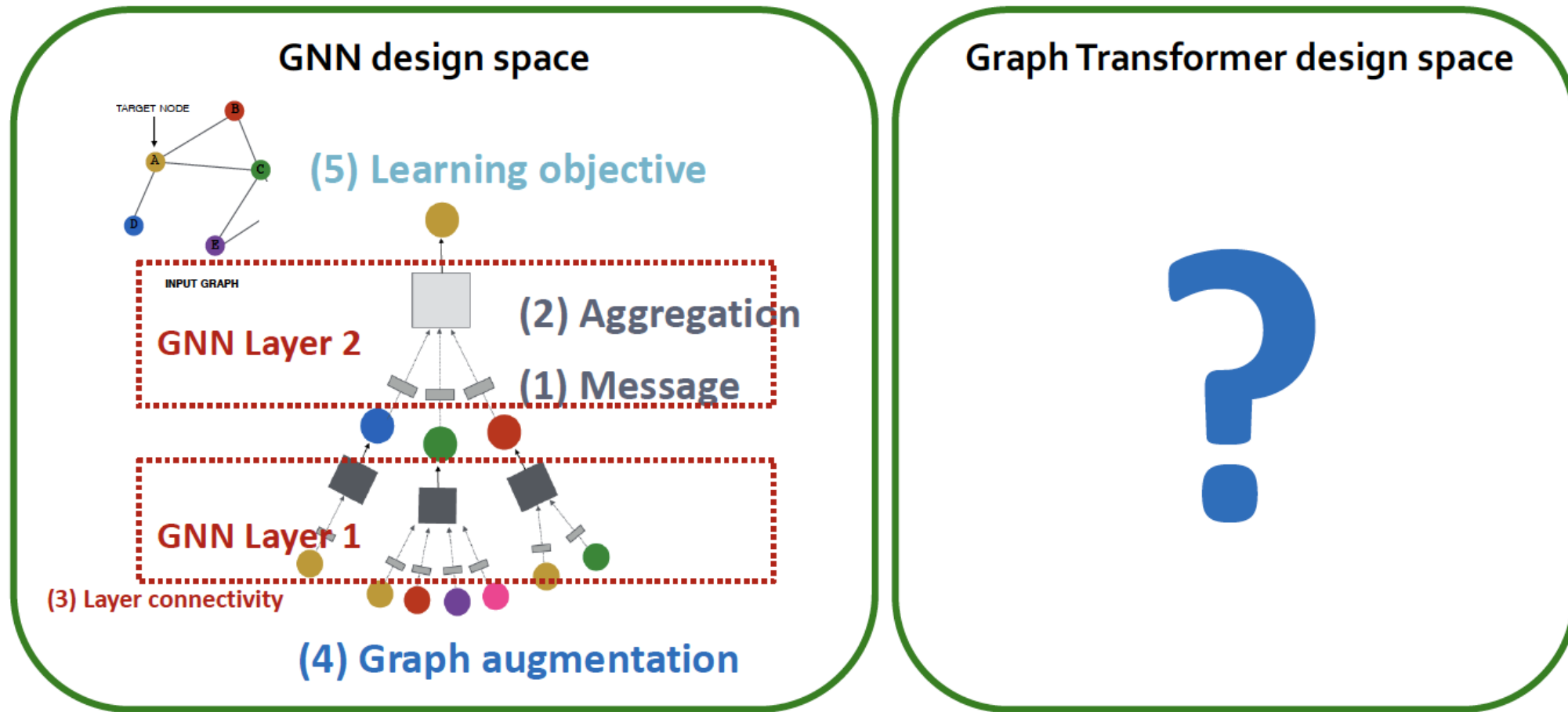
2. Compute query for 1: $q_1 = W^Q x_1$

3. Aggregate all messages: $Agg(\{MSG(x_j): j\}, q_1) = \sum_{j=1}^n softmax(q_1^T k_j) v_j$

A New Design Landscape for Graph Transformer

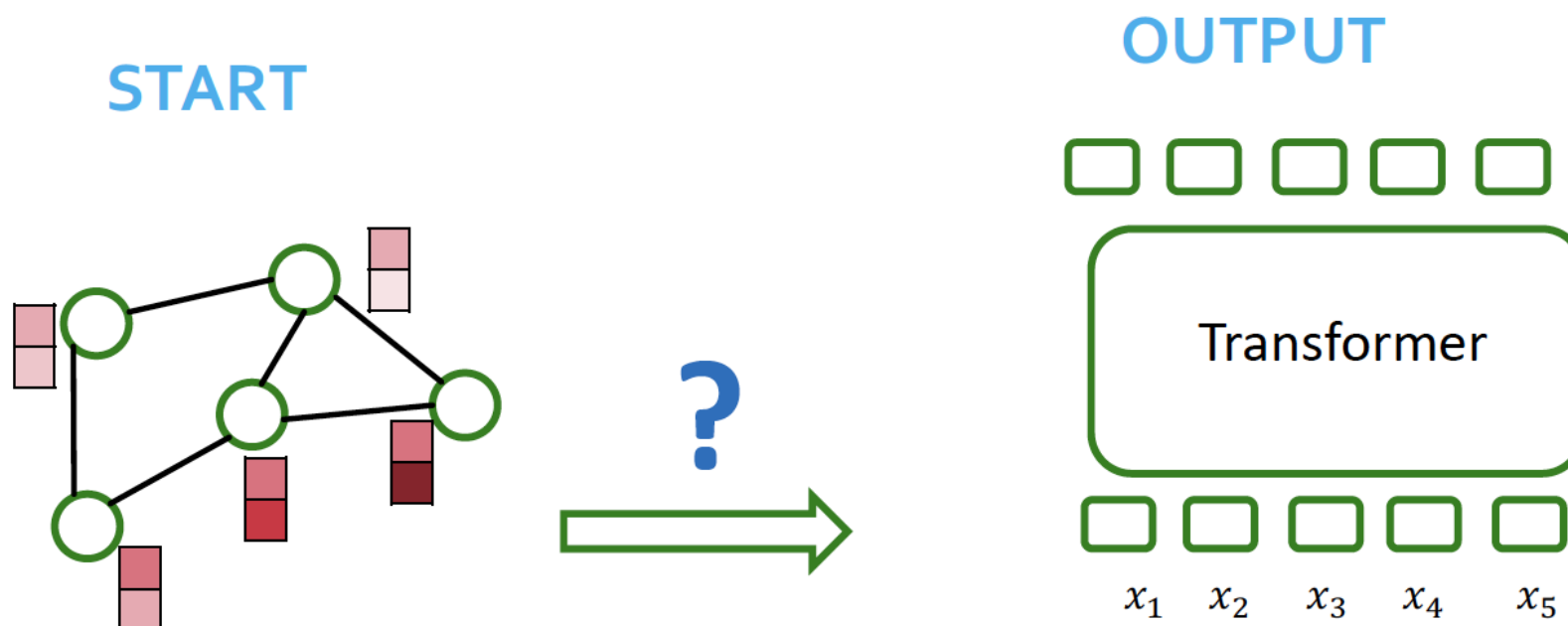
Recap: A General GNN Framework

- We know a lot about the design space of GNNs
- What does the corresponding design space for Graph Transformers look like?



Processing Graphs with Transformer

- We start with graph(s)
- How to input a graph into a Transformer?



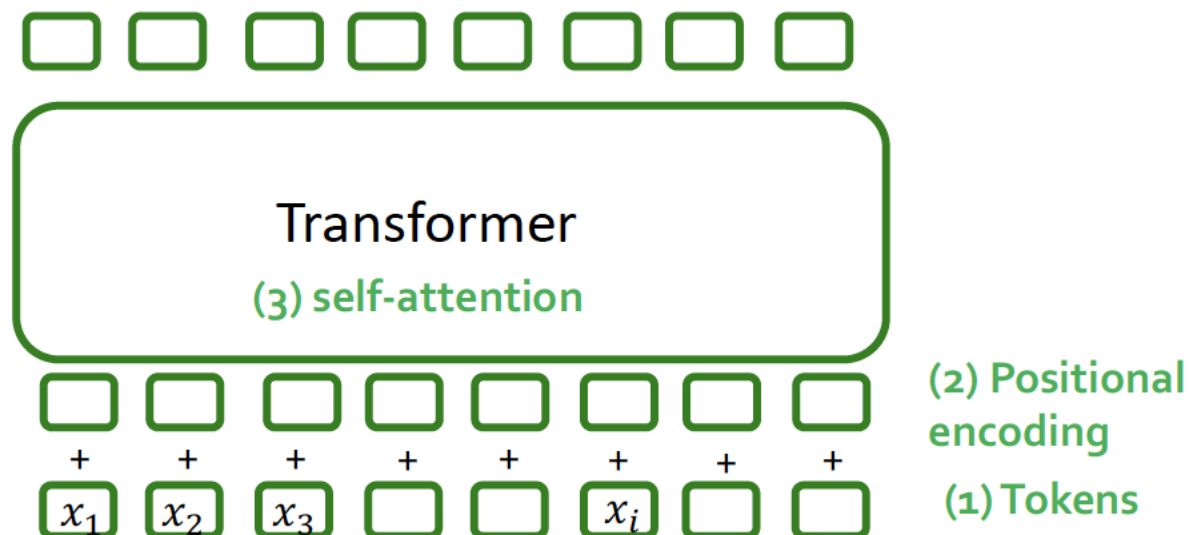
Components of a Transformer

- Key components of Transformer

1. tokenizing
2. positional encoding
3. self-attention

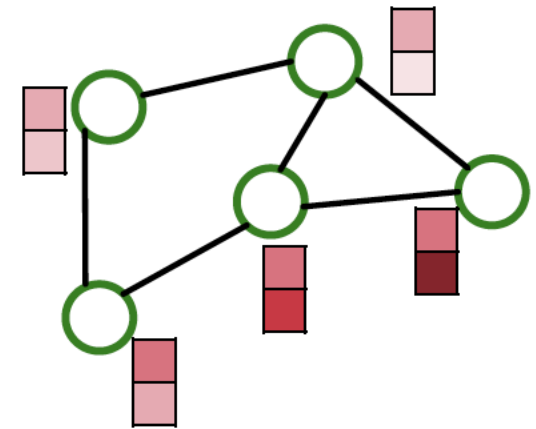
*How to chose these
for graph data?*

- Key question: What should these be for a graph input?



Processing Graphs with Transformers

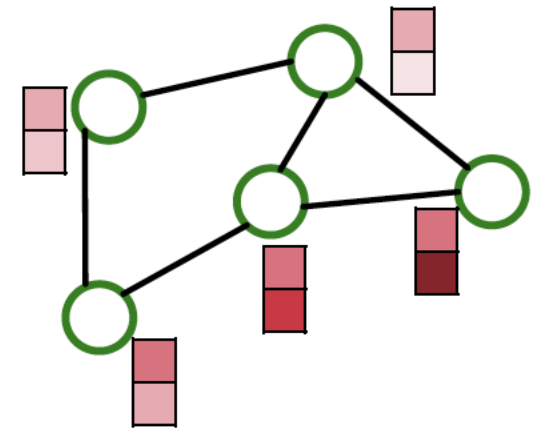
- A graph Transformer must take the following inputs:
 1. Node features?
 2. Adjacency information?
 3. Edge features?
- Key components of Transformer
 1. tokenizing
 2. positional encoding
 3. self-attention
- There are many ways to do this
- Different approaches correspond to different “matchings” between graph inputs (1), (2), (3) transformer components (1), (2), (3)



Processing Graphs with Transformers

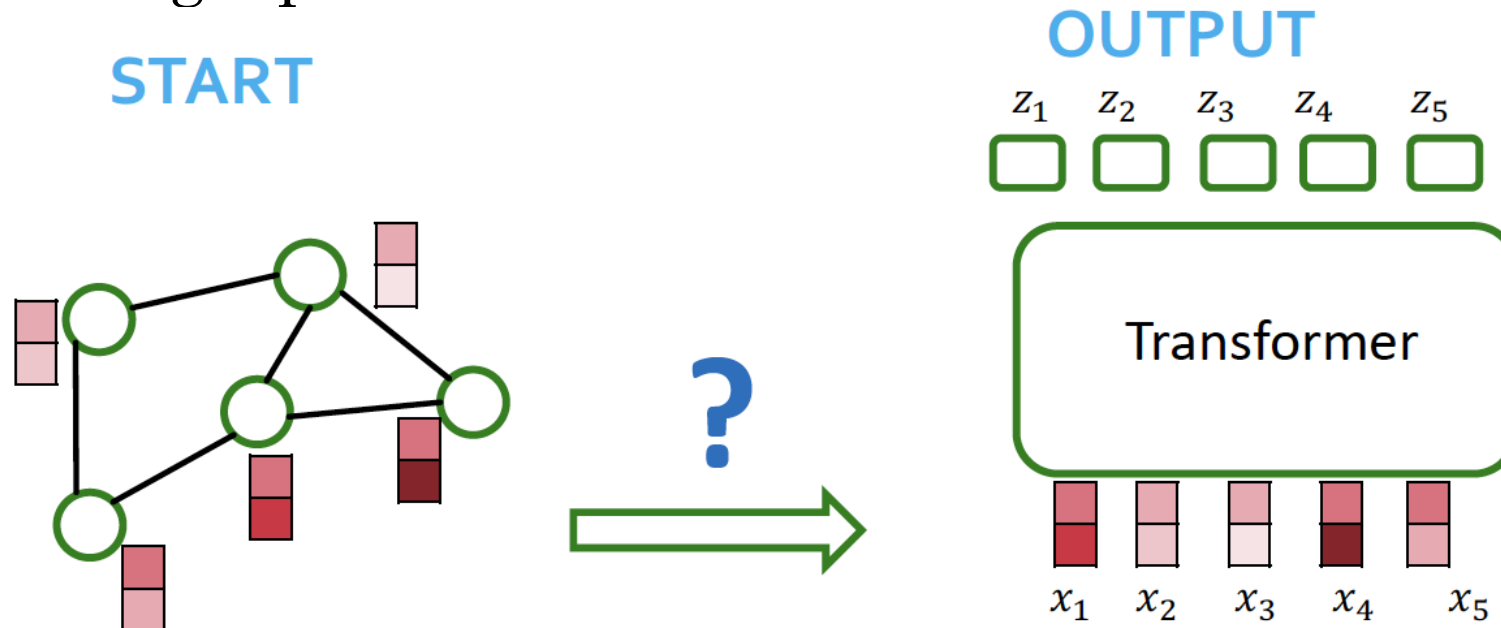
- A graph Transformer must take the following inputs:
 1. Node features?
 2. Adjacency information?
 3. Edge features?
 - Key components of Transformer
 1. tokenizing
 2. positional encoding
 3. self-attention
- Today

- There are many ways to do this
- Different approaches correspond to different “matchings” between graph inputs (1), (2), (3) transformer components (1), (2), (3)



Nodes as Tokens

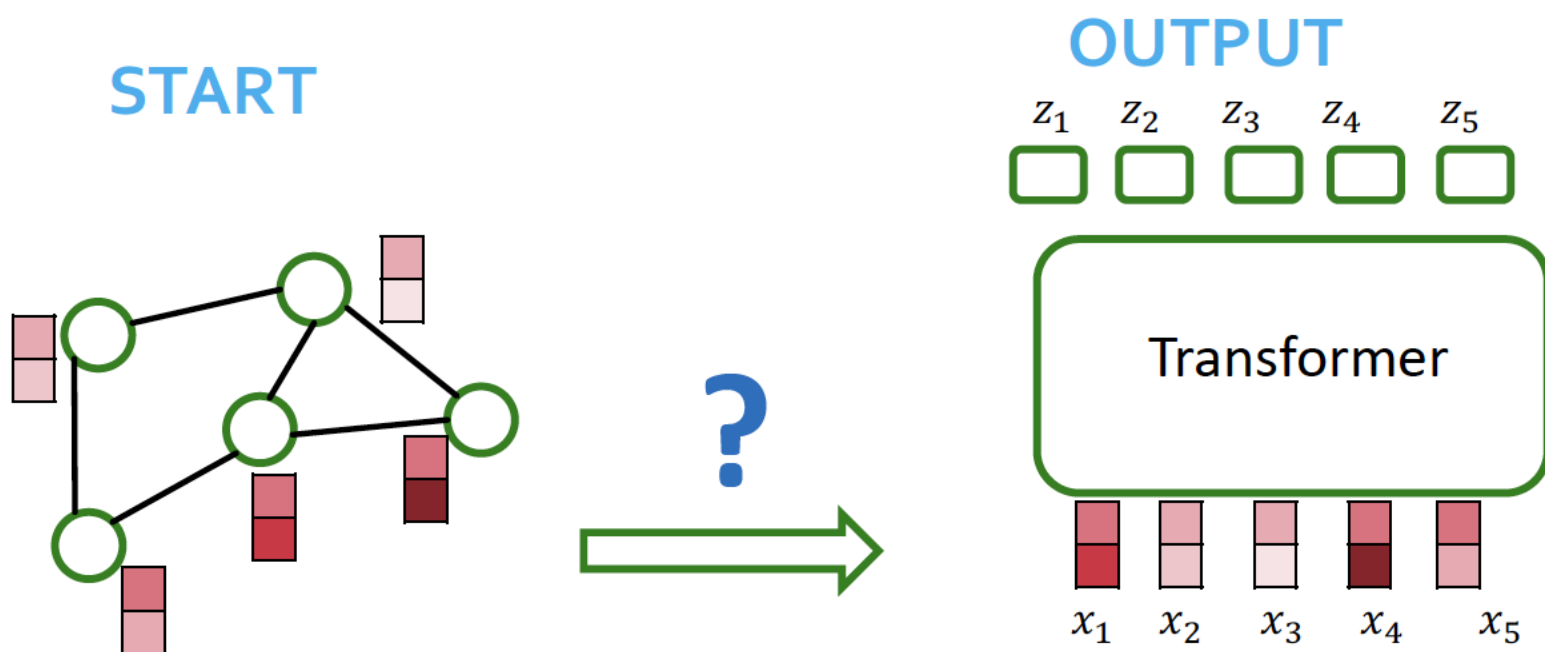
- Q1: what should our tokens be?
- **Sensible Idea:** node features = input tokens
- This matches the setting for the “attention is message passing on the fully connected graph” observation



(1) Input tokens = Node features

Processing Graphs with Transformers

- Problem? We completely lose adjacency info!
- How to also inject adjacency information?

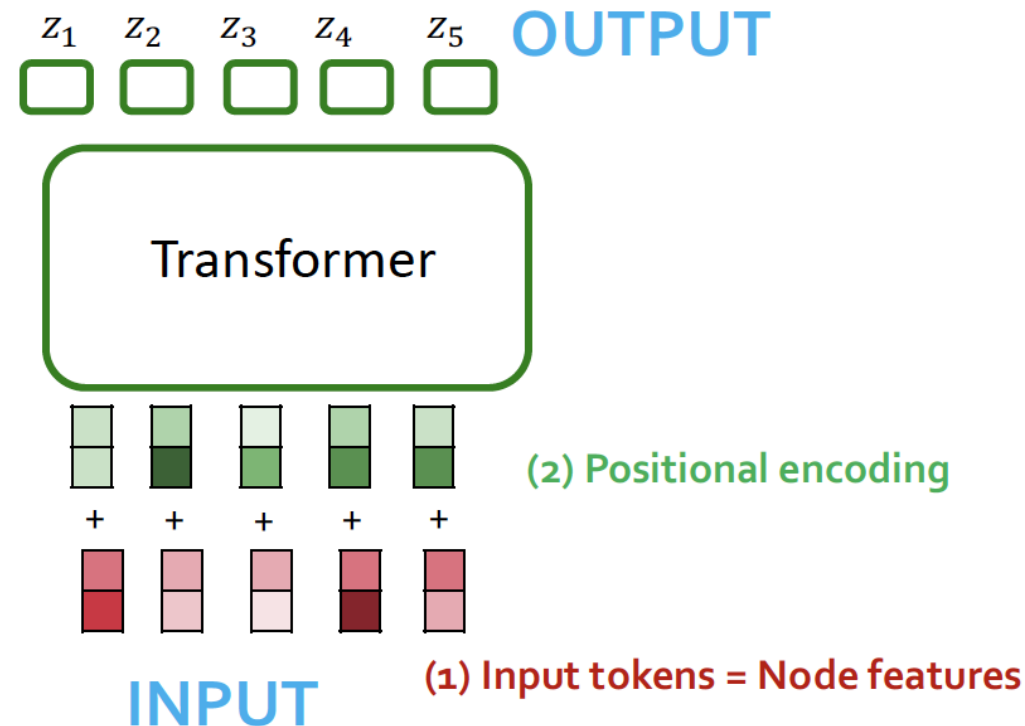
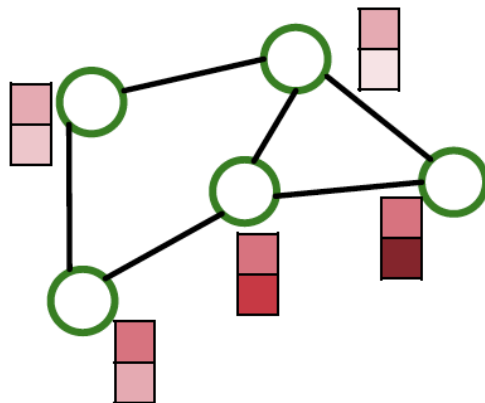


(1) Input tokens = Node features

How to Add Back Adjacency Info?

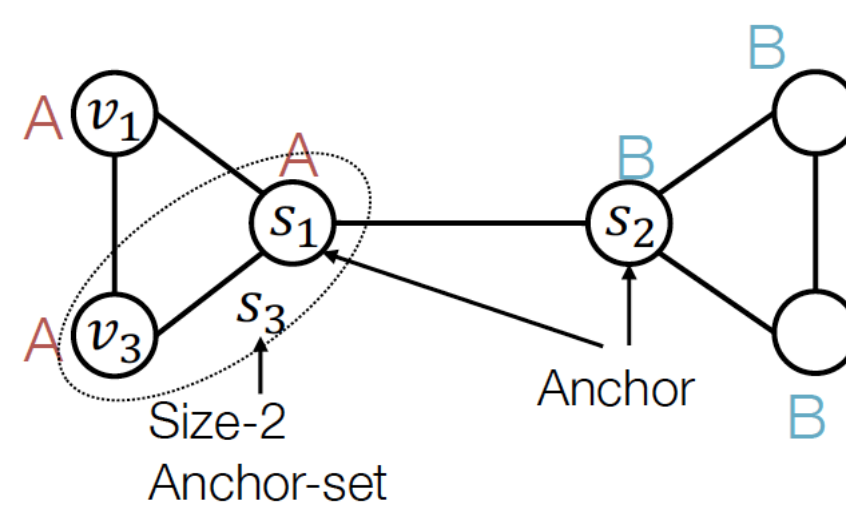
- Idea: Encode adjacency info in the **positional encoding** for each node
- Positional encoding describes **where** a node is in the graph

How to design a good positional encoding?



Option 1: Relative Distances

- Positional encoding based on relative distances
- Similar methods based on **random walks**
- **This is a good idea!** It works well in many cases
- Especially strong for tasks that require **counting cycles**



Positional encoding for node v_1



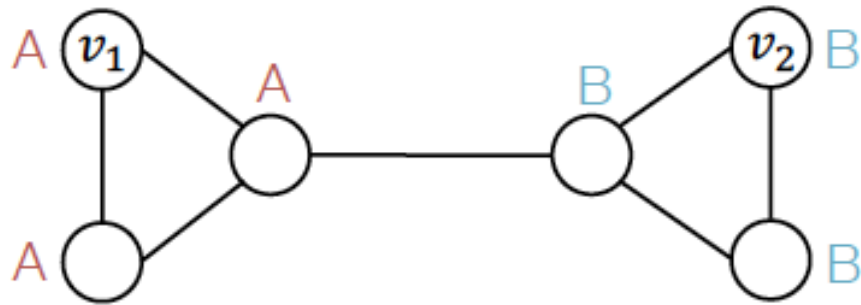
=

Relative Distances				
	s_1	s_2	s_3	
v_1	1	2	1	
v_3	1	2	0	

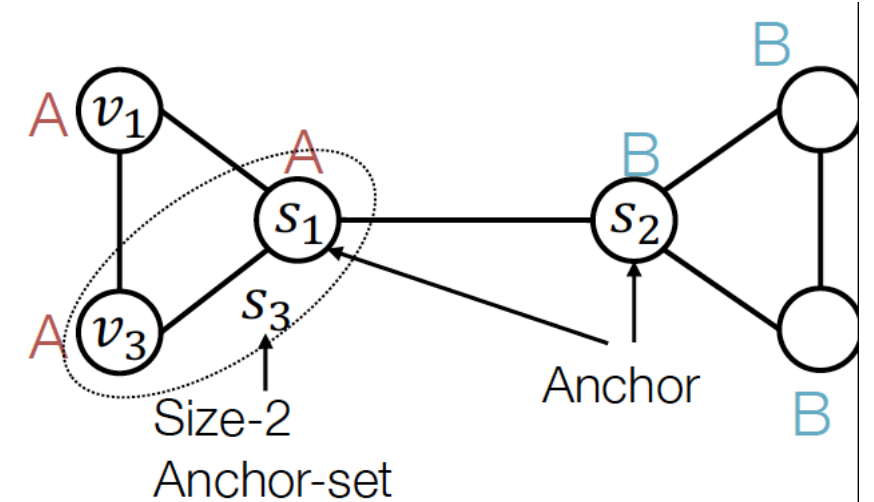
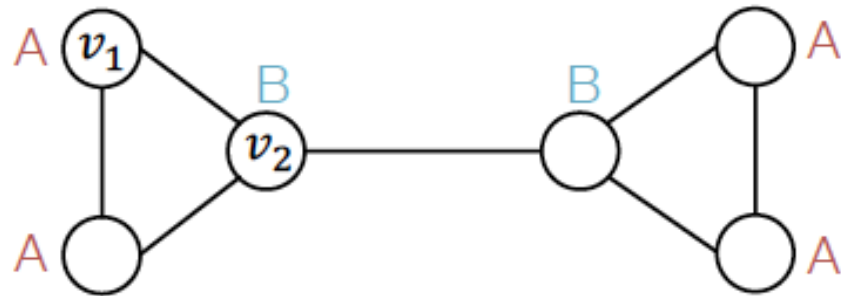
Anchor s_1, s_2 cannot differentiate node v_1, v_3 , but anchor-set s_3 can

Option 1: Relative Distances

- Relative distances useful for position-aware task



- But not suited to structure-aware tasks



Positional encoding for node v_1



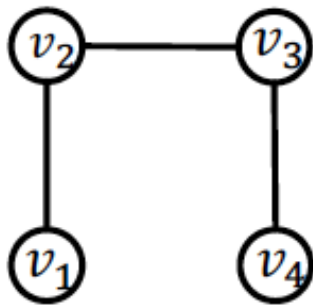
=

Relative Distances				
	s_1	s_2	s_3	
v_1	1	2	1	
v_3	1	2	0	

Anchor s_1, s_2 cannot differentiate node v_1, v_3 , but anchor-set s_3 can

Option 2: Laplacian Eigenvector Positional Encodings

- What other ways to make positional encoding?
- Draw on knowledge of **Graph Theory** (many useful and powerful tools)
- **Key object:** Laplacian Matrix $L = \text{Degrees} - \text{Adjacency}$
 - Each graph has its own Laplacian matrix
 - Laplacian encodes the graph structure
 - Several Laplacian variants that add degree information differently



$L =$

1	0	0	0
0	2	0	0
0	0	2	0
0	0	0	1

Degree of each node

$-$

0	1	0	0
1	0	1	0
0	1	0	1
0	0	1	0

Adjacency

Laplacian Eigenvector Positional Encodings

- Laplacian matrix captures graph structure
- Its eigenvectors inherit this structure
- This is important because eigenvectors are vectors (!) and so can be fed into a Transformer
- Eigenvectors with small eigenvalue = local structure, large eigenvalue = global symmetries

Refresher

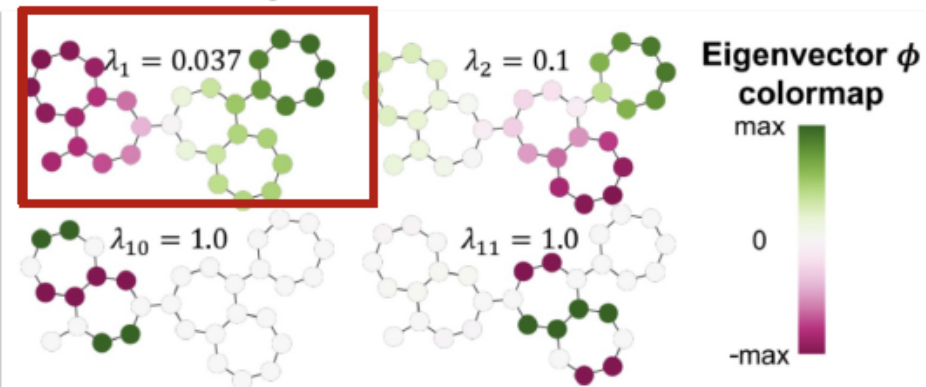
Eigenvector: v such that $Lv = \lambda v$

$L: n \times n$ matrix

$v: n$ dimensional vector

λ : Scalar eigenvalue

Visualize one eigenvector

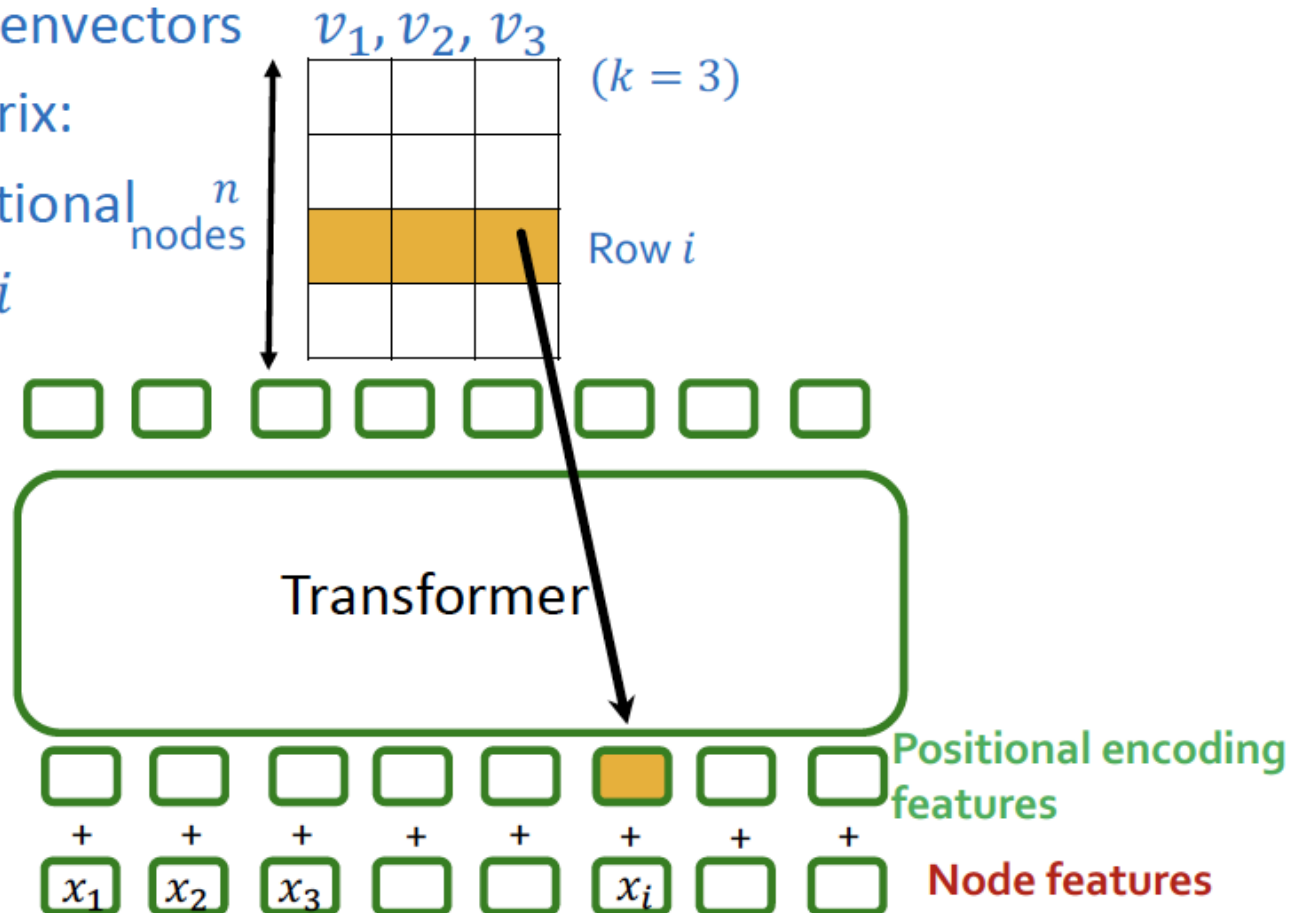
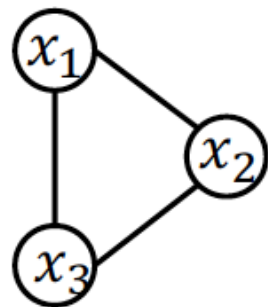


(Figure from Kreuzer* and Beaini* et al. 2021)

Laplacian Eigenvector Positional Encodings

Positional encoding steps:

- 1. compute k eigenvectors
- 2. Stack into matrix:
- 3. i th row is positional encoding for node i



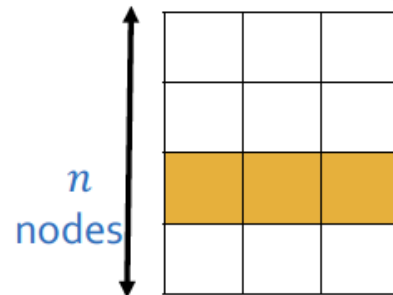
Summary: Laplacian Eigenvector Positional Encodings

- Laplacian Matrix $L = \text{Degrees} - \text{Adjacency}$

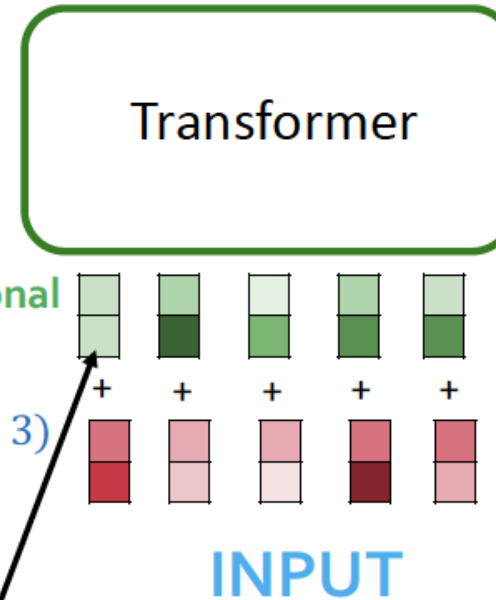
- Eigenvector: v such that $Lv = \lambda v$

- **Positional encoding steps:**

- 1. compute k eigenvectors v_1, v_2, v_3 ($k = 3$)
- 2. Stack into matrix:
- 3. i th row is positional encoding for node i



(2) Positional encoding



- Laplacian Eigenvector positional encodings can also be used with message-passing GNNs
 - This helps for same reasons as relative-distance based positional encodings in previous lecture

Laplacian Eigenvectors in Practice

- Task: given a graph, predict YES if it has a cycle, NO otherwise
- Recall, message-passing cannot solve this task!
- “PE” indicates using Laplacian Eigenvector Pos. Enc.

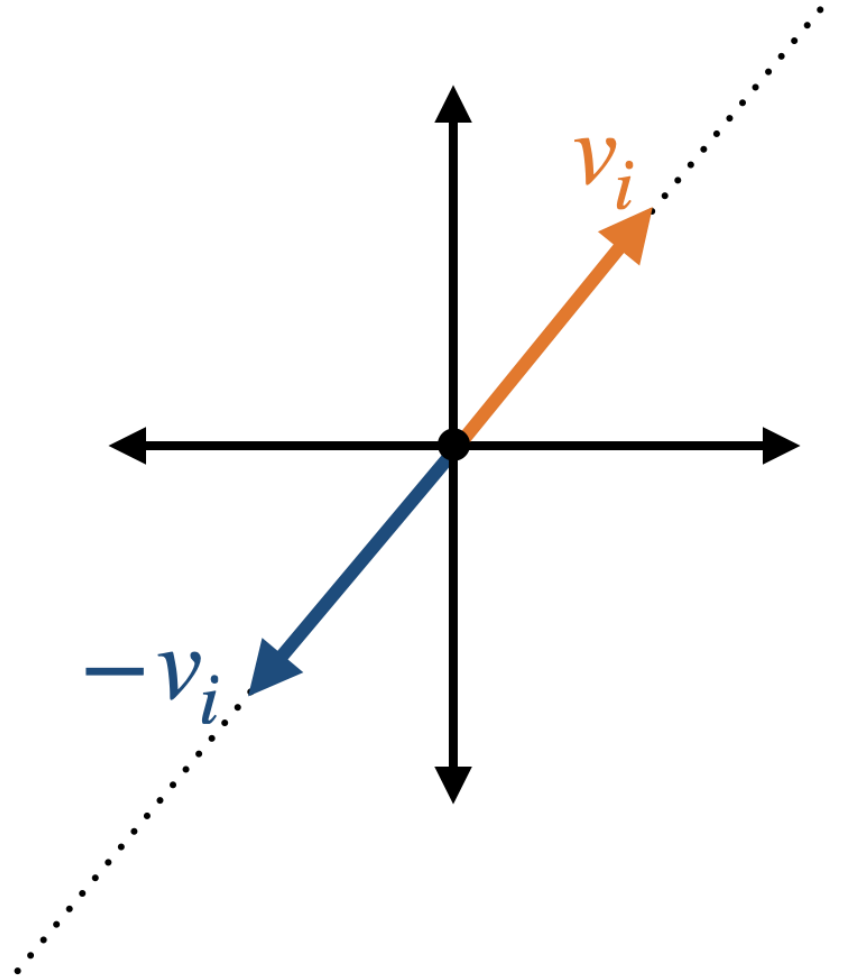
Train samples →			200	500	1000	5000
Model	L	#Param	Test Acc±s.d.			
GIN	4	100774	70.585±0.636	74.995±1.226	78.083±1.083	86.130±1.140
GIN-PE	4	102864	86.720±3.376	95.960±0.393	97.998±0.300	99.570±0.089
GatedGCN	4	103933	50.000±0.000	50.000±0.000	50.000±0.000	50.000±0.000
GatedGCN-PE	4	105263	95.082±0.346	96.700±0.381	98.230±0.473	99.725±0.027

Laplacian Eigenvector Positional Encodings

- Laplacian Eigenvector positional encodings work!
- But is this the best we can do?
 - Hint: no
- Q: What is the problem with the current approach?
 - A1: Eigenvectors are not arbitrary vectors
 - A2: They have special structure that we have been ignoring!
- To use eigenvectors properly we must account for their structure in our models

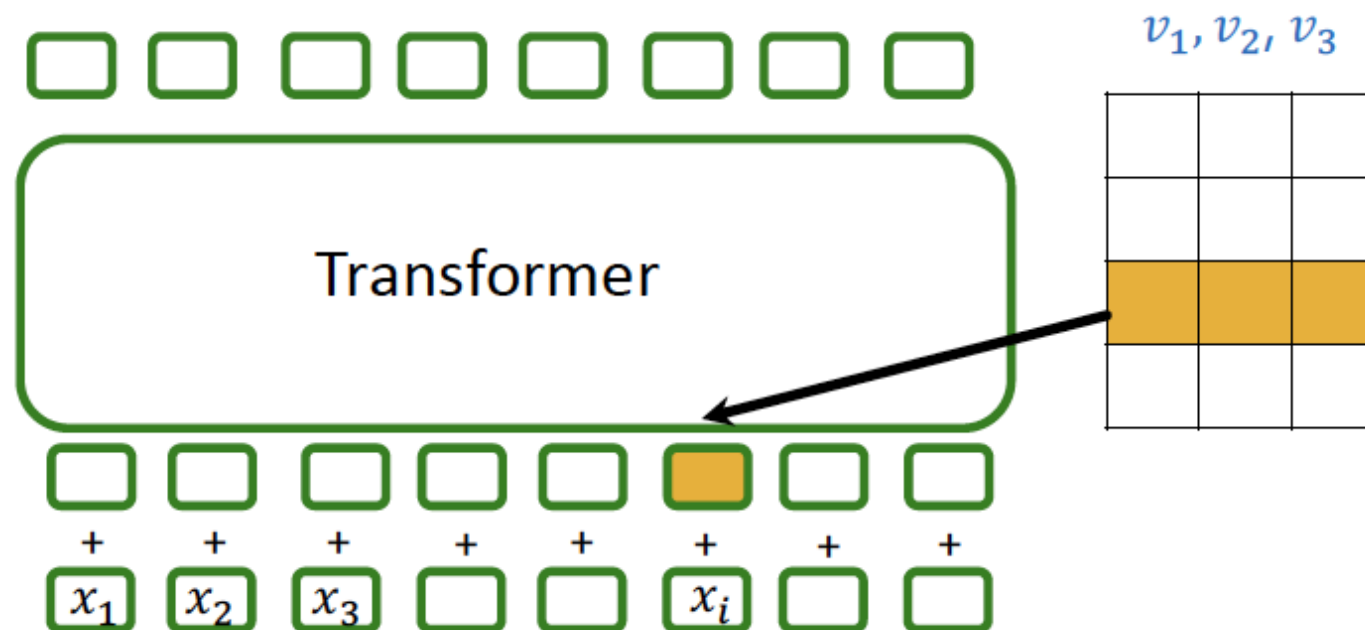
Eigenvector Sign Ambiguity

- Suppose v is a Laplacian eigenvector
 - So $Lv = \lambda v$
- But this means:
 - Also $L(-v) = \lambda(-v)$
- So $-v$ is also a Laplacian eigenvector
- The choice of sign is arbitrary!



Sign Ambiguity is a Problem

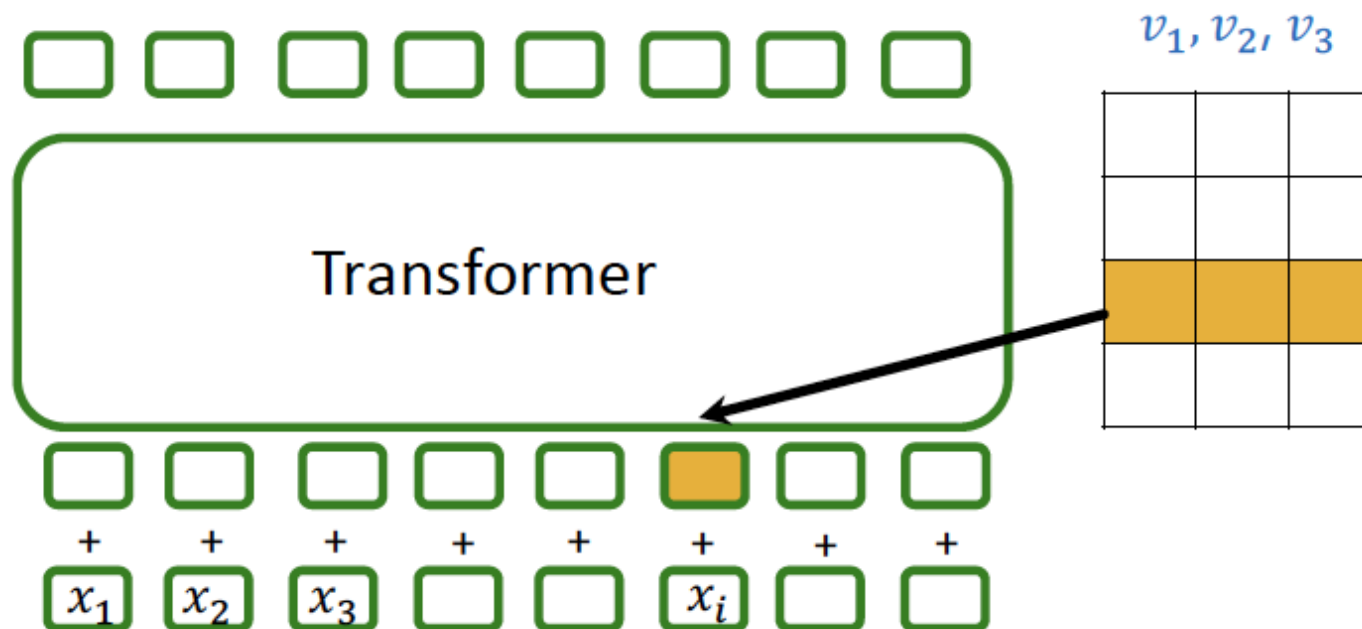
- Both v and $-v$ are eigen vectors
- But when we use them as positional encodings we pick one arbitrarily
- Why does this matter for positional encodings?



Sign Ambiguity is a Problem

- Both v and $-v$ are eigen vectors
- But when we use them as positional encodings we pick one arbitrarily
- Why does this matter for positional encodings?

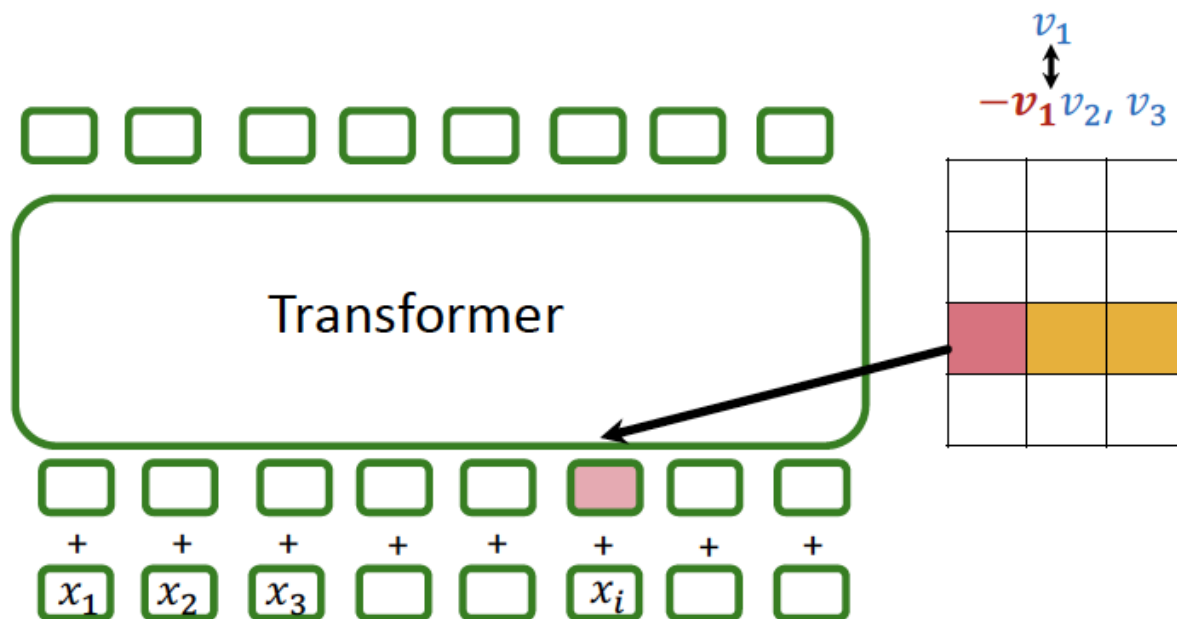
*What if we
picked the
other sign?*



Sign Ambiguity is a Problem

- What if we picked the other sign choice?
- Then the input PE changes
- => The models predictions will change!
- For k eigenvectors there are 2^k sign choices

2^k different
predictions for the
same input graph!

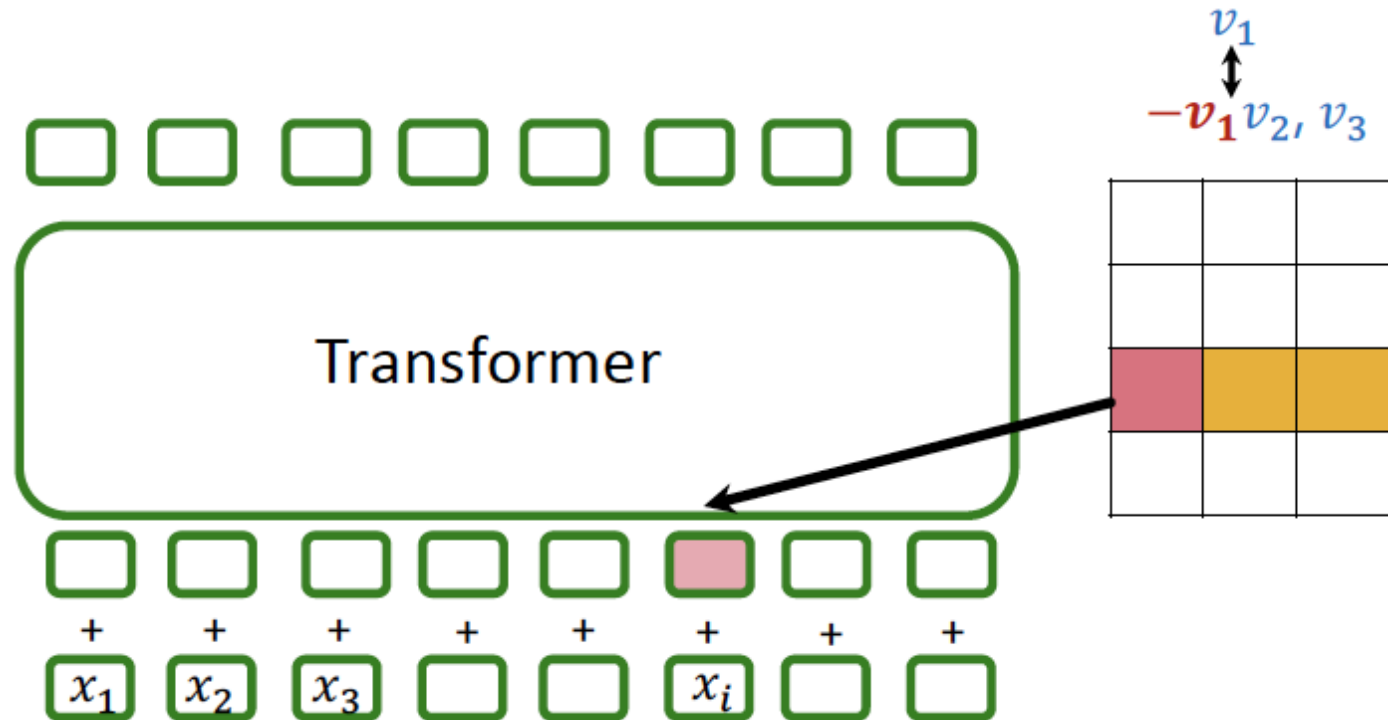


How to Fix Sign Ambiguity

- **Simple Idea:** randomly flip the signs of eigenvectors during training
 - I.e., data augmentation
 - Model will learn to not use the sign information
 - **Issue: exponentially many sign choices is very difficult to learn**

How to Fix Sign Ambiguity

- **Better Idea:** build a neural network that is **invariant** to sign choices!
- Since it is invariant, the predictions will no longer depend on the sign choice



Sign Invariant Neural Networks

- **Goal:** design a neural network $f(v_1, v_2, \dots, v_k)$ such that:
 - $f(v_1, v_2, \dots, v_k) = f(\pm v_1, \pm v_2, \dots, \pm v_k)$ for all $\pm$ choices
 - f is “expressive”: note that $f(v_1, v_2, \dots, v_k) = 0$ is sign invariant... but it’s a terrible neural network architecture
- **Warmup: one eigenvector**
 - What about $f(v_1)$ such that $f(v_1) = f(-v_1)$?

Sign Invariant Neural Networks

- Warmup: one eigenvector
- Goal: design a neural network $f(v_1)$ such that $f(v_1) = f(-v_1)$
- Proposition: f satisfies $f(v_1) = f(-v_1)$ if and only if there is a ϕ such that $f(v_1) = \phi(v_1) + \phi(-v_1)$

Sign Invariant Neural Networks

- Warmup: one eigenvector
- Goal: design a sign invariant neural network $f(v_1, v_2, \dots, v_k)$ in two steps:
 - Step 1: sign invariant $f_i(v_i)$ for each i
 - Step 2: COMBINE individual eigenvector embeddings into one:

$$f(v_1, v_2, \dots, v_k) = AGG(f_1(v_1), \dots, f_k(v_k))$$

Sign Invariant Neural Networks

- Warmup: one eigenvector
- Goal: design a sign invariant neural network $f(v_1, v_2, \dots, v_k)$ in two steps:
 - Step 1: sign invariant $f_i(v_i)$ for each i
 - Step 2: COMBINE individual eigenvector embeddings into one:

$$f(v_1, v_2, \dots, v_k) = AGG(f_1(v_1), \dots, f_k(v_k))$$

Use model for one eigenvector

$$f(v_1, v_2, \dots, v_k)$$

$$= AGG(\phi_1(v_1), +\phi_1(-v_1), \dots, \phi_k(v_k), +\phi_k(-v_k))$$

Combine using another neural net $AGG = \rho$

Sign Invariant Neural Networks

- Overall model:

$$f(v_1, v_2, \dots, v_k) = \rho((\phi_1(v_1), +\phi_1(-v_1), \dots, \phi_k(v_k), +\phi_k(-v_k)))$$

- Introducing k distinct neural nets is costly...
- Let's minimize extra parameters by **sharing one ϕ**

$$\begin{aligned} f(v_1, v_2, \dots, v_k) \\ = \rho(\phi(v_1), +\phi(-v_1), \dots, \phi(v_k), +\phi(-v_k)) \end{aligned}$$

ρ, ϕ = any neural network
(MLP, GNN etc.)

SignNet

Sign Invariant Neural Networks

- **Recall Goal:** design a neural network $f(v_1, v_2, \dots, v_k)$ such that:
 - $f(v_1, v_2, \dots, v_k) = f(\pm v_1, \pm v_2, \dots, \pm v_k)$ for all $\pm$ choices
 - **SignNet is sign invariant.**
 - f is “expressive”
 - **Is SignNet expressive?**

$$f(v_1, v_2, \dots, v_k) = \rho(\phi(v_1), +\phi(-v_1), \dots, \phi(v_k), +\phi(-v_k))$$

ρ, ϕ = any neural network
(MLP, GNN etc.)

SignNet

Sign Invariant Neural Networks

- **Recall Goal:** design a neural network $f(v_1, v_2, \dots, v_k)$ such that:
 - $f(v_1, v_2, \dots, v_k) = f(\pm v_1, \pm v_2, \dots, \pm v_k)$ for all $\pm$ choices
 - **SignNet is sign invariant.**
 - f is “expressive”
 - **Is SignNet expressive?**

Theorem: if f is sign invariant, then there exist functions ρ, ϕ such that

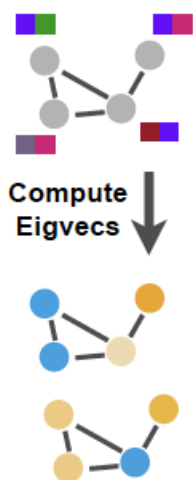
$$\begin{aligned} f(v_1, v_2, \dots, v_k) \\ = \rho(\phi(v_1), +\phi(-v_1), \dots, \phi(v_k), +\phi(-v_k)) \end{aligned}$$

SignNet can express all sign invariant functions!!

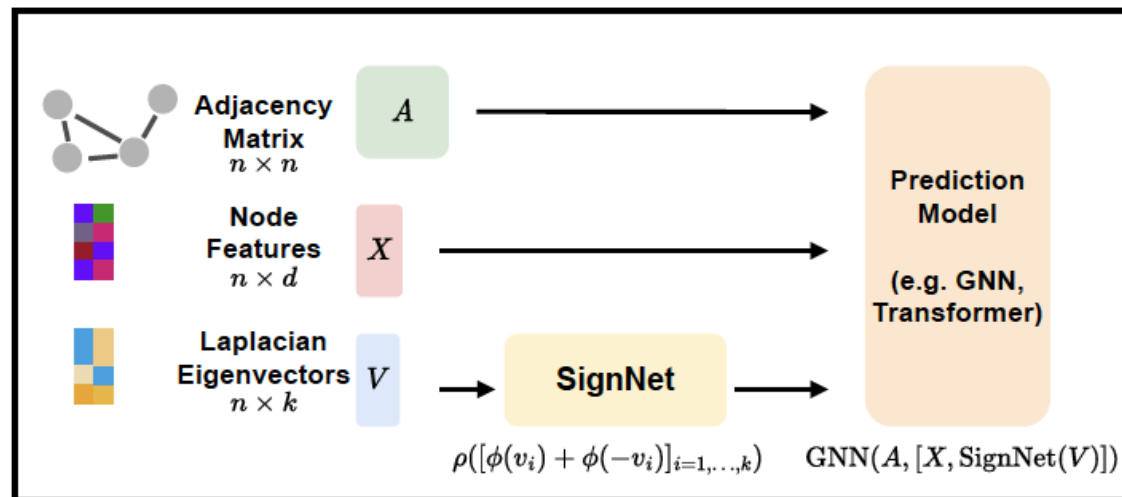
SignNet in Practice

- How to use SignNet in practice?
 - Step 1: Compute eigenvectors
 - Step 2: Get eigenvector embeddings using SignNet
 - Step 3: Concatenate SignNet embeddings with node features X
 - Step 4: Pass through main GNN/Transformer as usual.
 - Step 5: Backpropagate gradients to train SignNet + Prediction model jointly.

Input Graph



Model

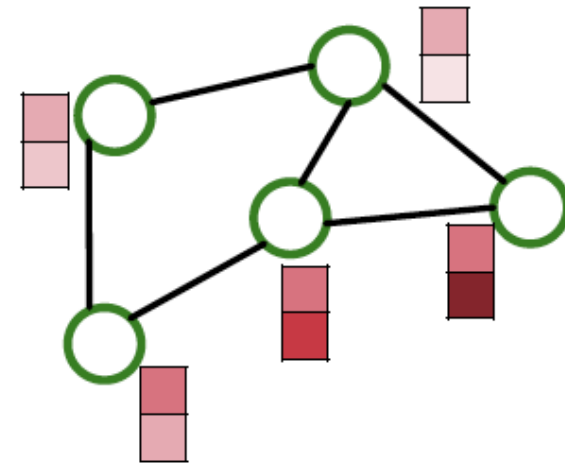


Processing Graphs with Transformers

- A graph Transformer must take the following inputs:
 - (1) Node features?
 - (2) Adjacency information?
 - (3) Edge features?
- Key components of Transformer
 - (1) tokenizing
 - (2) positional encoding
 - (3) self-attention

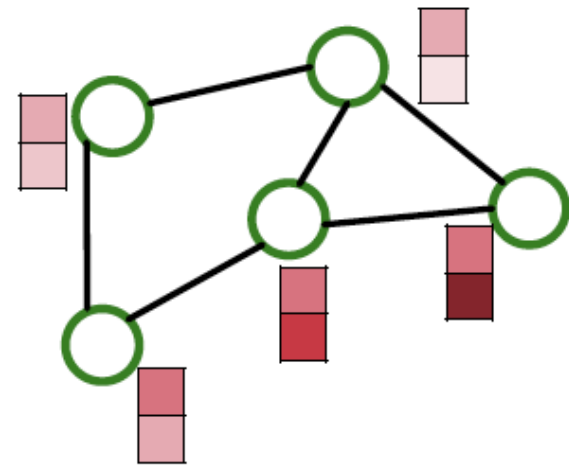
So far

- There are many ways to do this
- Different approaches correspond to different “matchings” between graph inputs (1), (2), (3) transformer components (1), (2), (3)



Processing Graphs with Transformers

- A graph Transformer must take the following inputs:
 - (1) Node features?
 - (2) Adjacency information?
 - (3) Edge features?
 - Key components of Transformer
 - (1) tokenizing
 - (2) positional encoding
 - (3) self-attention
- Left to do**
- There are many ways to do this
 - Different approaches correspond to different “matchings” between graph inputs (1), (2), (3) transformer components (1), (2), (3)



Edge Features in Self-Attention

- Not clear how to add edge features in the tokens or positional encoding
- How about in the attention? $Att(X) = softmax(K^T Q)V$
- $[k_{ij}] = K^T Q$ is an $n \times n$ matrix. Entry k_{ij} describes “how much” token j contributes to the update of token i

Edge Features in Self-Attention

- Not clear how to add edge features in the tokens or positional encoding
- How about in the attention? $Att(X) = softmax(K^T Q)V$
- $[k_{ij}] = K^T Q$ is an $n \times n$ matrix. Entry k_{ij} describes “how much” token j contributes to the update of token i
- Idea: adjust k_{ij} based on edge features. Replace with $k_{ij} + c_{ij}$ where c_{ij} depends on the edge features

Edge Features in Self-Attention

- Not clear how to add edge features in the tokens or positional encoding
- How about in the attention? $Att(X) = softmax(K^T Q)V$
- $[k_{ij}] = K^T Q$ is an $n \times n$ matrix. Entry k_{ij} describes “how much” token j contributes to the update of token i
- Idea: adjust k_{ij} based on edge features. Replace with $k_{ij} + c_{ij}$ where c_{ij} depends on the edge features

Implementation:

Learned parameters w_1

- If there is an edge between i and j with features e_{ij} , define $c_{ij} = w_1^T e_{ij}$
- If there is no edge, find shortest edge path between i and j $(e^1, e^2, \dots, e^N)$ and define $c_{ij} = \sum_n w_n^T e^n$

Learned parameters $w_1, \dots, w_N$

Summary: Graph Transformer Design Space

▪ (1) Tokenization

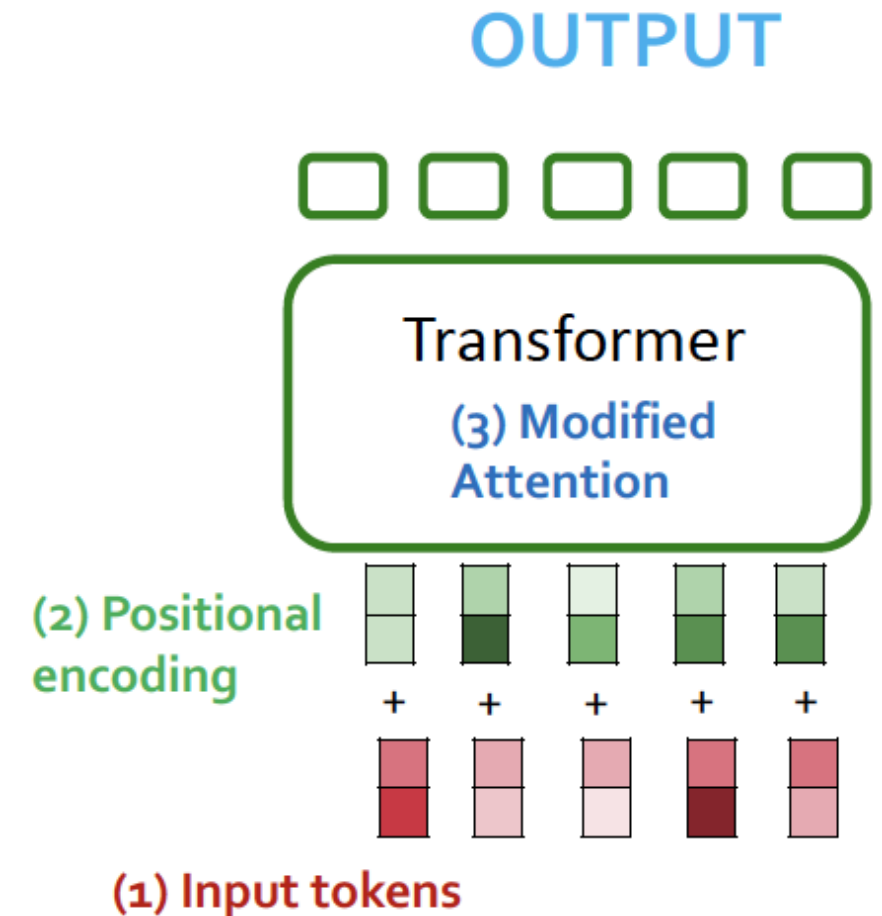
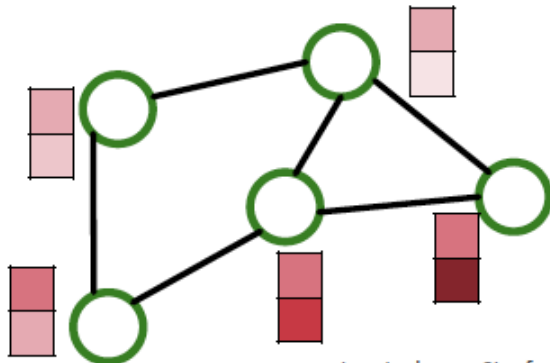
- Usually node features
- Other options, such as subgraphs, and node + edge features (not discussed today)

▪ (2) Positional Encoding

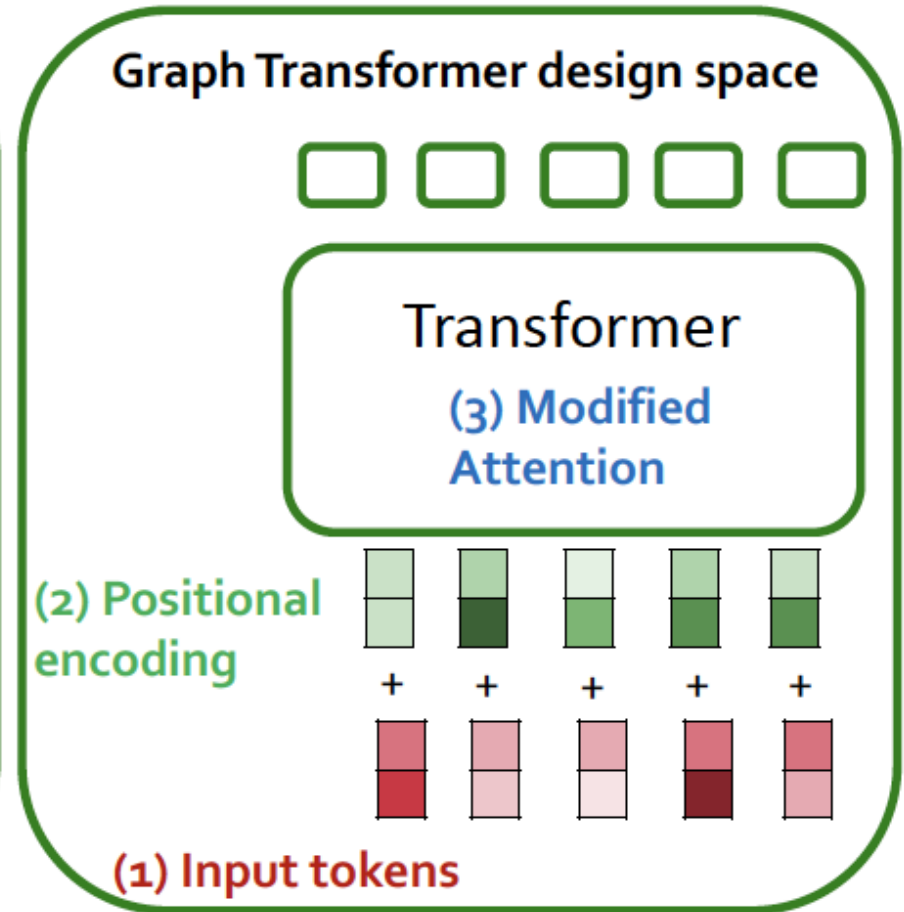
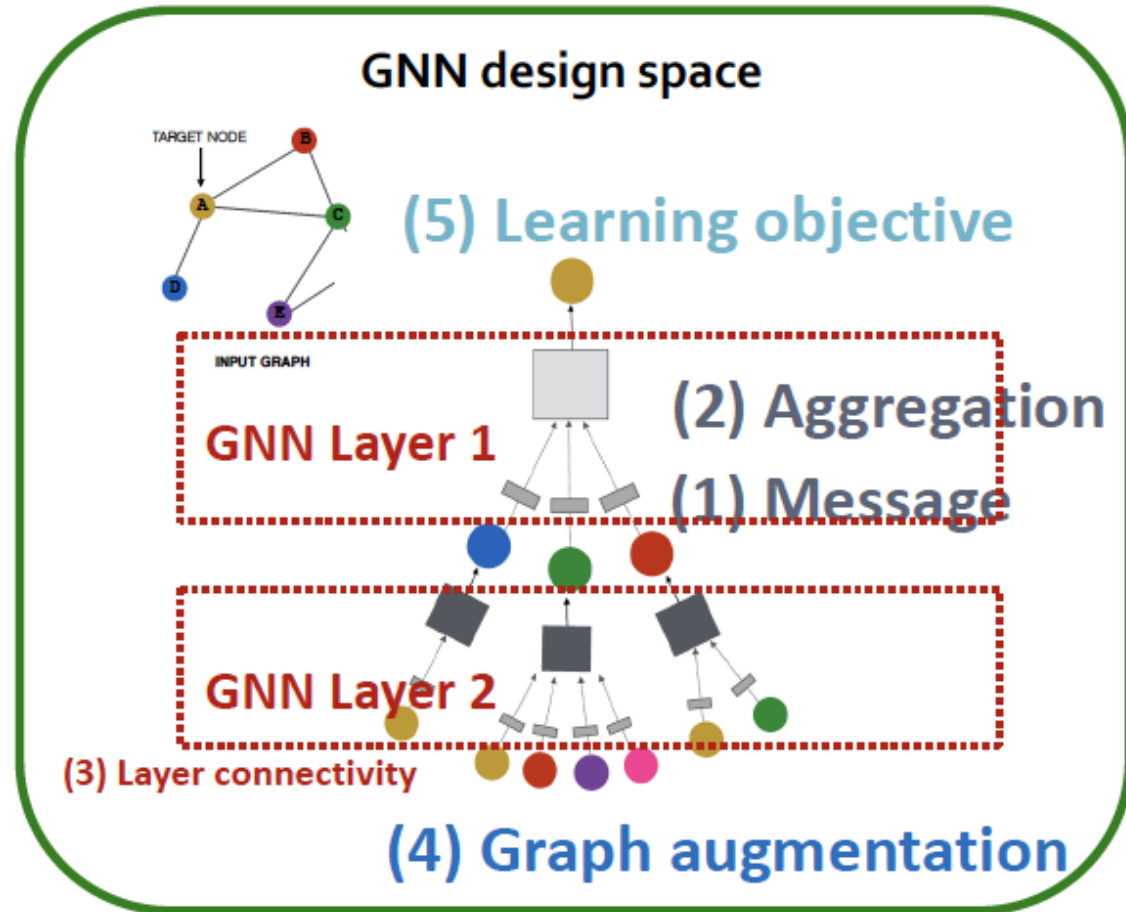
- Relative distances, or Laplacian eigenvectors
- Gives Transformer adjacency structure of graph

▪ (3) Modified Attention

- Reweight attention using edge features



Summary: Graph Transformer Design Space



Class Project

Class Projects: Components

- Project proposal (10%)
 - Deadline: today
- Project report (60%)
 - Deadline: July 17th, 2026.
- Project presentation (30%)
 - Saturday (July 18th) and Monday, (July 20th)

Class Projects: Project report

- Writing (40 points): 15-20 pages
 - Motivation & explanation of data/task (9 points)
 - Appropriateness & explanation of model(s) (9 points)
 - Insights + results (9 points)
 - Figures (9 points)
 - Code snippets (4 points)
- Submission:
 - Deadline: by **July 17th**.
 - Format: [NeurIPS2025 LaTeX style](#)
 - Submission link will be posted on Slack
- Colab (20 points)
 - Code: correctness, design (10 points)
 - Documentation: class/function descriptions, comments in code (10 points)

Class Projects: Project presentation

- Present at class with Q&A
- Time: 25 minutes